LAPORAN PRAKTIKUM

NATURAL LANGUAGE PROCESSOR



Annisa Almaghvirah

240306037

https://github.com/ANNISAALMAGHVIRAH

Program Studi Teknologi Informasi

Fakultas Dakwah dan Ilmu Komunikasi

Universitas Islam Negeri Mataram

2026

Modul	Modul 1 - Fundamental dan Text Processing
Topik	Eksplorasi Corpus
Repository	NATURAL-LANGUAGE-PROCESSING/Annisa Almaghvirah_240306037_Modul1 at main · ANNISAALMAGHVIRAH/NATURAL-LANGUAGE-PROCESSING
Tanggal	13 Oktober 2026

A. Tujuan

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Membaca Dataset CSV
2. Memahami Struktur Corpus
3. Mengetahui Jumlah Dokumen
4. Mengetahui Jumlah Kolom
5. Mengetahui Jumlah Data
6. Mengetahui Distribusi Data
7. Menghitung Jumlah Kalimat
8. Menghitung Jumlah Kata
9. Melakukan Eksplorasi Awal Dataset

B. Requiretment

1. Sistem Operasi yang digunakan : Windows 11 Home Single Language 64-bit (10.0, Build 26100)
2. Browser : Google Chrome Version 130.0.6723.117 (Official Build) (64-bit)
3. Tools yang digunakan : Visual Studio Code, Python , Jupyter Notebook, Virtual Environment (.venv), pandas, numpy, matplotlib, seaborn, nltk

C. Dasar Teori

1. Pengertian NLP

   Natural Language Processing (NLP) adalah cabang kecerdasan buatan yang memungkinkan komputer untuk memahami, mengolah, dan menghasilkan bahasa alami (bahasa manusia).

   Contoh aplikasi NLP:

   - Mesin penerjemah (Google Translate)
   - Asisten virtual (Siri, Google Assistant)
   - Deteksi spam email
   - Analisis sentimen ulasan produk
   - Chatbot layanan pelanggan
   - Ringkasan dokumen otomatis

2. NLP Klasik vs Modern

Era	Pendekatan	Contoh
Klasik	Berbasis aturan (rule-based)	Regex, grammar rules
Statistik	Probabilitas & frekuensi	N-gram, Naive Bayes
Machine Learning	Representasi fitur manual	SVM, Decision Tree
Deep Learning	Representasi otomatis	RNN, LSTM
Modern	Pretrained Transformer	BERT, GPT, IndoBERT

3. Level Pemrosesan Bahasa

   NLP memproses bahasa pada berbagai level:

   Lexical → Kata dan morfologi

   Syntactic → Struktur kalimat (grammar)

   Semantic → Makna kata dan kalimat

   Discourse → Hubungan antar kalimat

   Pragmatic → Konteks dan tujuan komunikasi

4. Apa Itu Corpus?

   Dalam NLP, corpus adalah kumpulan data bahasa yang digunakan untuk analisis atau pengembangan sistem NLP.

```
Dokumen 1
Dokumen 2
Dokumen 3
...
Dokumen N
```

Sebuah corpus dapat berupa:

- berita;
- review produk;
- komentar media sosial;
- tweet;
- dokumen akademik;
- percakapan;
- pengaduan masyarakat.

5. Dataset CSV

   Dataset yang digunakan mahasiswa minimal memiliki satu kolom teks.

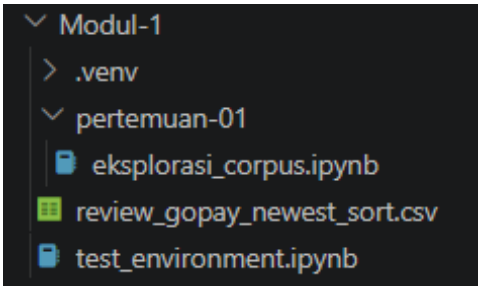
   Contoh:

id	text	label
1	Pelayanan hotel sangat bagus	Positif
2	Kamar hotel sangat kotor	Negatif
3	Lokasi hotel cukup strategis	positif

   Nama kolom setiap dataset dapat berbeda. Mahasiswa harus mengidentifikasi sendiri kolom yang berisi teks.

D. Implementasi

1. Membuat Folder Project



Folder paling luar (root) adalah `Modul-1`, yang di dalamnya langsung menampung folder lingkungan virtual `.venv`, sub-folder `pertemuan-01`, file dataset `review_gopay_newest_sort.csv`, dan file Jupyter Notebook `test_environment.ipynb`. Sementara itu, di dalam sub-folder `pertemuan-01` terdapat satu file tersendiri, yaitu `eksplorasi_corpus.ipynb`.

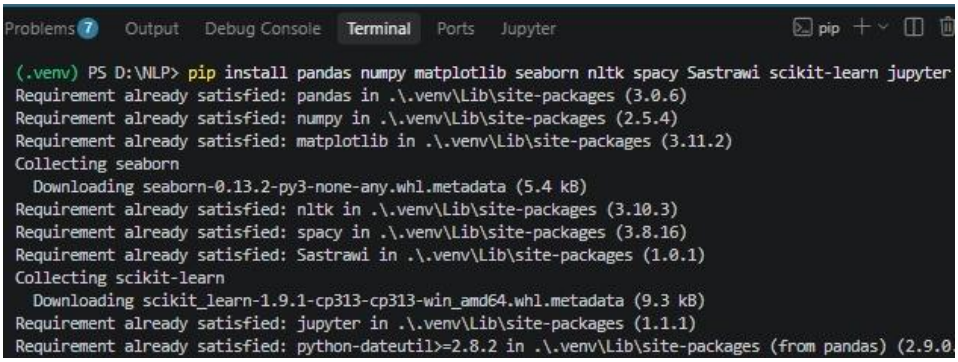
2. Membuat Virtual Environment

```
PS D:\NLP> python -m venv .venv
PS D:\NLP> Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
>> & .\.venv\Scripts\Activate.ps1
(.venv) PS D:\NLP> python --version
Python 3.13.15
(.venv) PS D:\NLP> python -c "import sys; print(sys.executable)"
D:\NLP\.venv\Scripts\python.exe
(.venv) PS D:\NLP>
```

Perintah tersebut diawali dengan membuat virtual environment baru bernama `.venv` melalui perintah `python -m venv .venv`. Setelah itu, kebijakan eksekusi PowerShell diizinkan untuk sesi aktif `(Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass)` agar script aktivasi `& .\.venv\Scripts\Activate.ps1` bisa dijalankan hingga muncul tanda `(.venv)`.

Terakhir, dilakukan dua tahap verifikasi untuk memastikan lingkungan virtual siap digunakan: mengecek versi Python yang terinstal (Python 3.13.15) dan memvalidasi jalur eksekusi Python yang sudah mengarah langsung ke folder virtual tersebut `(D:\NLP\.venv\Scripts\python.exe)`.

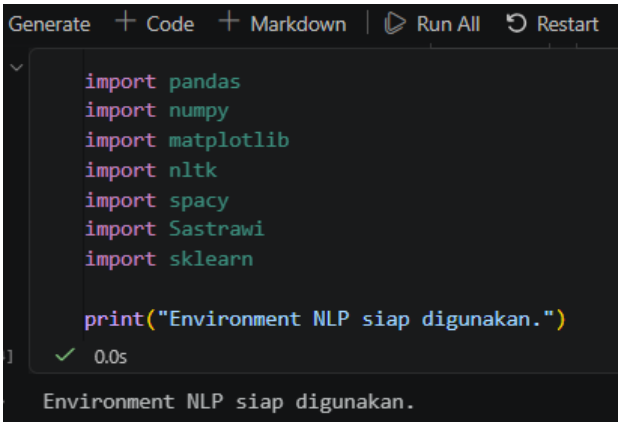
3. Install Library



Perintah `pip install` dijalankan di dalam lingkungan virtual .venv untuk menginstal beberapa library analisis data dan NLP sekaligus (`pandas`, `numpy`, `matplotlib`, `seaborn`, `nltk`, `spacy`, `Sastrawi`, `scikit-learn`, dan `jupyter`).

Hasilnya menunjukkan bahwa sebagian besar paket (seperti `pandas`, `numpy`, `matplotlib`, `nltk`, `spacy`, `Sastrawi`, dan `jupyter`) sudah terinstal sebelumnya (Requirement already satisfied). Sistem hanya mengunduh paket yang belum ada atau perlu diperbarui, yaitu seaborn dan scikit-learn beserta dependensinya.

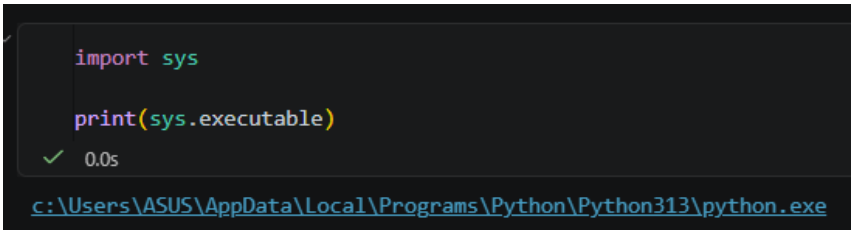
4. Memeriksa Instalasi



Pada file notebook test_environment.ipynb, dilakukan pengujian dengan memanggil seluruh library yang telah diinstal (`pandas`, `numpy`, `matplotlib`, `nltk`, `spacy`, `Sastrawi`, dan `sklearn`). Setelah semua perintah *import* berhasil dijalankan tanpa *error*, program mencetak output `"Environment NLP siap digunakan."` untuk mengonfirmasi bahwa seluruh pustaka telah terpasang dengan benar dan siap dipakai untuk proyek.

5. Memilih Kernel Jupyter

   Memeriksa Kernel yang Digunakan

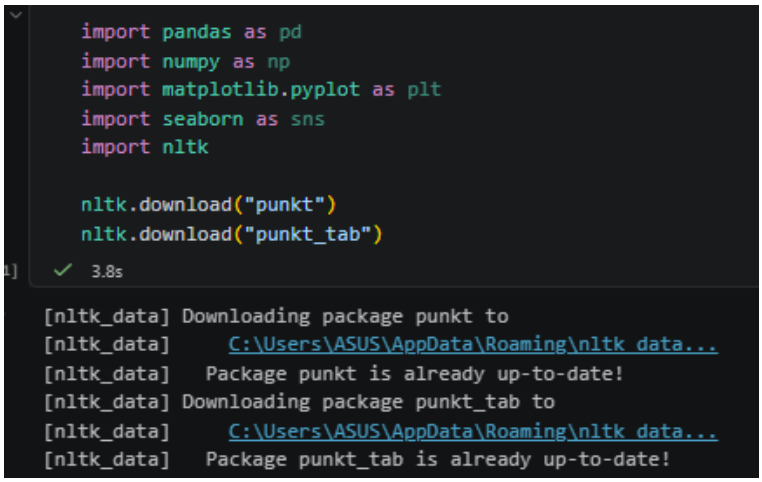


   Perintah `import sys` dan `print(sys.executable)` dijalankan untuk mengecek kernel Python yang sedang aktif di notebook. Output yang muncul menunjukkan bahwa kernel masih terhubung ke Python global sistem (`c:\Users\ASUS\...`), sehingga perlu diubah ke *virtual environment* (`.venv`) agar pustaka proyek terbaca.

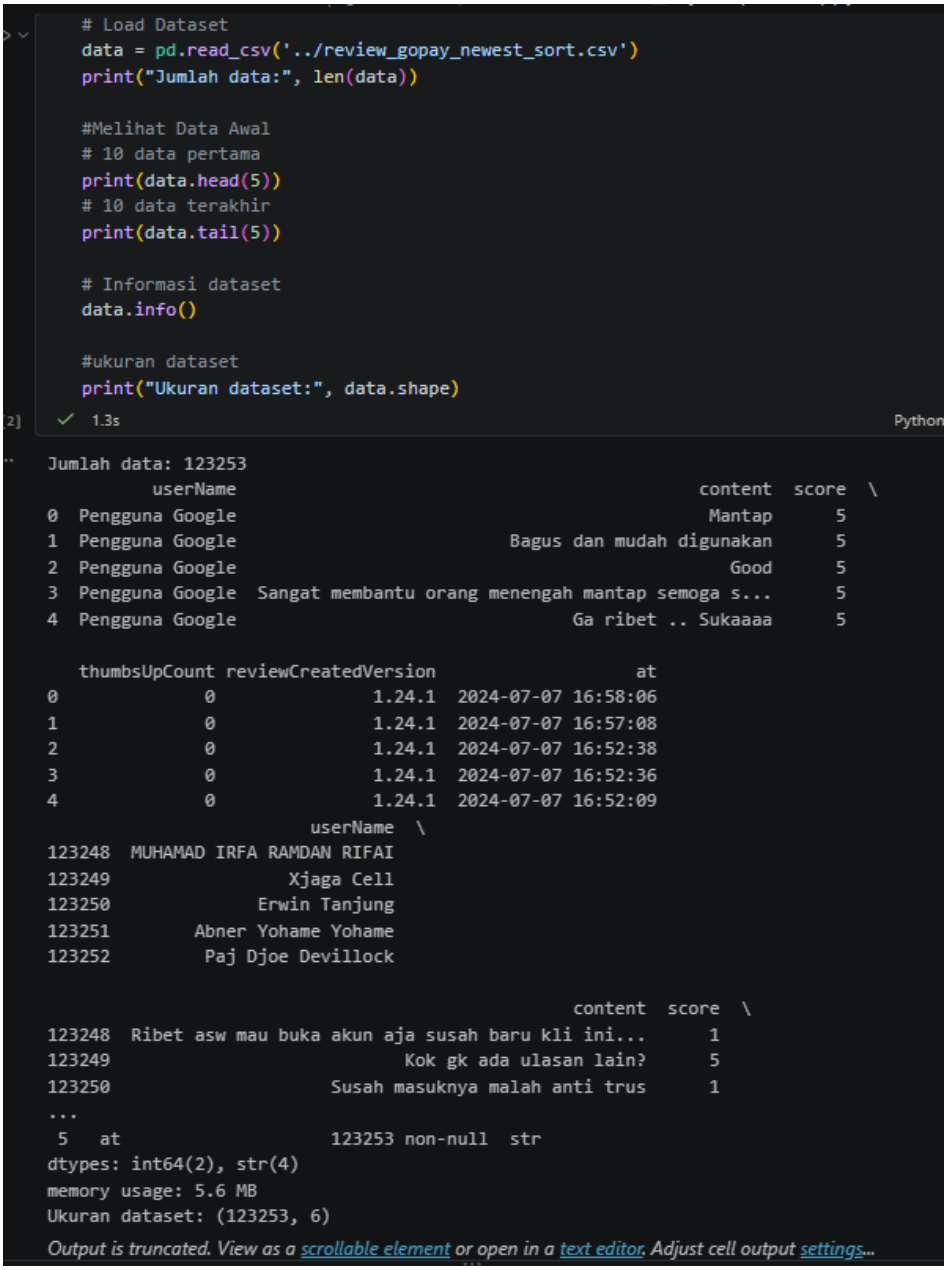
6. Membuat Notebook

   Buat file:

   `pertemuan-01/eksplorasi_corpus.ipynb`



   Perintah mengimpor beberapa library pemrosesan data dan visualisasi (`pandas`, `numpy`, `matplotlib`, `seaborn`, `nltk`) serta mengunduh modul pengolahan teks `punkt` dan `punkt_tab` menggunakan fungsi `nltk.download()`. Output menunjukkan bahwa kedua pustaka tokenisasi NLTK tersebut sudah terunduh dan siap digunakan (`Package is already up-to-date!`).



```python
# Load Dataset
data = pd.read_csv('../review_gopay_newest_sort.csv')
print("Jumlah data:", len(data))

#Melihat Data Awal
# 10 data pertama
print(data.head(5))
# 10 data terakhir
print(data.tail(5))

# Informasi dataset
data.info()

#ukuran dataset
print("Ukuran dataset:", data.shape)
```

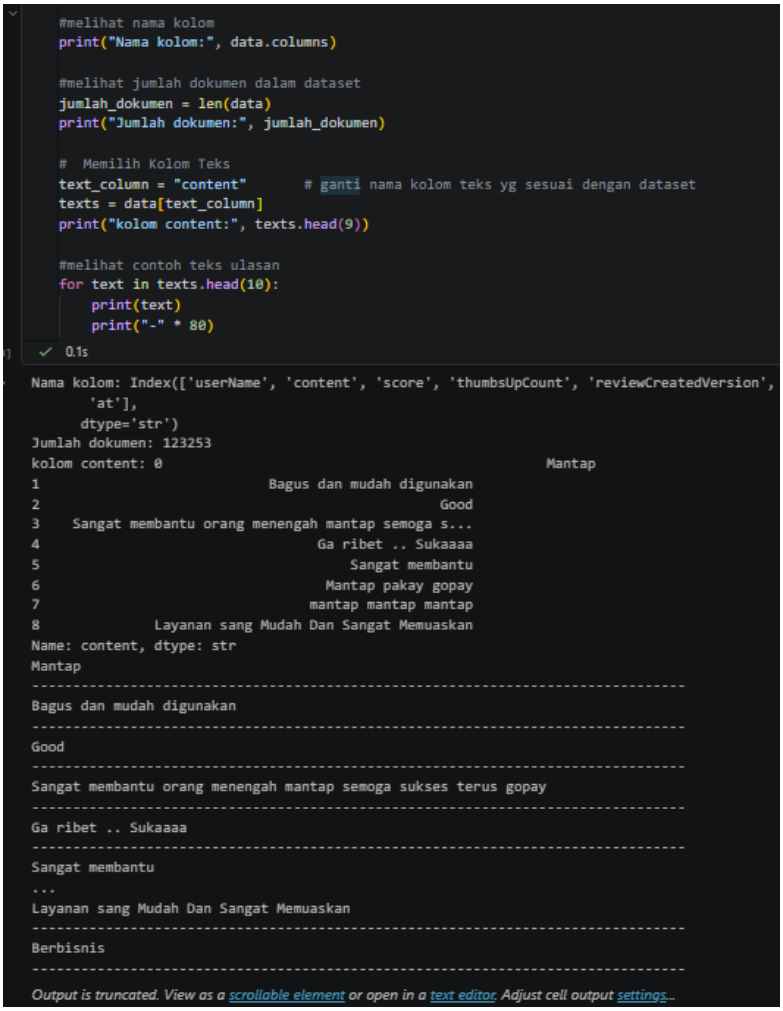
```
2]  ✓ 1.3s                                                    Python

Jumlah data: 123253
          userName                                              content  score  \
0  Pengguna Google                                               Mantap      5
1  Pengguna Google                                 Bagus dan mudah digunakan      5
2  Pengguna Google                                                 Good      5
3  Pengguna Google  Sangat membantu orang menengah mantap semoga s...      5
4  Pengguna Google                                 Ga ribet .. Sukaaaa      5

   thumbsUpCount reviewCreatedVersion                   at
0              0              1.24.1  2024-07-07 16:58:06
1              0              1.24.1  2024-07-07 16:57:08
2              0              1.24.1  2024-07-07 16:52:38
3              0              1.24.1  2024-07-07 16:52:36
4              0              1.24.1  2024-07-07 16:52:09
                        userName  \
123248  MUHAMAD IRFA RAMDAN RIFAI
123249                Xjaga Cell
123250             Erwin Tanjung
123251        Abner Yohame Yohame
123252         Paj Djoe Devillock

                                                content  score  \
123248  Ribet asw mau buka akun aja susah baru kli ini...      1
123249                           Kok gk ada ulasan lain?      5
123250                    Susah masuknya malah anti trus      1
...
 5   at                    123253 non-null  str
dtypes: int64(2), str(4)
memory usage: 5.6 MB
Ukuran dataset: (123253, 6)
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Proses memuat dataset `review_gopay_newest_sort.csv` dilakukan menggunakan fungsi `read_csv` dari pustaka Pandas untuk menampilkan ringkasan awal struktur data. Hasil eksekusi menunjukkan bahwa dataset ini terdiri dari 123.253 baris dan 6 kolom dengan tipe data gabungan antara numerik (`int64`) dan teks (`str`). Melalui fungsi `head(5)` dan `tail(5)`, kita dapat melihat sampel data awal dan akhir yang mencakup informasi pengguna (`userName`), teks ulasan (`content`), nilai rating (`score`), jumlah *thumbs up* (`thumbsUpCount`), versi aplikasi (`reviewCreatedVersion`), serta waktu ulasan dibuat (`at`). Seluruh kolom terisi penuh tanpa ada nilai yang hilang (*non-null*) dengan total penggunaan memori sekitar 5,6 MB.



```
#melihat nama kolom
print("Nama kolom:", data.columns)

#melihat jumlah dokumen dalam dataset
jumlah_dokumen = len(data)
print("Jumlah dokumen:", jumlah_dokumen)

#  Memilih Kolom Teks
text_column = "content"       # ganti nama kolom teks yg sesuai dengan dataset
texts = data[text_column]
print("kolom content:", texts.head(9))

#melihat contoh teks ulasan
for text in texts.head(10):
    print(text)
    print("-" * 80)
```

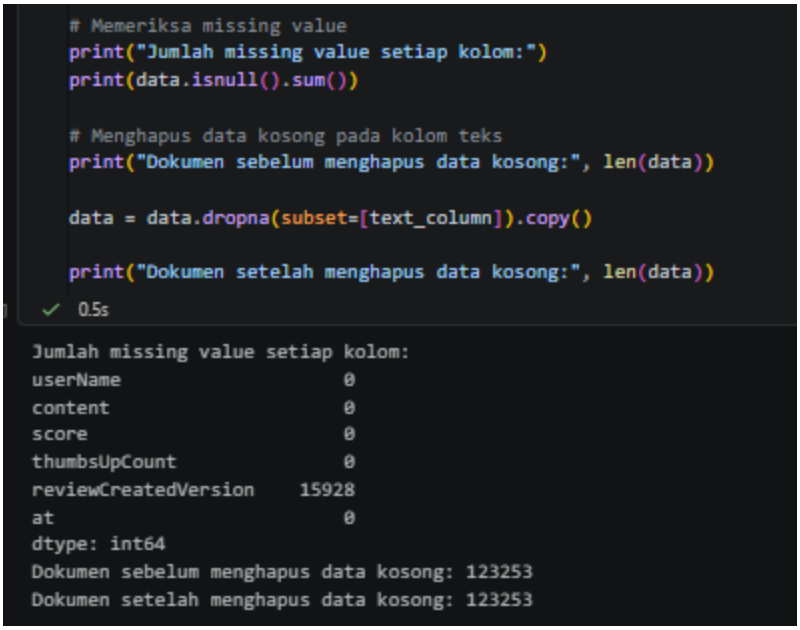
0.1s

```
Nama kolom: Index(['userName', 'content', 'score', 'thumbsUpCount', 'reviewCreatedVersion',
       'at'],
      dtype='str')
Jumlah dokumen: 123253
kolom content: 0                                                Mantap
1                          Bagus dan mudah digunakan
2                                               Good
3    Sangat membantu orang menengah mantap semoga s...
4                                Ga ribet .. Sukaaaa
5                                    Sangat membantu
6                                 Mantap pakay gopay
7                                mantap mantap mantap
8             Layanan sang Mudah Dan Sangat Memuaskan
Name: content, dtype: str
Mantap
--------------------------------------------------------------------------------
Bagus dan mudah digunakan
--------------------------------------------------------------------------------
Good
--------------------------------------------------------------------------------
Sangat membantu orang menengah mantap semoga sukses terus gopay
--------------------------------------------------------------------------------
Ga ribet .. Sukaaaa
--------------------------------------------------------------------------------
Sangat membantu
...
Layanan sang Mudah Dan Sangat Memuaskan
--------------------------------------------------------------------------------
Berbisnis
--------------------------------------------------------------------------------
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Perintah `print("Nama kolom:", data.columns)` mencetak seluruh daftar nama kolom yang ada pada DataFrame, lalu perintah `jumlah_dokumen = len(data)` menghitung total baris dataset yang kemudian dicetak melalui `print("Jumlah dokumen:", jumlah_dokumen)`.

Selanjutnya perintah `text_column = "content"` menetapkan nama kolom target yang berisi teks ulasan, diikuti perintah `texts = data[text_column]` yang mengambil seluruh isi kolom tersebut dan menyimpannya ke variabel texts.

Perintah `print("kolom content:", texts.head(9))` menampilkan 9 sampel baris pertama beserta indeksnya. Perintah `for text in texts.head(10):` melakukan perulangan pada 10 ulasan pertama untuk menjalankan perintah `print(text)` yang mencetak isi teks secara utuh, perintah `print("-" * 80)` yang mencetak garis pemisah sepanjang 80 karakter di setiap akhir ulasan agar tampilan data terstruktur dengan rapi.



```
# Memeriksa missing value
print("Jumlah missing value setiap kolom:")
print(data.isnull().sum())

# Menghapus data kosong pada kolom teks
print("Dokumen sebelum menghapus data kosong:", len(data))

data = data.dropna(subset=[text_column]).copy()

print("Dokumen setelah menghapus data kosong:", len(data))
```

✓ 0.5s

```
Jumlah missing value setiap kolom:
userName                  0
content                   0
score                     0
thumbsUpCount             0
reviewCreatedVersion    15928
at                        0
dtype: int64
Dokumen sebelum menghapus data kosong: 123253
Dokumen setelah menghapus data kosong: 123253
```

Perintah `print("Jumlah missing value setiap kolom:")` mencetak judul petunjuk, lalu diikuti perintah `print(data.isnull().sum())` untuk menghitung dan menampilkan total nilai kosong (*null/NaN*) pada tiap kolom, yang memperlihatkan adanya 15.928 data kosong di kolom `reviewCreatedVersion`.

Selanjutnya, perintah `print("Dokumen sebelum menghapus data kosong:", len(data))` mencetak jumlah dokumen awal yaitu 123.253. Perintah `data = data.dropna(subset=[text_column]).copy()` kemudian dijalankan untuk menghapus baris bernilai kosong khusus pada kolom teks target (`content`), dan ditutup dengan perintah `print("Dokumen setelah menghapus data kosong:", len(data))` untuk menampilkan jumlah akhir dokumen. Karena kolom `content` terisi penuh tanpa nilai kosong (bernilai `0`), total dokumen tidak berkurang dan tetap berjumlah 123.253.

```python
# Memeriksa duplikasi
data.duplicated().sum()
print("Jumlah dokumen duplikat:", data.duplicated().sum())

# Memeriksa duplikasi pada kolom teks
data[text_column].duplicated().sum()
print("Jumlah teks duplikat:",data[text_column].duplicated().sum())

# Menghapus teks duplikat
data = data.drop_duplicates(subset=[text_column])
print("Jumlah dokumen setelah menghapus duplikat:", len(data))

# Menghitung jumlah karakter
data["jumlah_karakter"] = data[text_column].astype(str).apply(len)
print(data["jumlah_karakter"].head(10))

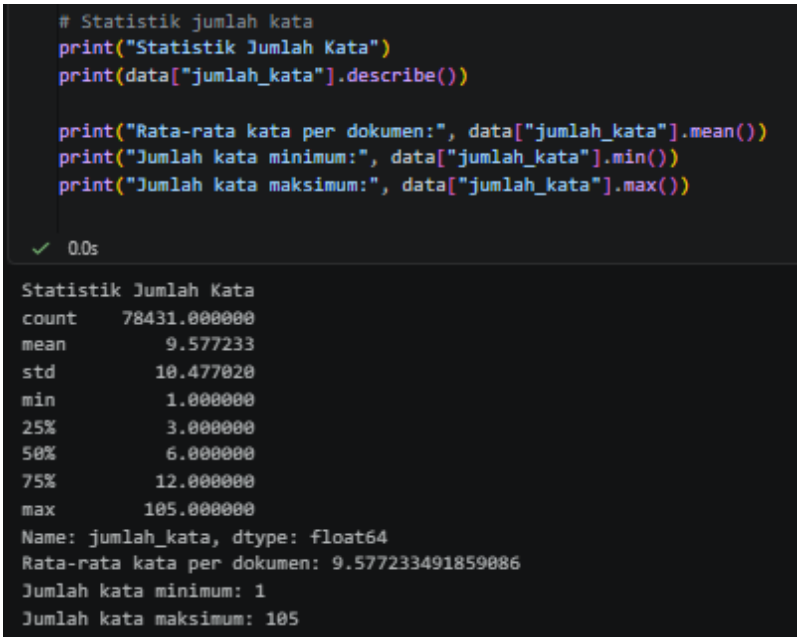
# Menghitung jumlah kata
data["jumlah_kata"] = data[text_column].astype(str).apply(
    lambda x: len(x.split())
)
print(data[[text_column, "jumlah_kata"]].head(10))
```

✓ 0.6s

```
Jumlah dokumen duplikat: 2
Jumlah teks duplikat: 44822
Jumlah dokumen setelah menghapus duplikat: 78431
0     6
1    25
2     4
3    63
4    19
5    15
6    18
7    20
8    39
9     9
Name: jumlah_karakter, dtype: int64
                                        content  jumlah_kata
0                                        Mantap            1
1                          Bagus dan mudah digunakan       4
2                                          Good            1
3  Sangat membantu orang menengah mantap semoga s...       9
4                               Ga ribet .. Sukaaaa        4
5                                   Sangat membantu        2
6                                Mantap pakay gopay        3
7                              mantap mantap mantap        3
8          Layanan sang Mudah Dan Sangat Memuaskan         6
9                                       Berbisnis          1
```

Perintah `data.duplicated().sum()` dan `print("Jumlah dokumen duplikat:", data.duplicated().sum())` digunakan untuk mengecek dan mencetak jumlah baris yang identik di seluruh kolom (ditemukan 2 duplikat). Perintah `data[text_column].duplicated().sum()` dan `print("Jumlah teks duplikat:", data[text_column].duplicated().sum())` menghitung serta mencetak duplikasi khusus pada kolom teks ulasan (ditemukan 44.822 teks duplikat). Perintah `data = data.drop_duplicates(subset=[text_column])` menghapus seluruh teks ulasan yang ganda, perintah `print("Jumlah dokumen setelah menghapus duplikat:", len(data))` yang menampilkan jumlah data unik tersisa yaitu 78.431 dokumen.

Perintah pembuatan kolom baru `data["jumlah_karakter"] = data[text_column].astype(str).apply(len)` untuk mengukur panjang teks berdasarkan jumlah karakter, lalu dicetak 10 sampel pertamanya melalui perintah `print(data["jumlah_karakter"].head(10))`. Bagian akhir diisi oleh perintah `data["jumlah_kata"] = data[text_column].astype(str).apply(lambda x: len(x.split()))` yang memotong teks berdasarkan spasi untuk menghitung total kata pada setiap ulasan. perintah `print(data[[text_column, "jumlah_kata"]].head(10))` untuk menampilkan 10 sampel teks ulasan berdampingan dengan hasil jumlah katanya.



```
# Statistik jumlah kata
print("Statistik Jumlah Kata")
print(data["jumlah_kata"].describe())

print("Rata-rata kata per dokumen:", data["jumlah_kata"].mean())
print("Jumlah kata minimum:", data["jumlah_kata"].min())
print("Jumlah kata maksimum:", data["jumlah_kata"].max())
```

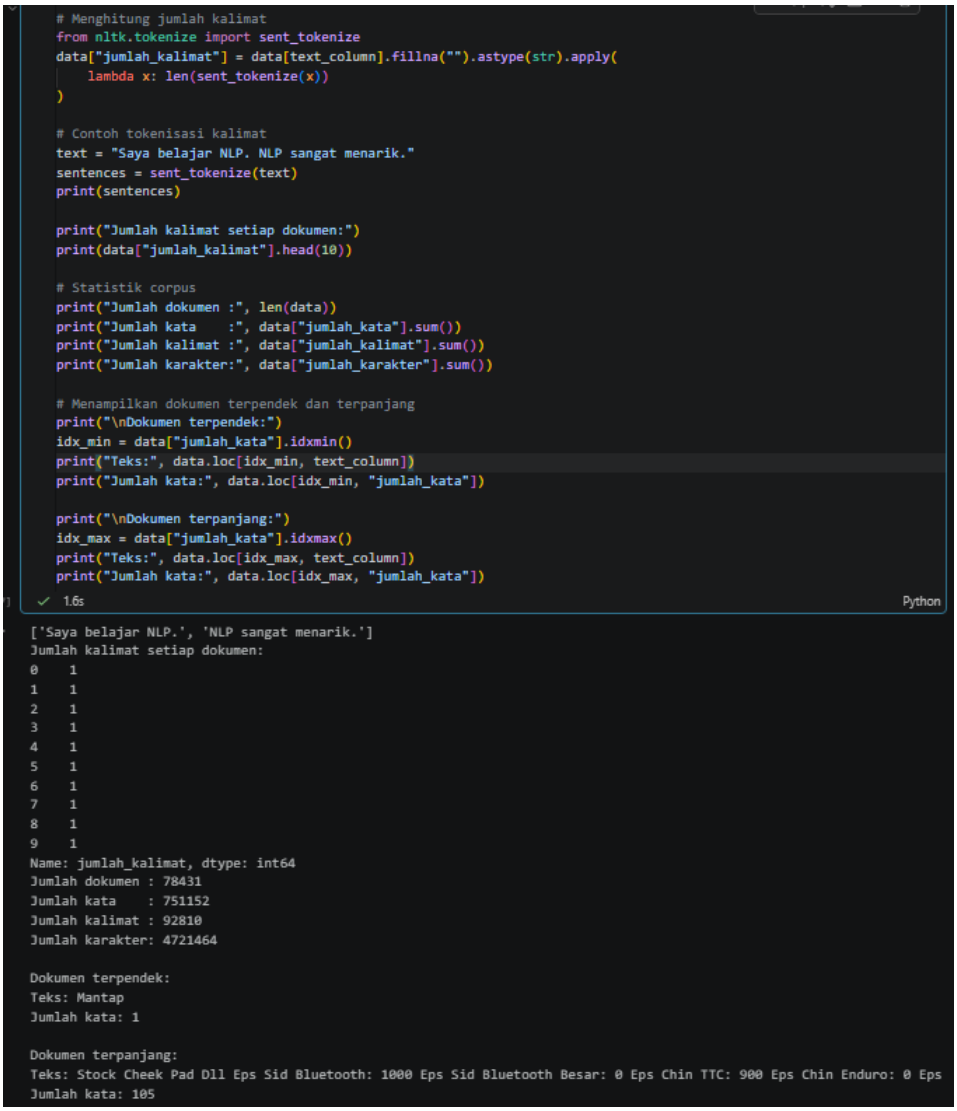
✓ 0.0s

```
Statistik Jumlah Kata
count    78431.000000
mean         9.577233
std         10.477020
min          1.000000
25%          3.000000
50%          6.000000
75%         12.000000
max        105.000000
Name: jumlah_kata, dtype: float64
Rata-rata kata per dokumen: 9.577233491859086
Jumlah kata minimum: 1
Jumlah kata maksimum: 105
```

Perintah `print("Statistik Jumlah Kata")` mencetak judul petunjuk, lalu diikuti perintah `print(data["jumlah_kata"].describe())` untuk menampilkan ringkasan statistik deskriptif seperti jumlah dokumen, rata-rata, standar deviasi, hingga nilai kuartil pada kolom `jumlah_kata`.

Selanjutnya, perintah `print("Rata-rata kata per dokumen:", data["jumlah_kata"].mean())` menghitung serta menampilkan nilai rata-rata kata sebesar 9,58. Perintah `print("Jumlah kata minimum:", data["jumlah_kata"].min())` menampilkan jumlah kata tersedikit yaitu 1 kata, dan ditutup oleh perintah `print("Jumlah kata maksimum:",`

`data["jumlah_kata"].max())` yang menampilkan jumlah kata terbanyak yaitu 105 kata dalam satu dokumen.



```
# Menghitung jumlah kalimat
from nltk.tokenize import sent_tokenize
data["jumlah_kalimat"] = data[text_column].fillna("").astype(str).apply(
    lambda x: len(sent_tokenize(x))
)

# Contoh tokenisasi kalimat
text = "Saya belajar NLP. NLP sangat menarik."
sentences = sent_tokenize(text)
print(sentences)

print("Jumlah kalimat setiap dokumen:")
print(data["jumlah_kalimat"].head(10))

# Statistik corpus
print("Jumlah dokumen :", len(data))
print("Jumlah kata    :", data["jumlah_kata"].sum())
print("Jumlah kalimat :", data["jumlah_kalimat"].sum())
print("Jumlah karakter:", data["jumlah_karakter"].sum())

# Menampilkan dokumen terpendek dan terpanjang
print("\nDokumen terpendek:")
idx_min = data["jumlah_kata"].idxmin()
print("Teks:", data.loc[idx_min, text_column])
print("Jumlah kata:", data.loc[idx_min, "jumlah_kata"])

print("\nDokumen terpanjang:")
idx_max = data["jumlah_kata"].idxmax()
print("Teks:", data.loc[idx_max, text_column])
print("Jumlah kata:", data.loc[idx_max, "jumlah_kata"])
```

✓ 1.6s Python

```
['Saya belajar NLP.', 'NLP sangat menarik.']
Jumlah kalimat setiap dokumen:
0    1
1    1
2    1
3    1
4    1
5    1
6    1
7    1
8    1
9    1
Name: jumlah_kalimat, dtype: int64
Jumlah dokumen : 78431
Jumlah kata    : 751152
Jumlah kalimat : 92810
Jumlah karakter: 4721464

Dokumen terpendek:
Teks: Mantap
Jumlah kata: 1

Dokumen terpanjang:
Teks: Stock Cheek Pad Dll Eps Sid Bluetooth: 1000 Eps Sid Bluetooth Besar: 0 Eps Chin TTC: 900 Eps Chin Enduro: 0 Eps
Jumlah kata: 105
```

Perintah `from nltk.tokenize import sent_tokenize` mengimpor fungsi pemotong kalimat, lalu perintah `data["jumlah_kalimat"] = data[text_column].fillna("").astype(str).apply(lambda x: len(sent_tokenize(x)))` membuat kolom baru untuk menghitung total kalimat pada tiap dokumen.

Selanjutnya, perintah contoh tokenisasi `text = "Saya belajar NLP. NLP sangat menarik.", sentences = sent_tokenize(text),` dan `print(sentences)` memperlihatkan cara kerja pemotongan teks menjadi daftar kalimat. Perintah `print("Jumlah kalimat setiap dokumen:")` dan `print(data["jumlah_kalimat"].head(10))` menampilkan 10

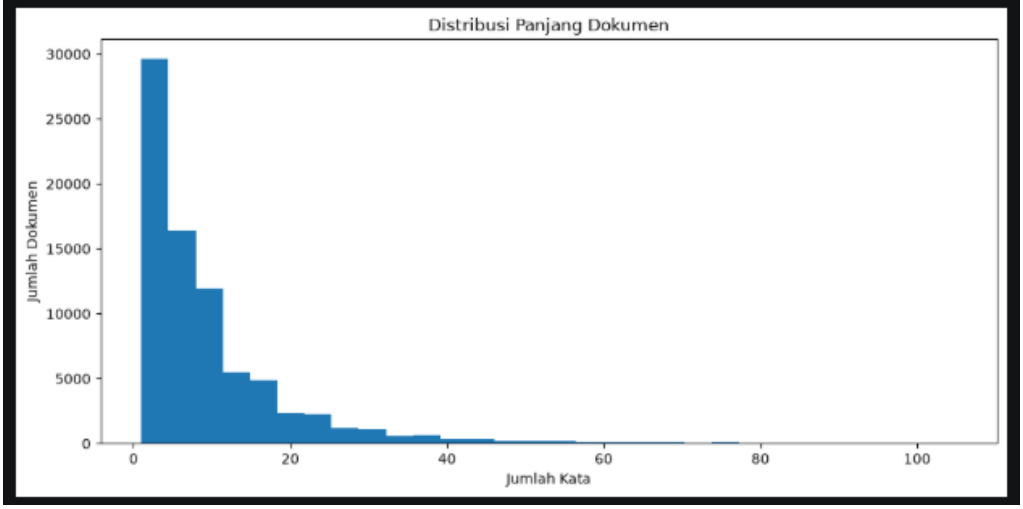
sampel hasil perhitungan jumlah kalimat awal. Tahap statistik korpus dijalankan oleh perintah `print("Jumlah dokumen :", len(data))`, `print("Jumlah kata :", data["jumlah_kata"].sum())`, `print("Jumlah kalimat :", data["jumlah_kalimat"].sum())`, serta `print("Jumlah karakter:", data["jumlah_karakter"].sum())` untuk mencetak total keseluruhan dokumen (78.431), kata (751.152), kalimat (92.810), dan karakter (4.721.464).

Rangkaian kode diakhiri dengan pencarian dokumen ekstrem: perintah `idx_min = data["jumlah_kata"].idxmin()` mengambil indeks ulasan terpendek untuk dicetak isi teks dan jumlah katanya (1 kata) via `print("Teks:", data.loc[idx_min, text_column])` dan `print("Jumlah kata:", data.loc[idx_min, "jumlah_kata"])`. Perintah `idx_max = data["jumlah_kata"].idxmax()` mengambil indeks ulasan terpanjang yang kemudian dicetak teks beserta jumlah katanya (105 kata) menggunakan `print("Teks:", data.loc[idx_max, text_column])` dan `print("Jumlah kata:", data.loc[idx_max, "jumlah_kata"])`.

```python
import pandas as pd
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))
plt.hist(data["jumlah_kata"], bins=30)
plt.xlabel("Jumlah Kata")
plt.ylabel("Jumlah Dokumen")
plt.title("Distribusi Panjang Dokumen")
plt.tight_layout()
plt.show()
```

0.5s



Perintah `import pandas as pd` dan `import matplotlib.pyplot as plt` mengimpor pustaka Pandas untuk manipulasi data serta Matplotlib untuk pembuatan visualisasi.

Selanjutnya, perintah `plt.figure(figsize=(10, 5))` mengatur ukuran kanvas grafik menjadi lebar 10 inci dan tinggi 5 inci. Perintah `plt.hist(data["jumlah_kata"], bins=30)` membuat grafik histogram dari kolom jumlah_kata dengan membaginya ke dalam 30 kelompok rentang data (*bins*).

Perintah `plt.xlabel("Jumlah Kata")` dan `plt.ylabel("Jumlah Dokumen")` memberi label keterangan pada sumbu horizontal (x) dan sumbu vertikal (y). Perintah `plt.title("Distribusi Panjang Dokumen")` untuk memberi judul utama grafik, `plt.tight_layout()` untuk menyesuaikan tata letak elemen agar tidak tumpang-tindih, serta `plt.show()` untuk menampilkan grafik histogram ke layar.

E. Tugas Analisis

Mahasiswa mampu menjawab:

1. Apa nama dataset?

   Dataset yang digunakan bernama `review_gopay_newest_sort`.
2. Dataset berasal dari mana?

   Dataset berasal dari Kaggle yang dibuat oleh Ardena Afif Pratama.
3. Berapa jumlah dokumen?

   123253
4. Berapa jumlah kolom?

   6 kolom
5. Apa nama kolom teks?

   Kolom yang digunakan untuk analisis teks adalah `content`.
6. Apakah terdapat missing value?

   Berdasarkan pemeriksaan missing value, terdapat 15.928 data kosong pada kolom reviewCreatedVersion. Sementara itu, kolom userName, content, score, thumbsUpCount, dan at tidak memiliki missing value.
7. Apakah terdapat data duplikat?

   Berdasarkan hasil pemeriksaan, duplikat Keseluruhan Baris (Dokumen): Terdapat 2 data duplikat di mana seluruh isi kolom pada baris tersebut sama persis. Duplikat Isi Teks (*Content*): Terdapat 44.822 teks duplikat pada kolom ulasan. Hal ini terjadi karena banyak pengguna memberikan ulasan singkat yang sama persis, seperti kata "Mantap", "Bagus", atau "Good".
8. Berapa rata-rata jumlah kata per dokumen?

   Rata-rata jumlah kata per dokumen adalah 9.577233491859086 kata.
9. Berapa dokumen terpendek?

   Dokumen terpendek memiliki 1 kata.
10. Berapa dokumen terpanjang?

    Dokumen terpanjang memiliki 105 kata.
11. Apakah dataset memiliki label?

    Dataset tidak memiliki label
12. Bagaimana distribusi labelnya?

Karena tidak terdapat kolom label, distribusi label tidak dapat dihitung. Analisis yang dilakukan berfokus pada karakteristik corpus, seperti jumlah dokumen, karakter, kata, kalimat, dan distribusi panjang komentar.

13. Apa karakteristik bahasa yang ditemukan?

    Berdasarkan eksplorasi dataset ulasan GoPay, karakteristik bahasa yang ditemukan adalah penggunaan bahasa yang relatif beragam, dengan ulasan yang memiliki panjang berbeda-beda. Sebagian besar ulasan cenderung pendek, sedangkan sebagian lainnya lebih panjang dan berisi informasi yang lebih banyak. Selain itu, dataset memiliki rating dari 1 sampai 5, dengan rating 5 sebagai rating yang paling dominan.

F. Pertanyaan Diskusi

1. Mengapa eksplorasi corpus penting sebelum memulai proses NLP?

   Jawaban:

   Eksplorasi corpus penting untuk memahami karakteristik dan kondisi dataset sebelum dilakukan pemrosesan teks. Melalui eksplorasi, kita dapat mengetahui jumlah dokumen, jumlah kata, struktur data, data kosong, dan duplikasi. Dengan begitu, kita dapat menentukan langkah preprocessing yang sesuai dan mengurangi kesalahan saat proses NLP.

2. Apa perbedaan dokumen, kalimat, kata, dan karakter dalam konteks NLP?

   Jawaban:

   Dokumen adalah satu unit teks yang dianalisis, misalnya satu ulasan pengguna aplikasi. Kalimat adalah bagian dari dokumen yang berisi satu rangkaian ungkapan, biasanya diakhiri tanda titik, tanda tanya, atau tanda seru. Kata adalah unit bahasa yang terbentuk dari rangkaian huruf dan dipisahkan oleh spasi atau aturan tokenisasi. Sementara itu, karakter adalah setiap simbol dalam teks, termasuk huruf, angka, spasi, dan tanda baca.

3. Apakah dataset yang seimbang (balanced) selalu lebih baik?

   Jawaban:

   Tidak selalu. Dataset yang seimbang berarti jumlah data pada setiap kelas relatif sama. Kondisi ini dapat membantu model mempelajari setiap kelas dengan lebih baik, terutama pada klasifikasi sentimen. Namun, dataset yang tidak seimbang juga dapat mencerminkan kondisi nyata. Oleh karena itu,

keseimbangan dataset perlu disesuaikan dengan tujuan penelitian dan jenis masalah yang diselesaikan.

4. Mengapa perlu memeriksa missing value dan duplikasi?

   Jawaban:

   Pemeriksaan missing value diperlukan untuk mengetahui adanya data kosong yang dapat mengganggu proses analisis atau menyebabkan kesalahan saat pemrosesan. Sementara itu, pemeriksaan duplikasi diperlukan untuk mengetahui apakah terdapat data yang tercatat lebih dari satu kali. Data kosong dan duplikat perlu ditangani sesuai kondisi dataset agar hasil analisis lebih akurat dan tidak biasa.

5. Apa yang dapat disimpulkan dari distribusi panjang dokumen?

   Jawaban:

   Distribusi panjang dokumen menunjukkan variasi jumlah kata atau karakter dalam setiap dokumen. Dari distribusi tersebut, kita dapat mengetahui apakah sebagian besar dokumen pendek, panjang, atau memiliki panjang yang beragam. Informasi ini dapat digunakan untuk menentukan langkah preprocessing, seperti penanganan dokumen yang terlalu pendek atau panjang, serta membantu menentukan batas panjang teks jika diperlukan dalam pemodelan NLP.

G. Kesimpulan

Kesimpulan yang dapat diperoleh dari praktikum ini adalah:

1. Eksplorasi Corpus: Praktikum berhasil melakukan analisis awal pada dataset review_gopay_newest_sort.csv untuk memahami struktur dan kualitas teks sebelum pemrosesan NLP.
2. Kondisi Data: Dataset awal berisi 123.253 dokumen. Setelah menghapus 44.822 ulasan duplikat (seperti kata "Mantap" atau "Bagus"), tersisa 78.431 dokumen unik.
3. Karakteristik Teks: Ulasan cenderung singkat dengan rata-rata 9,58 kata per dokumen (rentang 1–105 kata), total 751.152 kata, dan 92.810 kalimat.
4. Pentingnya Tahap Awal: Eksplorasi data dan penanganan duplikat penting dilakukan agar data teks bersih dan siap diproses ke tahap *preprocessing* maupun pemodelan NLP.

LAPORAN PRAKTIKUM

NATURAL LANGUAGE PROCESSOR



Annisa Almaghvirah

240306037

https://github.com/ANNISAALMAGHVIRAH

Program Studi Teknologi Informasi

Fakultas Dakwah dan Ilmu Komunikasi

Universitas Islam Negeri Mataram

2026

Modul	1 - Fundamental dan Text Processing
Topik	Tokenization
Repository	NATURAL-LANGUAGE-PROCESSING/Annisa Almaghvirah_240306037_Modul1 at main · ANNISAALMAGHVIRAH/NATURAL-LANGUAGE-PROCESSING
Tanggal	13 Oktober 2026

A. Tujuan

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Memahami konsep token
2. Melakukan sentence segmentation
3. Melakukan word tokenization
4. Melakukan character tokenization
5. Memahami subword tokenization
6. Membandingkan tokenizer yang berbeda

B. Requiretment

1. Sistem Operasi yang digunakan : Windows 11 Home Single Language 64-bit (10.0, Build 26100)
2. Browser : Google Chrome Version 130.0.6723.117 (Official Build) (64-bit)
3. Tools yang digunakan : Visual Studio Code, Python , Jupyter Notebook, Virtual Environment (.venv), pandas, numpy, matplotlib, seaborn, nltk

C. Dasar Teori

1. Apa Itu Token?

   Token adalah unit teks yang diproses oleh sistem NLP.

   Contoh kalimat:

   Saya belajar NLP

   dapat menjadi token:

   `["Saya", "belajar", "NLP"]`

   Token tidak selalu sama dengan kata.

2. Level Token

   Token dapat berada pada berbagai level:

   Character:

   `S, a, y, a`

   Word:

   Saya, belajar, NLP

   Subword:

   Contoh kata bermainnya dapat dipecah menjadi beberapa subword tergantung tokenizer yang digunakan. Subword tokenization digunakan oleh model Transformer modern (BPE, WordPiece, SentencePiece).

3. Jenis Tokenization

Jenis	Deskripsi	Contoh Input	Contoh Output
Word tokenization	Memecah teks menjadi kata	`"Saya belajar NLP."`	`["Saya", "belajar", "NLP", "."]`
Sentence tokenization	Memecah teks menjadi kalimat	`"Saya belajar. NLP menarik."`	`["Saya belajar.", "NLP menarik."]`
Character tokenization	Memecah teks menjadi karakter	`"NLP"`	`["N", "L", "P"]`
Subword tokenization	Memecah kata menjadi subword	`"bermainnya"`	bergantung tokenizer

4. Permasalahan Tokenisasi Bahasa Indonesia

   Bahasa Indonesia memiliki tantangan khusus dalam tokenisasi:

   - Variasi ejaan informal: tidak, nggak, gak, ga, tdk memiliki makna yang sama
   - Imbuhan kompleks: mempelajarinya, ketidakhadiran

- Campuran bahasa: campur kode Indonesia-Inggris, Indonesia-daerah
- Singkatan: yg, dgn, utk, krn
- Tanda baca ganda: !!!, ..., ???

Contoh:

Input : "pelayanannya bagus banget!!!"

Output: ["pelayanannya", "bagus", "banget", "!", "!", "!"]

D. Implementasi

1. Membuat Notebook

   Buat file

   ```
   Pertemuan-02/tokenization.ipynb
   ```

2. Import Library

```
#  Import Library
import pandas as pd
import nltk
from nltk.tokenize import sent_tokenize, word_tokenize

nltk.download("punkt")
nltk.download("punkt_tab")
```

✓ 0.0s

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\ASUS\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\ASUS\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

   Perintah `import pandas as pd` mengimpor pustaka Pandas untuk pengelolaan data tabel, lalu perintah `import nltk` mengimpor pustaka NLTK (Natural Language Toolkit) untuk pemrosesan bahasa alami.

   Selanjutnya, perintah `from nltk.tokenize import sent_tokenize`, word_tokenize mengimpor fungsi spesifik untuk memotong teks menjadi kalimat (`sent_tokenize`) dan kata (`word_tokenize`).

   Dan di akhiri dengan perintah `nltk.download("punkt")` serta `nltk.download("punkt_tab")` yang mengunduh model pemisahan teks bawaan NLTK agar fungsi tokenisasi dapat berjalan dengan akurat.

3. Load Dataset

```
# Load Dataset
data = pd.read_csv('../review_gopay_newest_sort.csv')
print("Jumlah data:", len(data))# load dataset

text_column = "content"  # sesuaikan dengan nama kolom
print("Kolom content:", data[text_column].head(9))
```

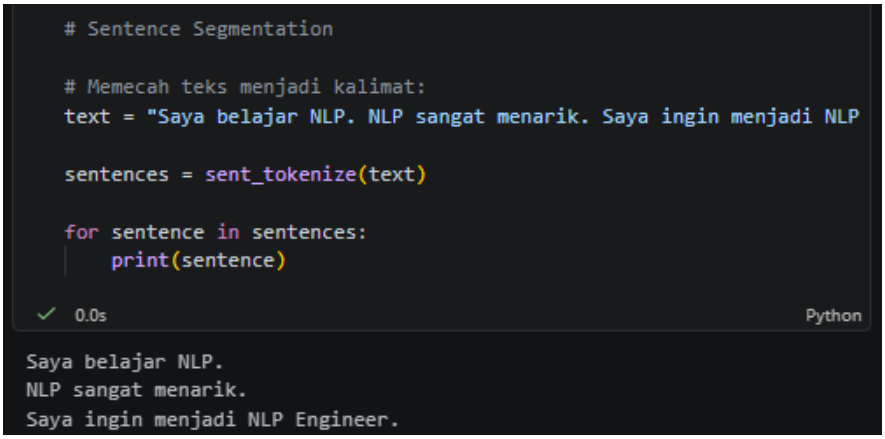
✓ 1.0s

```
Jumlah data: 123253
Kolom content: 0                                                Mantap
1                          Bagus dan mudah digunakan
2                                               Good
3    Sangat membantu orang menengah mantap semoga s...
4                                 Ga ribet .. Sukaaaa
5                                     Sangat membantu
6                                  Mantap pakay gopay
7                                 mantap mantap mantap
8             Layanan sang Mudah Dan Sangat Memuaskan
Name: content, dtype: str
```

Perintah `data = pd.read_csv('../review_gopay_newest_sort.csv')` berfungsi untuk membaca file CSV dan memuatnya ke dalam DataFrame Pandas. Kemudian, perintah `print("Jumlah data:", len(data))` menghitung serta menampilkan total baris data yang dimuat, yaitu sebanyak 123.253 baris.

Selanjutnya, perintah `text_column = "content"` menentukan nama kolom yang berisi teks ulasan. Dan perintah `print("Kolom content:", data[text_column].head(9))` untuk menampilkan 9 sampel ulasan pertama dari kolom tersebut.

4. Sentence Segmentation



Perintah `text = "Saya belajar NLP. NLP sangat menarik. Saya ingin menjadi NLP"` membuat variabel string berisi tiga kalimat sebagai data uji coba. Kemudian, perintah `sentences = sent_tokenize(text)` memotong string tersebut menjadi daftar kalimat (list of sentences) berdasarkan tanda baca titik menggunakan fungsi NLTK.

Selanjutnya, perintah `for sentence in sentences:` melakukan perulangan untuk mengambil setiap kalimat yang ada di dalam daftar dan perintah `print(sentence)` untuk mencetak masing-masing kalimat satu per satu ke layar secara terpisah.

5. Word Tokenization

```
# Word Tokenization
from nltk.tokenize import word_tokenize

text = "Saya belajar Natural Language Processing."

tokens = word_tokenize(text)

print(tokens)
```

✓ 0.0s

```
['Saya', 'belajar', 'Natural', 'Language', 'Processing', '.']
```

Perintah `from nltk.tokenize import word_tokenize` mengimpor fungsi pemotong kata dari pustaka NLTK. Selanjutnya, perintah `text = "Saya belajar Natural Language Processing."` menetapkan kalimat uji coba yang akan diproses.

Perintah `tokens = word_tokenize(text)` kemudian memecah string tersebut menjadi daftar kata (*tokens*) terpisah termasuk tanda baca. Dan perintah `print(tokens)` yang mencetak hasil pemotongannya ke dalam bentuk *list* `['Saya', 'belajar', 'Natural', 'Language', 'Processing', '.']`.

6. Character Tokenization

```
# Character Tokenization
text = "NLP"

characters = list(text)

print(characters)
```

✓ 0.0s

```
['N', 'L', 'P']
```

Perintah `text = "NLP"` menetapkan variabel string berisi kata uji coba. Selanjutnya, perintah `characters = list(text)` memotong string tersebut menjadi elemen huruf per huruf (*character tokenization*) dan menyimpannya ke dalam bentuk *list*. Ditutup dengan perintah `print(characters)` yang mencetak daftar karakter tersebut ke layar yaitu `['N', 'L', 'P']`.

7. Tokenisasi Dataset

```
# Tokenisasi Dataset

# Menerapkan word tokenization ke seluruh dataset:
data["tokens"] = data[text_column].astype(str).apply(word_tokenize)

data[[text_column, "tokens"]].head()

# Menghitung jumlah token per dokumen:
data["jumlah_token"] = data["tokens"].apply(len)
data["jumlah_token"].describe()
```

✓ 14.6s

```
count    123253.000000
mean          7.286995
std          10.540843
min           1.000000
25%           1.000000
50%           3.000000
75%           9.000000
max         493.000000
Name: jumlah_token, dtype: float64
```

Perintah `data["tokens"] = data[text_column].astype(str).apply(word_tokenize)` menerapkan fungsi *word tokenization* ke seluruh teks ulasan pada kolom target untuk mengubah setiap kalimat menjadi daftar kata/token. Kemudian, perintah `data[[text_column, "tokens"]].head()` menampilkan sampel teks awal beserta daftar token yang berhasil dipisah. Selanjutnya, perintah `data["jumlah_token"] = data["tokens"].apply(len)` menghitung total token yang ada di dalam tiap dokumen. Dan perintah `data["jumlah_token"].describe()` yang menampilkan ringkasan statistik deskriptif dari kolom `jumlah_token`, seperti total dokumen (123.253), rata-rata (7,28 token), nilai minimum (1 token), hingga nilai maksimum (493 token).

8. Membandingkan Kata dan Token

```
# Membandingkan Kata dan Token

# Hitung jumlah kata dengan split sederhana:
data["jumlah_kata"] = data[text_column].astype(str).apply(
    lambda x: len(x.split())
)

# Bandingkan:
data[["jumlah_kata", "jumlah_token"]].describe()
```

✓ 0.2s

	jumlah_kata	jumlah_token
count	123253.000000	123253.000000
mean	6.615368	7.286995
std	9.240311	10.540843
min	1.000000	1.000000
25%	1.000000	1.000000
50%	3.000000	3.000000
75%	8.000000	9.000000
max	105.000000	493.000000

Perintah `data["jumlah_kata"] = data[text_column].astype(str).apply(lambda x: len(x.split()))` memisahkan teks berdasarkan spasi untuk menghitung jumlah kata sederhana. Perintah `data[["jumlah_kata", "jumlah_token"]].describe()` membandingkan statistik dasar antara pemisahan spasi biasa dengan tokenisasi NLTK. Perbedaan terlihat pada nilai rata-rata (`6,61` kata vs `7,29` token) dan maksimum (`105` kata vs `493` token) karena tokenisasi NLTK memisahkan tanda baca dan simbol menjadi token tersendiri.

9. Eksperimen Tokenisasi

```
# Eksperimen Tokenisasi
texts = [
    "Saya suka produk ini.",
    "Bagus banget!!!",
    "Tidak bagus...",
    "Harga Rp50.000.",
    "email@example.com",
    "https://example.com",
    "yg bagus bgt sih",
    "aku udah beli nih kak"
]

for text in texts:
    print("TEXT :", text)
    print("TOKEN:", word_tokenize(text))
    print()
```

✓ 0.0s

```
TEXT : Saya suka produk ini.
TOKEN: ['Saya', 'suka', 'produk', 'ini', '.']

TEXT : Bagus banget!!!
TOKEN: ['Bagus', 'banget', '!', '!', '!']

TEXT : Tidak bagus...
TOKEN: ['Tidak', 'bagus', '...']

TEXT : Harga Rp50.000.
TOKEN: ['Harga', 'Rp50.000', '.']

TEXT : email@example.com
TOKEN: ['email', '@', 'example.com']

TEXT : https://example.com
TOKEN: ['https', ':', '//example.com']

TEXT : yg bagus bgt sih
TOKEN: ['yg', 'bagus', 'bgt', 'sih']

TEXT : aku udah beli nih kak
TOKEN: ['aku', 'udah', 'beli', 'nih', 'kak']
```

Perintah `texts = [...]` mendefinisikan *list* berisi beberapa contoh teks dengan variasi pola (seperti tanda baca berulang, nominal uang, email, URL,

hingga kata gaul/singkatan) untuk diuji coba. Kemudian, perintah `for text in texts:` melakukan perulangan pada setiap string. Di dalam perulangan, perintah `print("TEXT :", text)` menampilkan teks asli, `print("TOKEN:", word_tokenize(text))` memotong teks tersebut menjadi token menggunakan NLTK, dan `print()` memberi baris kosong antar-contoh. Hasil eksekusi memperlihatkan bagaimana `word_tokenize` memisahkan tanda baca seperti `!`, `@`, `:`, dan `/` menjadi token terpisah, serta mempertahankan singkatan atau kata biasa.

10. Perbandingan Tokenizer

```python
# Perbandingan Tokenizer
import spacy
import nltk
import pandas as pd
from nltk.tokenize import word_tokenize

nltk.download("punkt")
nltk.download("punkt_tab")

# Contoh teks
teks_list = [
    "Saya belajar NLP.",
    "Bagus banget!!!",
    "Harga Rp50.000"
]

# Membandingkan hasil tokenisasi
hasil = []

for teks in teks_list:
    nltk_tokens = word_tokenize(teks)
    split_tokens = teks.split()

    hasil.append({
        "Teks": teks,
        "NLTK word_tokenize": str(nltk_tokens),
        "split()": str(split_tokens)
    })

tabel = pd.DataFrame(hasil)
print(tabel.to_string(index=False))
```

✓ 3.3s

```
           Teks                NLTK word_tokenize                    split()
Saya belajar NLP.    ['Saya', 'belajar', 'NLP', '.'] ['Saya', 'belajar', 'NLP.']
  Bagus banget!!! ['Bagus', 'banget', '!', '!', '!']     ['Bagus', 'banget!!!']
   Harga Rp50.000            ['Harga', 'Rp50.000']      ['Harga', 'Rp50.000']
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\ASUS\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\ASUS\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

Perintah `import spacy`, `import nltk`, `import pandas as pd`, dan `from nltk.tokenize import word_tokenize` mengimpor pustaka untuk pemrosesan teks dan pembuatan tabel DataFrame. Perintah `nltk.download("punkt")` serta `nltk.download("punkt_tab")` mengunduh paket data pendukung tokenisasi NLTK.

Selanjutnya, `teks_list = [...]` menyiapkan sampel teks pengujian, lalu dilakukan perulangan `for teks in teks_list:` untuk memproses setiap kalimat menggunakan dua metode berbeda: `word_tokenize(teks)` dari NLTK dan metode bawaan `teks.split()`.

Hasil pemotongan token disimpan ke dalam daftar pustaka *dictionary* melalui `hasil.append({...})`. Kode diakhiri dengan `tabel = pd.DataFrame(hasil)` dan `print(tabel.to_string(index=False))` untuk menyajikan hasil perbandingan dalam bentuk tabel tanpa indeks baris.

Hasil output menunjukkan perbedaan utama: `word_tokenize` memisahkan tanda baca seperti titik `.` dan tanda seru `!` menjadi token terpisah, sedangkan `split()` hanya memotong teks berdasarkan spasi sehingga tanda baca tetap menempel pada kata.

E. Tugas Analisis

Jawab pertanyaan berikut berdasarkan eksperimen:

1. Apakah setiap kata selalu menghasilkan satu token?

   Jawaban:

   Tidak. Satu kata tidak selalu menghasilkan satu token karena hasil tokenisasi bergantung pada metode yang digunakan. Pada split(), teks biasanya dipisahkan berdasarkan spasi, sedangkan NLTK word_tokenize dapat memisahkan tanda baca menjadi token tersendiri. Pada subword tokenizer, satu kata juga dapat dipecah menjadi beberapa bagian.

2. Bagaimana tokenizer menangani tanda baca?

   Jawaban:

   Setiap tokenizer menangani tanda baca dengan cara berbeda. NLTK word_tokenize dapat memisahkan tanda baca dari kata, misalnya kalimat “Saya belajar NLP.” menjadi ['Saya', 'belajar', 'NLP', '.']. Sementara itu, split() menghasilkan ['Saya', 'belajar', 'NLP.'] karena titik masih menempel pada kata.

3. Bagaimana tokenizer menangani angka (contoh: Rp50.000)?

   Jawaban:

   Pada teks “Harga Rp50.000”, NLTK word_tokenize dan split() umumnya menghasilkan ['Harga', 'Rp50.000']. Artinya, Rp50.000 tetap menjadi satu token pada kedua metode tersebut. Namun, hasilnya dapat berbeda jika format angka mengandung spasi atau tanda baca lain.

4. Bagaimana tokenizer menangani URL?

   Jawaban:

   Tokenizer sederhana seperti split() biasanya mempertahankan URL sebagai satu bagian selama tidak ada spasi di dalamnya. NLTK juga dapat mempertahankan sebagian besar URL sebagai satu token, tetapi tanda baca yang menempel di akhir URL dapat dipisahkan atau ikut terbaca sebagai bagian URL, tergantung formatnya. Oleh karena itu, URL perlu diperiksa dan ditangani secara khusus saat preprocessing teks.

5. Bagaimana tokenizer menangani singkatan informal (yg, dgn, kak)?

   Jawaban:

   Tokenizer umumnya mengenali yg, dgn, dan kak sebagai token berdasarkan pemisahan teks, tetapi tidak otomatis mengubah yg menjadi “yang” atau dgn

menjadi “dengan”. Untuk analisis teks Bahasa Indonesia, singkatan informal dapat dinormalisasi menggunakan kamus kata tidak baku agar maknanya lebih konsisten.

6. Apa masalah tokenisasi yang spesifik untuk Bahasa Indonesia?

   Jawaban:

   Beberapa masalah tokenisasi dalam Bahasa Indonesia adalah banyaknya singkatan informal, variasi ejaan, penggunaan bahasa gaul, kata berimbuhan, pengulangan kata, dan campuran bahasa Indonesia dengan bahasa daerah atau bahasa Inggris. Contohnya, kata “memakannya” memiliki imbuhan yang perlu diperhatikan ketika teks dianalisis lebih lanjut. Selain itu, kata seperti “gak”, “nggak”, dan “enggak” dapat memiliki makna serupa tetapi bentuk penulisannya berbeda.

7. Mengapa subword tokenizer diperlukan pada model Transformer?

   Jawaban:

   Subword tokenizer diperlukan karena memungkinkan kata dipecah menjadi bagian yang lebih kecil apabila kata tersebut tidak ditemukan sebagai satu kesatuan dalam kosakata model. Metode ini membantu model menangani kata baru, kata langka, variasi imbuhan, dan istilah yang belum pernah ditemui. Dengan demikian, model Transformer tidak harus menyimpan setiap kemungkinan kata sebagai satu token tersendiri dan dapat memproses teks dengan kosakata yang lebih efisien.

F. Pertanyaan Diskusi

1. Apakah tokenizer yang baik untuk Bahasa Inggris otomatis baik untuk Bahasa Indonesia?

   Jawaban:

   Tidak. Tokenizer yang baik untuk Bahasa Inggris belum tentu memberikan hasil yang optimal untuk Bahasa Indonesia karena kedua bahasa memiliki struktur kata, imbuhan, dan kebiasaan penulisan yang berbeda. Bahasa Indonesia memiliki banyak imbuhan, singkatan informal, dan variasi bahasa sehari-hari. Oleh karena itu, tokenizer perlu disesuaikan dengan karakteristik bahasa yang digunakan.

2. Apa yang terjadi jika teks tidak di-tokenize terlebih dahulu?

   Jawaban:

   Jika teks tidak melalui proses tokenisasi, sistem NLP akan kesulitan memisahkan teks menjadi unit-unit yang dapat diproses, seperti kata atau subkata. Akibatnya, proses analisis teks dan pengolahan bahasa oleh model dapat menjadi kurang tepat. Namun, beberapa model memiliki tokenizer bawaan yang otomatis melakukan proses ini sebelum teks diproses lebih lanjut.

3. Mengapa tanda baca bisa menjadi token tersendiri?

   Jawaban:

   Tanda baca dapat menjadi token tersendiri karena memiliki fungsi untuk menunjukkan akhir kalimat, pertanyaan, penekanan, atau struktur suatu teks. Contohnya, kalimat “Bagus banget!!!” dapat dipisahkan menjadi ['Bagus', 'banget', '!', '!', '!'] oleh NLTK word_tokenize. Pemisahan ini membantu sistem mengenali struktur dan karakteristik teks dengan lebih jelas.

4. Apa hubungan tokenization dengan vocabulary dalam model NLP?

   Jawaban:

   Tokenisasi berhubungan langsung dengan vocabulary karena hasil tokenisasi akan dicocokkan dengan daftar token yang dikenal oleh model. Vocabulary berisi token-token yang memiliki representasi atau ID tertentu. Jika suatu kata tidak tersedia sebagai satu token dalam vocabulary, beberapa tokenizer dapat membaginya menjadi token yang lebih kecil agar tetap dapat diproses oleh model.

5. Mengapa model Transformer menggunakan subword tokenization?

   Jawaban:

   Model Transformer menggunakan subword tokenization untuk menangani kata yang jarang muncul, kata baru, dan variasi bentuk kata. Dengan metode ini, satu kata dapat dipecah menjadi beberapa bagian yang lebih kecil. Cara tersebut membantu mengurangi masalah kata yang tidak dikenal (out-of-vocabulary) dan membuat penggunaan vocabulary lebih efisien. Misalnya, kata berimbuhan dapat diwakili oleh beberapa subkata tanpa harus menyimpan setiap bentuk kata sebagai token yang berbeda.

G. Kesimpulan

Kesimpulan yang dapat diperoleh dari praktikum ini adalah:

1. Tingkat Tokenisasi: Praktikum berhasil menerapkan pemisahan teks pada tingkat kalimat (*sentence*), kata (*word*), dan huruf (*character*).
2. NLTK vs .split(): Fungsi .split() hanya memisahkan teks berdasarkan spasi, sedangkan word_tokenize NLTK memisahkan tanda baca menjadi token tersendiri (membuat rata-rata token naik dari 6,61 menjadi 7,29 per dokumen).
3. Tantangan Bahasa Indonesia: Tokenizer biasa belum bisa menormalisasi kata gaul/singkatan (yg, bgt) dan tanda baca berulang (!!!).
4. Solusi Subword: Model modern (Transformer) menggunakan *subword tokenization* untuk menangani kata berimbuhan dan kata baru (*Out-of-Vocabulary*).

LAPORAN PRAKTIKUM

NATURAL LANGUAGE PROCESSOR



Annisa Almaghvirah

240306037

https://github.com/ANNISAALMAGHVIRAH

Program Studi Teknologi Informasi

Fakultas Dakwah dan Ilmu Komunikasi

Universitas Islam Negeri Mataram

2026

Modul	Modul 1 - Fundamental dan Text Processing
Topik	Text Normalization
Repository	NATURAL-LANGUAGE-PROCESSING/Annisa Almaghvirah_240306037_Modul1 at main · ANNISAALMAGHVIRAH/NATURAL-LANGUAGE-PROCESSING
Tanggal	13 Oktober 2026

A. Tujuan

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Melakukan case folding
2. Membersihkan teks (url, mention, hashtag, angka, punctuation)
3. Menangani slang words
4. Melakukan stopword removal bahasa indonesia
5. Melakukan stemming bahasa indonesia dengan sastrawi
6. Memahami perbedaan stemming dan lemmatization
7. Membangun preprocessing pipeline lengkap

B. Requiretment

1. Sistem Operasi yang digunakan : Windows 11 Home Single Language 64-bit (10.0, Build 26100)
2. Browser : Google Chrome Version 130.0.6723.117 (Official Build) (64-bit)
3. Tools yang digunakan : Visual Studio Code, Python , Jupyter Notebook, Virtual Environment (.venv), pandas, numpy, matplotlib, seaborn, nltk

C. Dasar Teori

1. Mengapa Normalisasi Diperlukan?

   Data teks dunia nyata biasanya tidak bersih:

   `Input: "PELAYANANNYA BAGUUUS BANGET!!! 😍 https://toko.com"`

   Sistem NLP perlu menentukan informasi mana yang penting dan mana yang dapat dihilangkan sebelum diproses lebih lanjut.

   Masalah umum pada data teks:

   - Penggunaan huruf kapital yang tidak konsisten
   - Tanda baca berlebihan
   - URL, mention, hashtag yang tidak relevan
   - Angka dalam berbagai format
   - Slang dan singkatan informal
   - Typographical error

2. Tahapan Normalisasi

```
Raw Text
   ↓
Case Folding (lowercase)
   ↓
Cleaning (URL, mention, hashtag, angka)
   ↓
Punctuation Removal
   ↓
Whitespace Cleaning
   ↓
Tokenization
   ↓
Stopword Removal
   ↓
Stemming
   ↓
Processed Text
```

3. Stopword

   Stopword adalah kata yang sering muncul dan dalam konteks tertentu dianggap kurang informatif.

   Contoh stopword Bahasa Indonesia:

   ```
   yang, dan, di, ke, dari, untuk, dengan, adalah, ini, itu
   ```

   Perhatian penting:

   Stopword tidak selalu harus dihapus.

   Contoh pada sentiment analysis:

   ```
   Input : "tidak bagus"
   Setelah hapus "tidak" → "bagus"
   ```

   Makna berubah secara drastis. Keputusan menghapus stopword harus disesuaikan dengan task NLP.

4. Stemming vs Lemmatization

Aspek	Stemming	Lemmatization
Pendekatan	Berbasis aturan pemotongan	Berbasis analisis linguistik
Kompleksitas	Lebih sederhana	Lebih kompleks
Kecepatan	Cepat	Relatif lebih lambat
Hasil	Stem (mungkin bukan kata valid)	Lemma (kata dasar yang valid)
Contoh (EN)	`running` → `run`	`better` → `good`
Untuk Bahasa Indonesia	Sastrawi (stemmer)	Terbatas, umumnya menggunakan stemmer

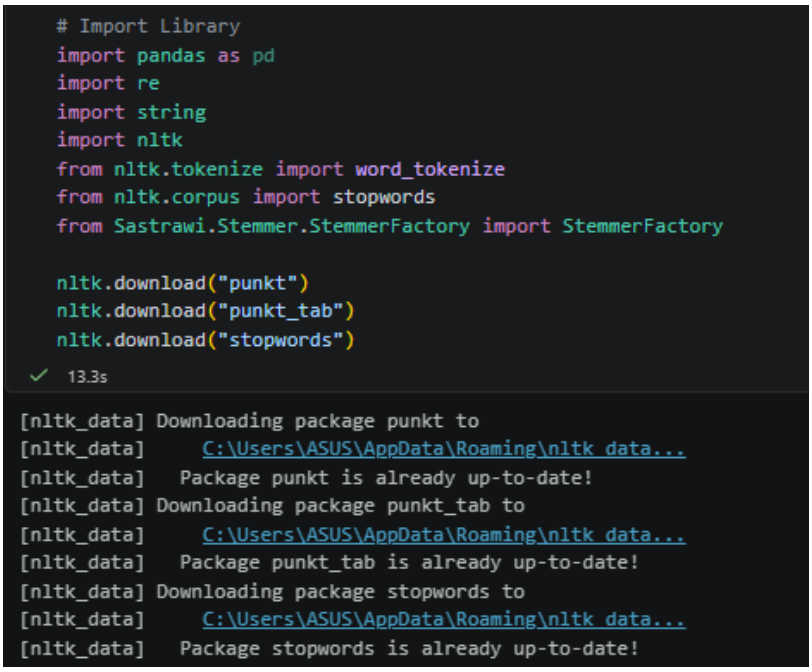
D. Implementasi

1. Membuat Notebook

   Buat file:

   ```
   pertemuan-03/text_normalization.ipynb
   ```

2. Import Library



   Perintah `import pandas as pd`, `import re`, `import string`, dan `import nltk` mengimpor library dasar untuk pemrosesan tabel, Regex, manipulative string, serta pemrosesan bahasa alami.

   Perintah `from nltk.tokenize import word_tokenize`, `from nltk.corpus import stopwords`, dan `from Sastrawi.Stemmer.StemmerFactory import StemmerFactory` mengimpor fungsi tokenisasi kata, daftar kata umum yang akan dibuang (*stopwords*), serta modul pemotong kata berimbuhan (*stemmer*) khusus bahasa Indonesia dari pustaka Sastrawi. Perintah `nltk.download("punkt")`, `nltk.download("punkt_tab")`, dan `nltk.download("stopwords")` yang mengunduh data pendukung NLTK untuk tokenisasi serta daftar *stopwords* agar siap digunakan dalam *pipeline preprocessing*.

3. Load Dataset

```
import pandas as pd
# Load Dataset
data = pd.read_csv('../review_gopay_newest_sort.csv')
text_column = "content"

print("Ukuran dataset:", data.shape)
print("Daftar Kolom:", data.columns.tolist())
print("Jumlah data:", len(data))

data[[text_column]].head(10)
```

✓ 0.8s

```
Ukuran dataset: (123253, 6)
Daftar Kolom: ['userName', 'content', 'score', 'thumbsUpCount', 'reviewCreatedVersion', 'at']
Jumlah data: 123253
```

	content
0	Mantap
1	Bagus dan mudah digunakan
2	Good
3	Sangat membantu orang menengah mantap semoga s...
4	Ga ribet .. Sukaaaa
5	Sangat membantu
6	Mantap pakay gopay
7	mantap mantap mantap
8	Layanan sang Mudah Dan Sangat Memuaskan
9	Berbisnis

Perintah pemuatan dataset diawali dengan mengimpor pustaka manipulasi data melalui baris `import pandas as pd`. Selanjutnya, baris `data = pd.read_csv('../review_gopay_newest_sort.csv')` membaca file CSV data ulasan dan menyimpannya ke dalam DataFrame `data`, diikuti penentuan variabel `text_column = "content"` sebagai acuan nama kolom teks utama. Pada bagian penampil informasi dataset, baris `print("Ukuran dataset:", data.shape)` mencetak dimensi matriks data Frame yaitu 123.253 baris dan 6 kolom. `Baris print("Daftar Kolom:", data.columns.tolist())` mengekstraksi serta menampilkan seluruh nama kolom yang tersedia (`userName, content, score, thumbsUpCount, reviewCreatedVersion, dan at`).

Rangkaian ini dilanjutkan oleh baris `print("Jumlah data:", len(data))` yang mencetak total keseluruhan baris data ulasan sejumlah 123.253 baris. Rangkaian perintah diakhiri oleh baris `data[[text_column]].head(10)` yang menampilkan 10 sampel data teratas dari kolom "`content`" ke dalam bentuk tabel interaktif.

4. Case Folding

```
# Case Folding

#Mengubah seluruh teks menjadi huruf kecil (lowercase):
def case_folding(text):
    return str(text).lower()

# Contoh
text = "Saya Belajar NLP"
print(case_folding(text))
# Output: saya belajar nlp
```

✓ 0.0s

```
saya belajar nlp
```

Perintah `def case_folding(text):` mendefinisikan fungsi bernama case_folding yang menerima masukan berupa teks. Perintah `return str(text).lower()` memastikan teks diubah ke tipe *string* lalu mengonversi seluruh huruf kapital menjadi huruf kecil. Perintah `text = "Saya Belajar NLP"` menyiapkan variabel teks contoh yang berisi kombinasi huruf kapital. Perintah `print(case_folding(text))` yang memanggil fungsi tersebut dan mencetak hasilnya ke layar, yaitu saya belajar nlp.

5. Menghapus URL

```
# Menghapus URL
def remove_url(text):
    return re.sub(r"http\S+|www\S+", "", text)

# Contoh
text = "Kunjungi https://example.com sekarang"
print(remove_url(text))
```

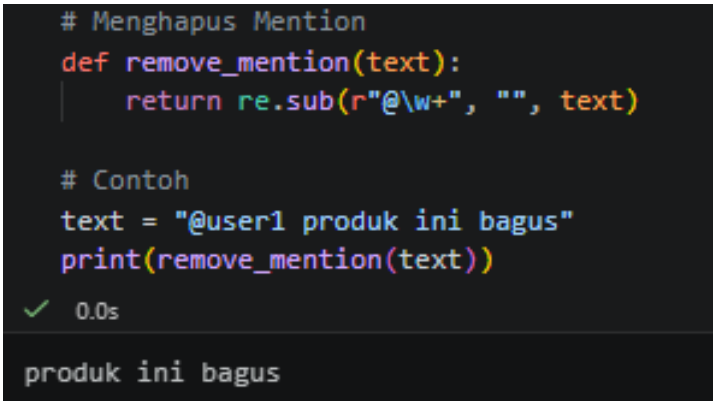
✓ 0.0s

```
Kunjungi  sekarang
```

Perintah `def remove_url(text):` mendefinisikan fungsi bernama `remove_url` untuk menghapus alamat situs web (URL) dari teks. Perintah `return re.sub(r"http\S+|www\S+", "", text)` menggunakan modul Regular Expression (`re`) untuk mencari pola teks yang diawali dengan http atau www beserta karakter setelahnya tanpa spasi `(\S+)`, lalu menggantinya dengan string kosong `""`. Perintah `text = "Kunjungi [https://example.com](https://example.com) sekarang"` menyiapkan

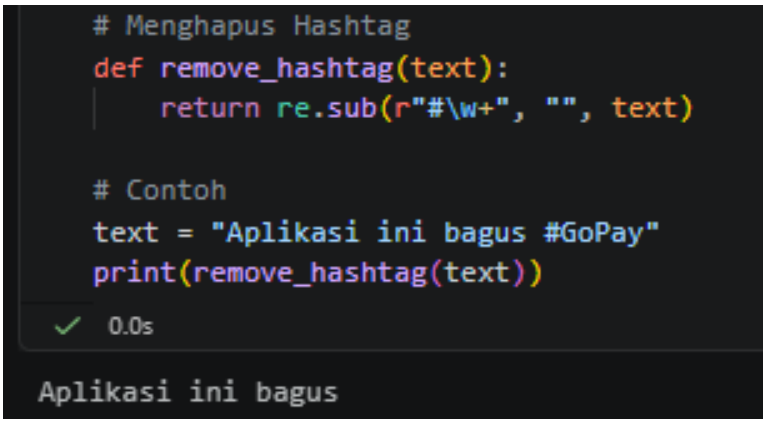
contoh string yang mengandung tautan URL. Perintah `print(remove_url(text))` yang memanggil fungsi tersebut dan mencetak hasil pembersihannya ke layar, yaitu `Kunjungi sekarang` (tautan URL berhasil dihapus).

6. Menghapus Mention



Perintah `def remove_mention(text):` mendefinisikan fungsi bernama `remove_mention` untuk menghapus *sebutan pengguna* (*mention*) pada teks. Perintah `return re.sub(r"@\w+", "", text)` menggunakan Regular Expression (`re`) untuk mencari pola yang diawali simbol `@` lalu diikuti karakter kata (`\w+`), lalu menggantinya dengan string kosong `""`. Perintah `text = "@user1 produk ini bagus"` menyiapkan contoh string yang mengandung sebutan akun `@user1`. Dan perintah `print(remove_mention(text))` yang memanggil fungsi tersebut dan mencetak hasilnya ke layar, yaitu `produk ini bagus` (*mention* berhasil dihapus).

7. Menghapus Hashtag



Perintah `def remove_hashtag(text):` mendefinisikan fungsi bernama `remove_hashtag` untuk menghapus tagar (*hashtag*) dari teks. Perintah `return re.sub(r"#\w+", "", text)` menggunakan Regular Expression (`re`) untuk mencari pola yang diawali simbol `#` lalu diikuti karakter kata (`\w+`), lalu menggantinya dengan string kosong `""`. Perintah `text = "Aplikasi ini bagus #GoPay"` menyiapkan contoh string yang mengandung tagar `#GoPay`. Perintah `print(remove_hashtag(text))` yang memanggil fungsi tersebut dan mencetak hasilnya ke layar, yaitu `Aplikasi ini bagus` (*hashtag* berhasil dihapus).

8. Menghapus Angka

```
# Menghapus Angka
def remove_number(text):
    return re.sub(r"\d+", "", text)

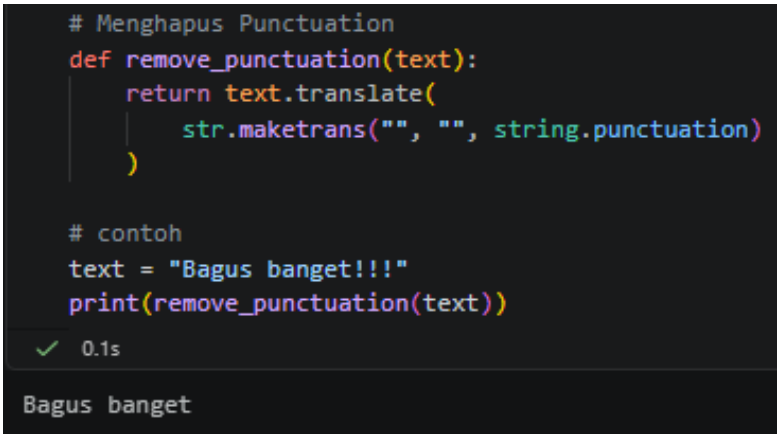
# Contoh
text = "Harga Rp.50.000"
print(remove_number(text))
```

✓ 0.1s

```
Harga Rp..
```

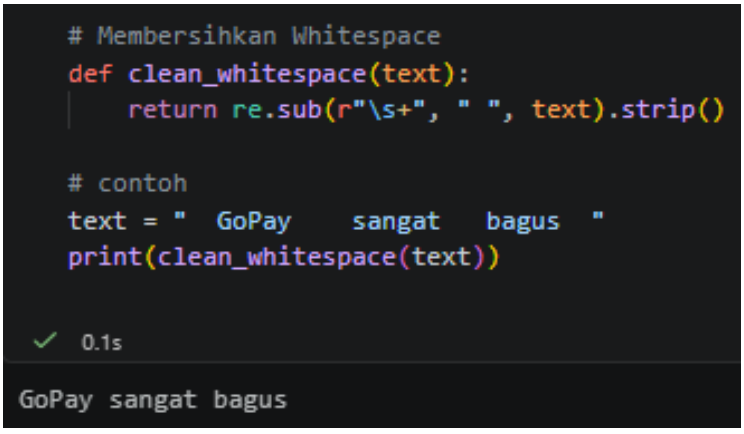
Perintah `def remove_number(text):` mendefinisikan fungsi bernama `remove_number` untuk menghapus semua digit angka dari teks. Perintah `return re.sub(r"\d+", "", text)` menggunakan Regular Expression (`re`) untuk mencari pola digit angka (`\d+`), lalu menggantinya dengan string kosong `""`. Perintah `text = "Harga Rp.50.000"` menyiapkan contoh string yang memuat nominal angka. Dan Perintah `print(remove_number(text))` yang memanggil fungsi tersebut dan mencetak hasilnya ke layar, yaitu `Harga Rp..` (seluruh digit angka terhapus, menyisakan tanda titik dari nominal angka tersebut).

9. Menghapus Punctuation



Perintah `def remove_punctuation(text):` mendefinisikan fungsi bernama `remove_punctuation` untuk menghapus semua tanda baca dari teks. Di dalamnya, perintah `return text.translate(str.maketrans("", "", string.punctuation))` membuat pemetaan karakter untuk membuang seluruh simbol tanda baca bawaan Python (`string.punctuation`). Perintah `text = "Bagus banget!!!"` menyiapkan contoh string yang diakhiri tanda seru. Dan perintah `print(remove_punctuation(text))` yang memanggil fungsi tersebut dan mencetak hasilnya ke layar, yaitu `Bagus banget` (seluruh tanda seru `!` berhasil dihapus).

10. Membersihkan Whitespace



Perintah `def clean_whitespace(text):` mendefinisikan fungsi bernama `clean_whitespace` untuk merapikan spasi berlebih pada teks. Perintah `return re.sub(r"\s+", " ", text).strip()` menggunakan Regular Expression (`re`) untuk mengganti spasi berurut/ganda (`\s+`) menjadi satu

spasi saja `"  "`, lalu menghapus spasi di awal dan akhir kalimat dengan fungsi `.strip()`. Perintah `text = " GoPay sangat bagus "` menyiapkan contoh string yang memiliki banyak spasi tak beraturan di awal, tengah, dan akhir kalimat. Perintah `print(clean_whitespace(text))` yang memanggil fungsi tersebut dan mencetak hasilnya ke layar, yaitu `GoPay sangat bagus` (spasi menjadi rapi dan konsisten).

11. Fungsi Cleaning Lengkap

```python
# Fungsi Cleaning Lengkap

# Gabungkan semua fungsi menjadi satu pipeline:
def clean_text(text):
    text = str(text)
    text = text.lower()          # case folding
    # Cleaning
    text = remove_url(text)      # hapus URL
    text = remove_mention(text)  # hapus mention
    text = remove_hashtag(text)  # hapus hashtag
    text = remove_number(text)   # hapus angka
    text = remove_punctuation(text)  # hapus punctuation
    text = clean_whitespace(text) # bersihkan whitespace
    return text

# Menerapkan ke dataset
text_column = "content"
data["clean_text"] = data[text_column].apply(clean_text)

data[[text_column, "clean_text"]].head(10)
```

✓ 4.1s

	content	clean_text
0	Mantap	mantap
1	Bagus dan mudah digunakan	bagus dan mudah digunakan
2	Good	good
3	Sangat membantu orang menengah mantap semoga s...	sangat membantu orang menengah mantap semoga s...
4	Ga ribet .. Sukaaaa	ga ribet sukaaaa
5	Sangat membantu	sangat membantu
6	Mantap pakay gopay	mantap pakay gopay
7	mantap mantap mantap	mantap mantap mantap
8	Layanan sang Mudah Dan Sangat Memuaskan	layanan sang mudah dan sangat memuaskan
9	Berbisnis	berbisnis

Perintah `def clean_text(text):` mendefinisikan fungsi *pipeline* utama yang menggabungkan seluruh tahapan pembersihan teks. Di dalamnya, teks diubah menjadi string dan di-case folding ke huruf kecil, lalu dibersihkan secara berurutan menggunakan fungsi `remove_url`, `remove_mention`, `remove_hashtag`, `remove_number`, `remove_punctuation`, dan `clean_whitespace` sebelum mengembalikan teks akhir. P

erintah `text_column = "content"` menentukan kolom target ulasan, dan `data["clean_text"] = data[text_column].apply(clean_text)`

menerapkan fungsi pembersihan tersebut ke seluruh baris dataset lalu menyimpannya dalam kolom baru clean_text. Kode diakhiri dengan `data[[text_column, "clean_text"]].head(10)` untuk menampilkan 10 baris pertama perbandingan teks ulasan asli dan hasil pembersihannya.

12. Stopword Removal

```
# Stopword Removal
stop_words = set(stopwords.words("indonesian"))

def remove_stopwords(tokens):
    return [
        token for token in tokens
        if token not in stop_words
    ]

# Contoh
tokens = ["saya", "sedang", "belajar", "nlp", "yang", "menarik"]
print(remove_stopwords(tokens))
```

✓ 0.0s

```
['belajar', 'nlp', 'menarik']
```

Perintah `stop_words = set(stopwords.words("indonesian"))` memuat daftar kata-kata umum dalam bahasa Indonesia dari pustaka NLTK dan mengubahnya menjadi *set* agar pencarian token lebih cepat. Perintah `def remove_stopwords(tokens)` : mendefinisikan fungsi untuk menyaring kata-kata penyambung/kata umum tersebut.

Konstruksi *list comprehension* [`token for token in tokens if token not in stop_words]` digunakan untuk mengambil setiap kata yang tidak terdaftar dalam *stop_words*. Perintah `tokens = ["saya", "sedang", "belajar", "nlp", "yang", "menarik"]` menyiapkan daftar token contoh.

Kode diakhiri dengan `print(remove_stopwords(tokens))` yang menampilkan hasil penyaringan, yaitu `['belajar', 'nlp', 'menarik']` (kata "saya", "sedang", dan "yang" berhasil dibuang).

13. Stemming dengan Sastrawi

```
import pandas as pd

data = pd.read_csv('../review_gopay_newest_sort.csv')
text_column = "content"

# Stemming dengan Sastrawi
factory = StemmerFactory()
stemmer = factory.create_stemmer()

def stem_tokens(tokens):
    return [stemmer.stem(token) for token in tokens]

# Contoh
text = "Mahasiswa sedang mempelajari pemrograman."
print(stemmer.stem(text))
```

✓ 2.8s

```
mahasiswa sedang ajar pemrograman
```

Perintah `import pandas as pd` mengimpor pustaka Pandas untuk manipulasi data berbentuk tabel. Selanjutnya, perintah `data = pd.read_csv('../review_gopay_newest_sort.csv')` serta `text_column = "content"` berfungsi untuk memuat berkas dataset ulasan dan menetapkan nama kolom target.

Perintah `factory = StemmerFactory()` dan `stemmer = factory.create_stemmer()` membuat objek *stemmer* dari pustaka Sastrawi. Perintah `def stem_tokens(tokens)`: mendefinisikan fungsi yang menerima daftar token dan mengonversi setiap kata menjadi kata dasar melalui konstruksi `[stemmer.stem(token) for token in tokens]`.

Perintah pengujian pada `variabel text = "Mahasiswa sedang mempelajari pemrograman."` menggunakan perintah `print(stemmer.stem(text))`.

Hasil output yang tercetak di layar adalah `mahasiswa sedang ajar pemrograman` (kata "mempelajari" berhasil diubah menjadi kata dasar "ajar").

14.. Pipeline Preprocessing

```python
# Pipeline Preprocessing

def preprocessing_pipeline(text):
    # 1. Cleaning
    text = clean_text(text)

    # 2. Tokenization
    tokens = word_tokenize(text)

    # 3. Stopword Removal
    tokens = remove_stopwords(tokens)

    # 4. Stemming
    tokens = stem_tokens(tokens)

    return tokens

# Menerapkan preprocessing secara bertahap
import time

text_column = "content"
batch_size = 200
processed_tokens = []

start_time = time.time()

for start in range(0, len(data), batch_size):
    end = min(start + batch_size, len(data))

    batch = data[text_column].iloc[start:end]
    hasil = batch.apply(preprocessing_pipeline)

    processed_tokens.extend(hasil.tolist())

    print(f"Data {end} dari {len(data)} selesai diproses")

# Menyimpan hasil tokenisasi dan preprocessing
data["processed_tokens"] = processed_tokens

print("\nPreprocessing selesai!")
print(f"Waktu proses: {time.time() - start_time:.2f} detik")

# Menampilkan 10 hasil pertama
print(data[["content", "processed_tokens"]].head(10))

# Menerapkan ke dataset
data["processed_tokens"] = data[text_column].apply(preprocessing_pipeline)
print(data[["content", "processed_tokens"]].head(10))
```

```
Data 123200 dari 123253 selesai diproses
Data 123253 dari 123253 selesai diproses

Preprocessing selesai!
Waktu proses: 10909.84 detik
```

Perintah `def preprocessing_pipeline(text):` mendefinisikan fungsi *pipeline* lengkap yang menggabungkan empat tahapan *preprocessing*, yaitu pembersihan teks (`clean_text`), tokenisasi (`word_tokenize`), pembuangan kata umum (`remove_stopwords`), dan pengubahan kata dasar (`stem_tokens`) sebelum mengembalikan daftar token akhir.

Perintah `import time` mengimpor modul waktu untuk mengukur durasi proses. Variabel `text_column = "content", batch_size = 200,` dan `processed_tokens = []` disiapkan untuk membagi proses eksekusi ke dalam kelompok kecil agar riwayat kemajuan dapat dipantau. Perintah `start_time = time.time()` mencatat waktu mulai, lalu perulangan `for`

`start in range(0, len(data), batch_size)`: memproses dataset per 200 baris menggunakan `batch.apply(preprocessing_pipeline)` dan menggabungkan hasilnya ke dalam `processed_tokens`. Setiap *batch* selesai diproses, perintah `print(f"Data {end} dari {len(data)} selesai diproses")` menampilkan log kemajuan hingga mencapai data terakhir (123.253 data).

Setelah perulangan selesai, perintah `data["processed_tokens"] = processed_tokens` menyimpan seluruh daftar token ke kolom baru. Pesan selesai serta total durasi pemrosesan dicetak menggunakan `print("\nPreprocessing selesai!")` dan `print(f"Waktu proses: {time.time() - start_time:.2f} detik")`. Perintah `print(data[["content", "processed_tokens"]].head(10))` untuk menampilkan 10 baris pertama sampel perbandingan antara teks ulasan asli dan token hasil pemrosesan.

15. Menggabungkan Token Kembali

```
#  Menggabungkan Token Kembali
data["processed_text"] = data["processed_tokens"].apply(
    lambda tokens: " ".join(tokens)
)

# Menampilkan hasil preprocessing
print(data[["content", "processed_tokens", "processed_text"]].head(10))
```

✓ 0.4s

```
                                        content  \
0                                        Mantap
1                         Bagus dan mudah digunakan
2                                          Good
3  Sangat membantu orang menengah mantap semoga s...
4                               Ga ribet .. Sukaaaa
5                                   Sangat membantu
6                                Mantap pakay gopay
7                               mantap mantap mantap
8            Layanan sang Mudah Dan Sangat Memuaskan
9                                       Berbisnis

                                    processed_tokens  \
0                                           [mantap]
1                                     [bagus, mudah]
2                                           [good]
3  [bantu, orang, tengah, mantap, moga, sukses, g...
4                                [ga, ribet, sukaaaa]
5                                           [bantu]
6                              [mantap, pakay, gopay]
7                             [mantap, mantap, mantap]
8                           [layan, sang, mudah, muas]
9                                           [bisnis]
```

```
                                processed_text
0                                       mantap
1                                  bagus mudah
2                                         good
3  bantu orang tengah mantap moga sukses gopay
4                               ga ribet sukaaaa
5                                        bantu
6                            mantap pakay gopay
7                          mantap mantap mantap
8                         layan sang mudah muas
9                                       bisnis
```

Perintah `data["processed_text"] = data["processed_tokens"].apply(lambda tokens: " ".join(tokens))` membuat kolom baru bernama `processed_text` dengan menggabungkan setiap kata di dalam *list* `processed_tokens` menggunakan spasi sebagai pemisah.

Perintah `print(data[["content", "processed_tokens", "processed_text"]].head(10))` yang menampilkan 10 baris pertama untuk membandingkan tiga kolom sekaligus, yaitu teks ulasan asli (`content`), hasil tokenisasi bersih (`processed_tokens`), dan teks akhir yang sudah siap digunakan untuk pemodelan NLP (`processed_text`).

16. Perbandingan Raw vs Processed

```python
# Perbandingan Raw vs Processed
for i in range(5):
    print("RAW:")
    print(data.iloc[i][text_column])

    print("\nPROCESSED:")
    print(data.iloc[i]["processed_text"])

    print("=" * 80)
```

✓ 0.0s



Perintah `for i in range(5)`: melakukan perulangan sebanyak lima kali untuk mengambil lima baris data awal. Di dalam perulangan, perintah `print("RAW:")` dan `print(data.iloc[i][text_column])` menampilkan teks mentah dari kolom asli. Kemudian, perintah `print("\nPROCESSED:")` dan `print(data.iloc[i]["processed_text"])` mencetak teks yang telah dibersihkan. Perulangan diakhiri dengan `print("=" * 80)` untuk mencetak garis pemisah sepanjang 80 karakter agar hasil perbandingan antarbaris terlihat rapi.

Hasil output menunjukkan perubahan teks secara jelas, seperti hilangnya kata hubung/stopword (contohnya kata "dan" serta "sangat"), pengubahan huruf besar menjadi huruf kecil (*case folding*), serta penerapan *stemming* yang mengubah kata berimbuhan menjadi kata dasar (seperti kata "membantu" dan "semoga" yang berubah menjadi "bantu" dan "moga").

17. Menyimpan Dataset Hasil Preprocessing

```
# Menyimpan Dataset Hasil Preprocessing
output_columns = [text_column, "clean_text", "processed_text"]

data[["content", "processed_text"]].to_csv(
    "dataset_preprocessed.csv",
    index=False,
    encoding="utf-8-sig"
)

print("Dataset berhasil disimpan!")
```

✓ 1.2s

```
Dataset berhasil disimpan!
```

Perintah `output_columns = [text_column, "clean_text", "processed_text"]` mendefinisikan daftar nama kolom yang relevan dari hasil pemrosesan. Selanjutnya, perintah `data[["content", "processed_text"]].to_csv("dataset_preprocessed.csv", index=False, encoding="utf-8-sig")` memilih kolom teks ulasan asli (`content`) serta teks hasil pembersihan akhir (`processed_text`), lalu mengekspornya ke berkas `dataset_preprocessed.csv`. Pengaturan `index=False` digunakan agar nomor indeks baris tidak ikut tersimpan sebagai kolom baru, sementara `encoding="utf-8-sig"` memastikan berkas dapat dibaca dengan benar tanpa masalah karakter di berbagai aplikasi seperti Microsoft Excel. Dan perintah `print("Dataset berhasil disimpan!")` yang memunculkan konfirmasi di layar bahwa proses penyimpanan ke berkas CSV telah selesai dijalankan

18. Tugas Eksperimen

```
# TUGAS EKSPERIMEN
hasil = data[[text_column]].head(20).copy()
hasil["lowercase"] = hasil[text_column].apply(case_folding)
hasil["cleaning"] = hasil[text_column].apply(clean_text)
hasil["tokenisasi"] = hasil["cleaning"].apply(word_tokenize)
hasil["stopword"] = hasil["tokenisasi"].apply(
    remove_stopwords
)
hasil["stemming"] = hasil["stopword"].apply(stem_tokens)
hasil["stemming"] = hasil["stemming"].apply(
    lambda tokens: " ".join(tokens)
)
hasil.columns = [
    "Raw Text",
    "Setelah Lowercase",
    "Setelah Cleaning",
    "Setelah Tokenisasi",
    "Setelah Stopword",
    "Setelah Stemming"
]
hasil
```

	Raw Text	Setelah Lowercase	Setelah Cleaning
0	Mantap	mantap	mantap
1	Bagus dan mudah digunakan	bagus dan mudah digunakan	bagus dan mudah digunakan
2	Good	good	good
3	Sangat membantu orang menengah mantap semoga s...	sangat membantu orang menengah mantap semoga s...	sangat membantu orang menengah mantap semoga s...
4	Ga ribet .. Sukaaaa	ga ribet .. sukaaaa	ga ribet sukaaaa
5	Sangat membantu	sangat membantu	sangat membantu
6	Mantap pakay gopay	mantap pakay gopay	mantap pakay gopay
7	mantap mantap mantap	mantap mantap mantap	mantap mantap mantap
8	Layanan sang Mudah Dan Sangat Memuaskan	layanan sang mudah dan sangat memuaskan	layanan sang mudah dan sangat memuaskan
9	Berbisnis	berbisnis	berbisnis
10	Mohon di buka kembali goppy saya 🙏🙏	mohon di buka kembali goppy saya 🙏🙏	mohon di buka kembali goppy saya 🙏🙏
11	Cepat	cepat	cepat
12	keren, dan sangat terbantu	keren, dan sangat terbantu	keren dan sangat terbantu
13	Sangat mudah	sangat mudah	sangat mudah
14	Sip dah	sip dah	sip dah
15	Gopay yang sangat mantap respon cepat	gopay yang sangat mantap respon cepat	gopay yang sangat mantap respon cepat
16	Memudahkan driver..mantap	memudahkan driver..mantap	memudahkan drivermantap
17	Keren	keren	keren
18	Mudah dan sangat ringkas	mudah dan sangat ringkas	mudah dan sangat ringkas
19	Bagus aplikasinya saya suka	bagus aplikasinya saya suka	bagus aplikasinya saya suka

Setelah Tokenisasi	Setelah Stopword	Setelah Stemming
[mantap]	[mantap]	mantap
[bagus, dan, mudah, digunakan]	[bagus, mudah]	bagus mudah
[good]	[good]	good
[sangat, membantu, orang, menengah, mantap, se...	[membantu, orang, menengah, mantap, semoga, su...	bantu orang tengah mantap moga sukses gopay
[ga, ribet, sukaaaa]	[ga, ribet, sukaaaa]	ga ribet sukaaaa
[sangat, membantu]	[membantu]	bantu
[mantap, pakay, gopay]	[mantap, pakay, gopay]	mantap pakay gopay
[mantap, mantap, mantap]	[mantap, mantap, mantap]	mantap mantap mantap
[layanan, sang, mudah, dan, sangat, memuaskan]	[layanan, sang, mudah, memuaskan]	layan sang mudah muas
[berbisnis]	[berbisnis]	bisnis
[mohon, di, buka, kembali, goppy, saya 🙏🙏]	[mohon, buka, goppy, saya 🙏🙏]	mohon buka goppy saya
[cepat]	[cepat]	cepat
[keren, dan, sangat, terbantu]	[keren, terbantu]	keren bantu
[sangat, mudah]	[mudah]	mudah
[sip, dah]	[sip, dah]	sip dah
[gopay, yang, sangat, mantap, respon, cepat]	[gopay, mantap, respon, cepat]	gopay mantap respon cepat
[memudahkan, drivermantap]	[memudahkan, drivermantap]	mudah drivermantap
[keren]	[keren]	keren
[mudah, dan, sangat, ringkas]	[mudah, ringkas]	mudah ringkas
[bagus, aplikasinya, saya, suka]	[bagus, aplikasinya, suka]	bagus aplikasi suka

Perintah `hasil = data[[text_column]].head(20).copy()` mengambil 20 baris data pertama dari kolom ulasan asli. Selanjutnya, setiap tahapan pemrosesan diterapkan satu per satu dan disimpan ke kolom baru: `case_folding` membuat teks menjadi huruf kecil (`lowercase`), `clean_text` menghapus *noise* (`cleaning`), `word_tokenize` memecah teks menjadi daftar kata (`tokenisasi`), `remove_stopwords` membuang kata umum (`stopword`), dan `stem_tokens` mengubah kata menjadi kata dasar (`stemming`). Setelah itu, perintah `hasil["stemming"] = hasil["stemming"].apply(lambda tokens: " ".join(tokens))` menggabungkan kembali daftar token dasar menjadi kalimat utuh. Dan Perintah mengubah nama seluruh kolom menggunakan `hasil.columns = [...]` agar tampilan tabel lebih mudah dibaca (yaitu "Raw Text", "Setelah Lowercase", "Setelah Cleaning", "Setelah

Tokenisasi", "Setelah Stopword", dan "Setelah Stemming"), lalu memanggil `hasil` untuk menampilkan tabel perbandingan eksperimen tersebut.

```
import os
import pandas as pd

file_final = "dataset_preprocessed.csv"

print("File ditemukan:", os.path.exists(file_final))

if os.path.exists(file_final):
    cek = pd.read_csv(file_final, low_memory=False)
    print("Jumlah baris:", len(cek))
    print("Kolom tersedia:", cek.columns.tolist())
    display(cek[["content", "processed_text"]].head(5))
```

✓ 1.7s

```
File ditemukan: True
Jumlah baris: 123253
Kolom tersedia: ['content', 'processed_text']
```

	content	processed_text
0	Mantap	mantap
1	Bagus dan mudah digunakan	bagus mudah
2	Good	good
3	Sangat membantu orang menengah mantap semoga s...	bantu orang tengah mantap moga sukses gopay
4	Ga ribet .. Sukaaaa	ga ribet sukaaaa

Kode tersebut diawali dengan mengimpor modul sistem operasi (`os`) serta pustaka manipulasi data (`pandas`). Nama berkas target ditentukan dalam variabel file`_final = "dataset_preprocessed.csv"`. Keberadaan berkas di dalam direktori kerja diperiksa terlebih dahulu dan hasilnya dicetak ke layar melalui perintah `print("File ditemukan:", os.path.exists(file_final))`.

Setelah itu, alur program dilanjutkan dengan pengondisian `if os.path.exists(file_final):`. Jika berkas terdeteksi, dataset CSV dimuat ke dalam DataFrame cek menggunakan perintah `pd.read_csv(file_final, low_memory=False)`. Informasi dasar mengenai data kemudian ditampilkan berturut-turut, yaitu total baris data via `print("Jumlah baris:", len(cek))` dan daftar nama kolom yang ada lewat `print("Kolom tersedia:", cek.columns.tolist())`.

Sebagai penutup, program menampilkan lima baris pertama dari sampel data ulasan asli dan hasil pemrosesan teks. Tampilan tabel ini difokuskan pada kolom `content` dan processed_text menggunakan perintah `display(cek[["content", "processed_text"]].head(5))`.

E. Tugas Analisis

Mahasiswa mampu menjawab:

1. Apakah semua stopword harus dihapus? Berikan contoh ketika stopword penting untuk dipertahankan.

   Jawaban:

   Tidak, semua stopword tidak harus dihapus. Stopword merupakan kata-kata umum yang sering muncul, seperti dan, yang, di, ke, dan tidak. Kata-kata tersebut biasanya dihapus karena dianggap kurang memberikan informasi penting. Namun, dalam beberapa kasus, stopword justru memengaruhi makna kalimat.

   Contohnya, kalimat "Aplikasi ini bagus" dan "Aplikasi ini tidak bagus" memiliki makna yang berbeda. Jika kata "tidak" dihapus, kedua kalimat tersebut dapat memiliki makna yang sama, padahal sentimennya berbeda.

2. Apakah stemming selalu meningkatkan kualitas data? Kapan stemming dapat merugikan?

   Jawaban:

   Tidak, stemming tidak selalu meningkatkan kualitas data. Stemming adalah proses mengubah kata berimbuhan menjadi kata dasar, misalnya "membayarkan" menjadi "bayar". Proses ini dapat membantu menyamakan kata-kata yang memiliki bentuk berbeda tetapi berkaitan dengan makna yang sama.

   Namun, stemming dapat merugikan jika menghasilkan kata dasar yang tidak tepat atau menghilangkan perbedaan makna. Hal ini dapat terjadi pada kata tertentu yang memiliki bentuk atau penggunaan khusus. Selain itu, stemming juga kurang cocok untuk tugas yang membutuhkan informasi bentuk kata secara lengkap, seperti analisis tata bahasa.

3. Apakah angka harus selalu dihapus? Berikan contoh kasus di mana angka penting.

   Jawaban:

   Tidak, angka tidak harus selalu dihapus. Angka dapat mengandung informasi penting, tergantung pada tujuan analisis. Dalam analisis sentimen ulasan aplikasi, misalnya, angka seperti "versi 5.2.1" dapat menunjukkan versi aplikasi yang sedang dibahas. Angka seperti "bintang 1" atau "rating 5" juga

dapat menunjukkan tingkat kepuasan pengguna. Jika semua angka dihapus, informasi tersebut bisa hilang dan memengaruhi hasil analisis. Karena itu, angka sebaiknya dipertahankan apabila berkaitan dengan tujuan pengolahan data.

4. Apakah URL selalu tidak berguna? Kapan URL dapat menjadi fitur yang informatif?

   Jawaban:

   Tidak, URL tidak selalu tidak berguna. URL terkadang dapat memberikan informasi tentang sumber atau konteks suatu teks. Contohnya, dalam analisis keamanan siber, URL dapat digunakan untuk mengenali tautan yang berpotensi berbahaya. Dalam analisis berita, domain situs juga dapat membantu mengidentifikasi sumber informasi.

   Namun, jika tujuan preprocessing adalah menganalisis sentimen ulasan pengguna, URL yang panjang biasanya tidak terlalu membantu dalam memahami pendapat pengguna. URL dapat dihapus atau diubah menjadi penanda seperti [URL], sesuai kebutuhan analisis.

5. Apakah hashtag harus selalu dihapus?

   Jawaban:

   Tidak, hashtag tidak harus selalu dihapus. Hashtag dapat mengandung informasi penting mengenai topik, konteks, atau sentimen suatu teks. Contohnya, hashtag #Kecewa, #AplikasiError, dan #PelayananBagus dapat membantu memahami pendapat pengguna.

   Jika hashtag dihapus seluruhnya, informasi tersebut bisa hilang. Oleh karena itu, hashtag sebaiknya dipertahankan atau simbol # dihilangkan dengan tetap menyimpan kata di belakangnya, sesuai kebutuhan analisis.

6. Apakah preprocessing yang sama cocok untuk semua task NLP?

   Tidak, preprocessing yang sama tidak selalu cocok untuk semua tugas NLP. Setiap tugas memiliki tujuan dan kebutuhan data yang berbeda. Pada analisis sentimen, kata negasi seperti “tidak” perlu dipertahankan karena memengaruhi makna kalimat. Pada klasifikasi topik, penghapusan beberapa stopword mungkin dapat membantu. Sementara itu, pada penerjemahan mesin atau analisis tata bahasa, urutan kata, tanda baca, dan bentuk kata bisa menjadi informasi penting.

F. Pertanyaan Diskusi

1. Mengapa preprocessing berbeda untuk task sentiment analysis vs text classification?

   Jawaban:

   Preprocessing berbeda karena tujuan dari kedua tugas tersebut tidak sama. Sentiment analysis bertujuan mengetahui perasaan atau pendapat seseorang, misalnya positif, negatif, atau netral. Sementara itu, text classification bertujuan mengelompokkan teks ke dalam kategori tertentu, seperti berita olahraga, politik, atau teknologi.

   Pada sentiment analysis, kata-kata yang menunjukkan emosi dan negasi seperti “bagus”, “kecewa”, dan “tidak” perlu dipertahankan. Pada text classification, beberapa kata umum dapat dihapus untuk membantu model berfokus pada kata-kata yang membedakan kategori. Namun, preprocessing tetap bergantung pada jenis klasifikasi yang dilakukan.

2. Apa dampak menghapus kata "tidak" dalam kalimat "tidak bagus"?

   Jawaban:

   Menghapus kata “tidak” dapat mengubah makna kalimat dan menyebabkan kesalahan analisis sentimen. Kalimat “tidak bagus” memiliki sentimen negatif, sedangkan “bagus” memiliki sentimen positif.

   Jika kata “tidak” dihapus, kalimat “tidak bagus” menjadi “bagus”. Akibatnya, model dapat salah mengklasifikasikan sentimen negatif sebagai positif. Oleh karena itu, kata negasi seperti “tidak”, “bukan”, dan “belum” sebaiknya dipertahankan dalam analisis sentimen.

3. Bagaimana cara menangani slang Bahasa Indonesia (gue, lu, bgt)?

   Jawaban:

   Slang atau bahasa tidak baku dapat ditangani melalui proses normalisasi, yaitu mengubah kata tidak baku menjadi bentuk yang lebih umum atau baku. Contohnya, “gue” menjadi “saya”, “lu” menjadi “kamu”, dan “bgt” menjadi “banget”. Normalisasi dapat dilakukan menggunakan kamus slang yang berisi pasangan kata tidak baku dan bentuk normalisasinya.

4. Apakah Sastrawi cocok digunakan untuk semua domain teks?

   Jawaban:

Tidak, Sastrawi tidak selalu cocok untuk semua domain teks. Sastrawi merupakan library stemming Bahasa Indonesia yang digunakan untuk mengubah kata berimbuhan menjadi kata dasar. Library ini cukup membantu untuk teks Bahasa Indonesia, tetapi hasilnya belum tentu sesuai untuk setiap jenis teks.

Misalnya, pada ulasan aplikasi, Sastrawi dapat membantu menyederhanakan kata seperti "menggunakan" menjadi "guna". Namun, pada teks yang mengandung istilah teknologi, nama produk, singkatan, atau bahasa gaul, hasil stemming perlu diperiksa karena bisa saja menghasilkan bentuk yang kurang tepat.

Oleh karena itu, penggunaan Sastrawi sebaiknya disesuaikan dengan domain teks dan diuji terlebih dahulu untuk memastikan hasil preprocessing tetap mempertahankan makna.

5. Apa perbedaan pendekatan normalisasi untuk teks Twitter vs teks berita formal?

   Jawaban:

   Normalisasi teks Twitter dan berita formal berbeda karena karakteristik bahasanya juga berbeda. Teks Twitter atau media sosial biasanya mengandung singkatan, slang, hashtag, emoji, pengulangan huruf, dan kesalahan penulisan. Sementara itu, teks berita formal umumnya menggunakan bahasa yang lebih baku dan struktur kalimat yang lebih teratur.

   Pada teks Twitter, normalisasi dapat mencakup perubahan slang menjadi kata baku, penanganan hashtag dan emoji, serta pengurangan huruf berulang, misalnya "baguuuus" menjadi "bagus". Namun, emoji atau hashtag yang menunjukkan sentimen sebaiknya tetap dipertahankan atau diubah menjadi bentuk teks yang sesuai.

   Pada berita formal, normalisasi biasanya lebih berfokus pada pembersihan karakter yang tidak diperlukan, penanganan tanda baca, dan konsistensi penulisan. Slang biasanya tidak perlu banyak ditangani karena lebih jarang ditemukan.

G. Kesimpulan

Kesimpulan yang dapat diambil dari praktikum ini adalah:

1. Praktikum berhasil mengimplementasikan *pipeline* normalisasi teks secara berurutan: *case folding*, *cleaning* (URL, mention, hashtag, angka, tanda baca, spasi), *tokenization*, *stopword removal*, dan *stemming*.
2. Modul ini memanfaatkan *stemmer* Sastrawi untuk mengubah kata berimbuhan Bahasa Indonesia menjadi kata dasarnya (seperti "mempelajari" menjadi "ajar").
3. Pemrosesan seluruh 123.253 data ulasan secara bertahap memakan waktu sekitar 10.909,84 detik dan berhasil diekspor ke file CSV baru.
4. Normalisasi harus disesuaikan dengan *task* NLP; menghapus elemen seperti angka, hashtag, atau kata negasi (seperti "tidak") dapat mengubah makna teks secara drastis pada analisis sentimen.

LAPORAN PRAKTIKUM

NATURAL LANGUAGE PROCESSOR



Annisa Almaghvirah

240306037

https://github.com/ANNISAALMAGHVIRAH

Program Studi Teknologi Informasi

Fakultas Dakwah dan Ilmu Komunikasi

Universitas Islam Negeri Mataram

2026

Modul	Modul 1 - Fundamental dan Text Processing
Topik	Linguistic Processing
Repository	NATURAL-LANGUAGE-PROCESSING/Annisa Almaghvirah_240306037_Modul1 at main · ANNISAALMAGHVIRAH/NATURAL-LANGUAGE-PROCESSING
Tanggal	13 Oktober 2026

A. Tujuan

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Memahami Lexical Analysis
2. Memahami Morphology Bahasa Indonesia
3. Memahami Konsep Part-Of-Speech (Pos)
4. Melakukan Pos Tagging
5. Memahami Syntax Dan Dependency Parsing
6. Membaca Struktur Kalimat
7. Memvisualisasikan Dependency Tree
8. Menganalisis Kesalahan Linguistic Processing

B. Requiretment

1. Sistem Operasi yang digunakan : Windows 11 Home Single Language 64-bit (10.0, Build 26100)
2. Browser : Google Chrome Version 130.0.6723.117 (Official Build) (64-bit)
3. Tools yang digunakan : Visual Studio Code, Python , Jupyter Notebook, Virtual Environment (.venv), pandas, numpy, matplotlib, seaborn, nltk

C. Dasar Teori

1. Lexical Analysis

   Lexical analysis berkaitan dengan unit-unit kata dalam bahasa. Dalam NLP, setiap kata dalam kalimat dianalisis secara individual.

   Contoh:

   ```
   Kalimat : "Mahasiswa belajar NLP."
   Token   : ["Mahasiswa", "belajar", "NLP", "."]
   ```

2. Morphology

   Morphology mempelajari struktur pembentukan kata. Bahasa Indonesia kaya akan imbuhan (afiks).

   Contoh pembentukan kata:

   ```
   ajar
    ↓ + be- + -ar
   belajar
    ↓ + mem-pe- + -i
   mempelajari
    ↓ + pe- + -an
   Pembelajaran
   ```

   Contoh lain:

   ```
   main → bermain → permainan → memainkan → dimainkan
   hadir → kehadiran → ketidakhadiran
   ```

   Stemmer (seperti Sastrawi) mencoba menemukan bentuk dasar/stem dari kata berimbuhan.

3. Part-of-Speech (POS)

   POS menunjukkan kelas kata (word class) dalam kalimat.

   Contoh:

   ```
   Saya      → PRON  (Pronoun / Kata ganti)
   membaca   → VERB  (Verb / Kata kerja)
   buku      → NOUN  (Noun / Kata benda)
   menarik   → ADJ   (Adjective / Kata sifat)
   sangat    → ADV   (Adverb / Kata keterangan)
   ```

Tag POS yang umum digunakan (Universal POS Tags):

Tag	Kelas Kata	Contoh
NOUN	Kata benda	buku, universitas
VERB	Kata kerja	belajar, membaca
ADJ	Kata sifat	bagus, menarik
ADV	Kata keterangan	sangat, sudah
PRON	Kata ganti	saya, dia, mereka
DET	Kata penentu	ini, itu
ADP	Preposisi	di, ke, dari, untuk
NUM	Angka	satu, pertama
CONJ	Konjungsi	dan, tetapi, karena
PUNCT	Tanda baca	. , ! ?

4. POS Tagging

   POS Tagging adalah proses memberikan label kelas kata kepada setiap token dalam kalimat.

   ```
   Saya/PRON membaca/VERB buku/NOUN yang/DET menarik/ADJ .
   ```

5. Dependency Parsing

   Dependency parsing mencoba menemukan hubungan gramatikal antar token.

   Contoh:

   ```
   Mahasiswa → membaca → buku
   ```

   Hubungan:

   - `Mahasiswa` adalah subjek dari `membaca`
   - `buku` adalah objek dari `membaca`

   Representasi dependency tree:

   ```
          membaca (ROOT)
          /       \
   Mahasiswa      buku
   (nsubj)        (obj)
   ```

D. Implementasi

1. Membuat file

   Buat file

   `pertemuan-04/linguistic_processing.ipynb`

2. Install dan load spacy

```
(.venv) PS D:\NLP> python -m pip install spacy-stanza
Collecting spacy-stanza
  Downloading spacy_stanza-1.0.4-py3-none-any.whl.metadata (8.6 kB)
Requirement already satisfied: spacy<4.0.0,>=3.0.0 in .\.venv\Lib\site-packages (from spacy-stanza) (3.8.16)
Collecting stanza<1.7.0,>=1.2.0 (from spacy-stanza)
  Downloading stanza-1.6.1-py3-none-any.whl.metadata (13 kB)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in .\.venv\Lib\site-packages (from spacy<4.0.0,>=3.0.0-
tanza) (3.0.12)
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in .\.venv\Lib\site-packages (from spacy<4.0.0,>=3.0.0-
```

Perintah `python -m pip install spacy-stanza` digunakan untuk menginstal pustaka spacy-stanza ke dalam lingkungan Python Anda melalui pip

"Di modul pilihannya model `xx_ent_wiki_sm` atau model khusus bahasa Indonesia `id_ud_gsd`. Tetapi disini saya menggunakan `spacy-stanza` karena membutuhkan pipeline yang mendukung analisis linguistik bahasa Indonesia."

```
#Import Library
import sys
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display

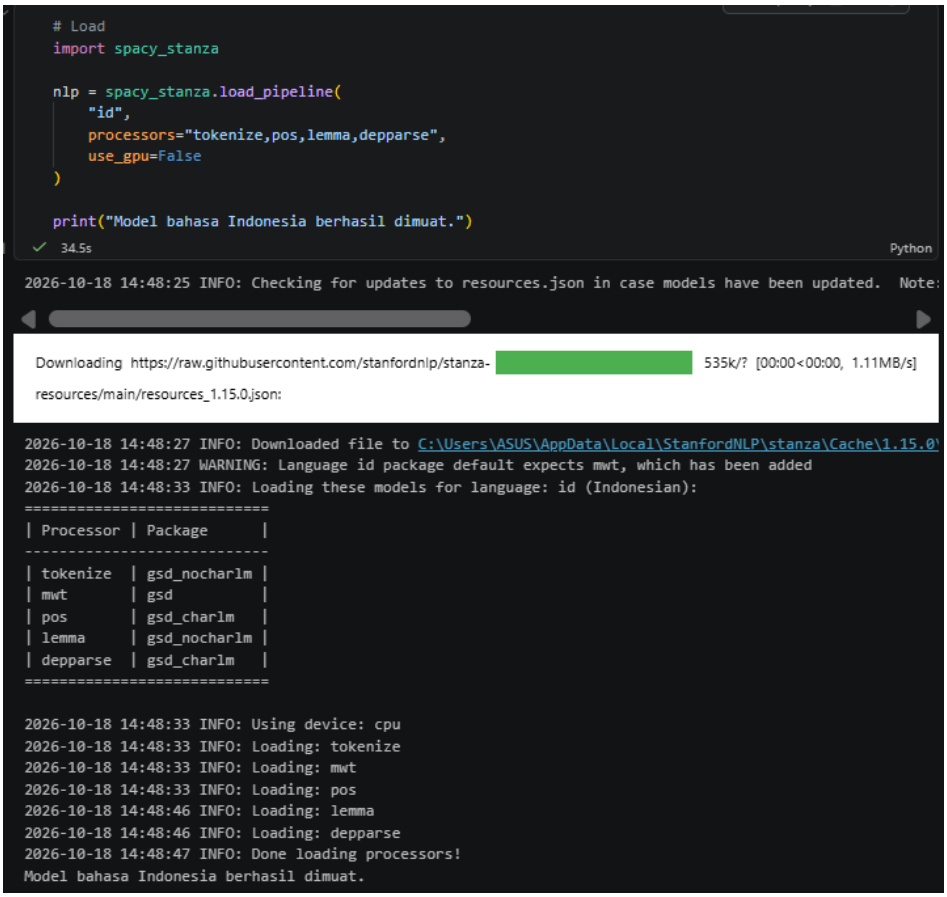
print("Python:", sys.version)
print("Pandas:", pd.__version__)
print("Matplotlib:", plt.matplotlib.__version__)
```

```
Python: 3.13.15 (tags/v3.13.15:4061bc4, Aug  5 2026, 13:05:39) [MSC v.1944 64 bit (AMD64)]
Pandas: 3.0.6
Matplotlib: 3.11.2
```

Perintah `import sys`, `from pathlib import Path`, `import pandas as pd`, `import matplotlib.pyplot as plt`, serta `from IPython.display import display` untuk mengimpor modul sistem dan pustaka pendukung lingkungan kerja.

Kode diakhiri dengan mengeksekusi tiga perintah pencetakan untuk memverifikasi versi pustaka yang digunakan, yaitu `print("Python:", sys.version)` untuk memeriksa versi Python, `print("Pandas:", pd.__version__)` untuk memeriksa versi Pandas, dan

`print("Matplotlib:", plt.matplotlib.__version__)` untuk memeriksa versi Matplotlib.



Perintah tersebut mengimpor `spacy_stanza` melalui `import spacy_stanza` dan memuat *pipeline* pemrosesan bahasa Indonesia (`id`) menggunakan fungsi `nlp = spacy_stanza.load_pipeline("id", processors="tokenize,pos,lemma,depparse", use_gpu=False)`. Parameter `processors` mengaktifkan modul tokenisasi, POS tagging, lematisasi, dan *dependency parsing* pada CPU.

Saat pertama kali dijalankan, sistem mengunduh model Stanford NLP secara otomatis. Setelah semua prosesor dimuat, perintah `print("Model bahasa Indonesia berhasil dimuat.")` akan mencetak pesan konfirmasi bahwa *pipeline* siap digunakan.



```
# Uji Model
teks = "Mahasiswa sedang mempelajari pemrosesan bahasa alami."
doc = nlp(teks)

# Validasi berdasarkan hasil pemrosesan
assert len(doc) > 0, "Tidak ada token yang dihasilkan."
assert any(token.pos_ for token in doc), "POS belum tersedia."
assert any(token.dep_ for token in doc), "Dependency belum tersedia."

print("Token berhasil dibuat:", len(doc))
print("POS tagging tersedia: Ya")
print("Dependency parsing tersedia: Ya")
print("\nHasil analisis:")

for token in doc:
    print(
        f"{token.text:15} | "
        f"{token.pos_:8} | "
        f"{token.lemma_:12} | "
        f"{token.dep_:10} | "
        f"Kepala: {token.head.text}"
    )
```

```
Token berhasil dibuat: 7
POS tagging tersedia: Ya
Dependency parsing tersedia: Ya

Hasil analisis:
Mahasiswa       | PROPN    | mahasiswa    | nsubj      | Kepala: mempelajari
sedang          | AUX      | sedang       | aux        | Kepala: mempelajari
mempelajari     | VERB     | pelajar      | root       | Kepala: mempelajari
pemrosesan      | NOUN     | proses       | obj        | Kepala: mempelajari
bahasa          | NOUN     | bahasa       | compound   | Kepala: pemrosesan
alami           | ADJ      | alami        | amod       | Kepala: bahasa
.               | PUNCT    | .            | punct      | Kepala: mempelajari
```

Perintah `teks = "Mahasiswa sedang mempelajari pemrosesan bahasa alami."` Untuk mendefinisikan kalimat uji coba, lalu memproses kalimat tersebut ke dalam objek `doc = nlp(teks)`.

Selanjutnya, tiga baris validasi dijalankan menggunakan perintah `assert len(doc) > 0, "Tidak ada token yang dihasilkan."`, `assert any(token.pos_ for token in doc), "POS belum tersedia."`, dan `assert any(token.dep_ for token in doc), "Dependency belum tersedia."` untuk memastikan proses tokenisasi, POS tagging, dan *dependency parsing* telah berfungsi dengan benar.

Setelah validasi berhasil, status kesiapan dicetak menggunakan perintah `print("Token berhasil dibuat:", len(doc))`, `print("POS tagging tersedia: Ya")`, dan `print("\nHasil analisis:")`.

Kode diakhiri dengan perulangan `for token in doc:` yang mencetak atribut linguistik setiap kata, meliputi kata asli melalui `token.text`, kelas kata melalui `token.pos_`, lemma atau kata dasar melalui `token.lemma_`, relasi sintaksis melalui `token.dep_`, serta kata induknya melalui `token.head.text`.

3. Memproses Kalimat

```
# Memproses Kalimat
sentence = "Mahasiswa belajar Natural Language Processing."
doc = nlp(sentence)

print("Kalimat:", sentence)
print("\nToken hasil analisis:")
for token in doc:
    print(f"{token.text:<20} POS={token.pos_:<10} Dependency={token.dep_}")
```

```
Kalimat: Mahasiswa belajar Natural Language Processing.

Token hasil analisis:
Mahasiswa            POS=PROPN      Dependency=nsubj
belajar              POS=VERB       Dependency=root
Natural              POS=PROPN      Dependency=obj
Language             POS=PROPN      Dependency=flat:name
Processing           POS=PROPN      Dependency=flat:name
.                    POS=PUNCT      Dependency=punct
```

Perintah `sentence = "Mahasiswa belajar Natural Language Processing."` Untuk mendefinisikan kalimat baru, yang kemudian diproses ke dalam objek `doc = nlp(sentence)`.

Selanjutnya, perintah `print("Kalimat:", sentence)` dan `print("\nToken hasil analisis:")` mencetak kalimat uji coba beserta teks pembuka hasil analisis. Kode diakhiri dengan perulangan `for token in doc:` yang mengeksekusi perintah `print(f"{token.text:<20} POS={token.pos_:<10} Dependency={token.dep_}")` untuk menampilkan setiap token kata, label kelas kata melalui `POS` (seperti `PROPN`, `VERB`, dan `PUNCT`), serta peran sintaksisnya melalui `Dependency` (seperti `nsubj` untuk subjek, `root` untuk verba utama, `obj` untuk objek, `flat:name` untuk nama majemuk, dan punct untuk tanda baca).

4. Analisis POS Token

```
# Analisis POS Token
doc = nlp("Mahasiswa belajar Natural Language Processing.")

print(f"{'Token':<18} {'POS':<12} {'Lemma'}")
print("-" * 55)
for token in doc:
    print(f"{token.text:<18} {token.pos_:<12} {token.lemma_}")
```

```
Token              POS          Lemma
-------------------------------------------------------
Mahasiswa          PROPN        mahasiswa
belajar            VERB         belajar
Natural            PROPN        natural
Language           PROPN        language
Processing         PROPN        processing
.                  PUNCT        .
```

Perintah `doc = nlp("Mahasiswa belajar Natural Language Processing.")` untuk memproses kalimat uji coba. Selanjutnya, perintah `print(f"{'Token':<18} {'POS':<12} {'Lemma'}")` dan `print("-" * 55)` digunakan untuk mencetak tata letak *header* tabel beserta garis pemisah horizontal. K

ode diakhiri dengan perulangan `for token in doc:` yang mengeksekusi perintah `print(f"{token.text:<18} {token.pos_:<12} {token.lemma_}")` untuk menampilkan setiap kata secara teratur, meliputi teks token melalui `token.text`, label kelas kata melalui `token.pos_` (seperti `PROPN`, `VERB`, dan `PUNCT`), serta bentuk kata dasar melalui token.lemma_.

5. Membuat Tabel Analisis POS

```
# Membuat tabel analisis POS
import pandas as pd

def analyze_pos(text, nlp_model=nlp):
    doc = nlp_model(str(text))

    return pd.DataFrame([{
        "Token": token.text,
        "POS": token.pos_,
        "POS Tag": token.tag_,
        "Lemma": token.lemma_,
        "Dependency": token.dep_,
        "Head": token.head.text
    } for token in doc])

data_pos = analyze_pos("Mahasiswa belajar NLP.")
display(data_pos)
```

✓ 1.2s

	Token	POS	POS Tag	Lemma	Dependency	Head
0	Mahasiswa	PROPN	NSD	mahasiswa	nsubj	belajar
1	belajar	VERB	NSD	belajar	root	belajar
2	NLP	PROPN	X--	nlp	obj	belajar
3	.	PUNCT	Z--	.	punct	belajar

Perintah `import pandas as pd` untuk mengimpor pustaka Pandas. Selanjutnya, didefinisikan fungsi `def analyze_pos(text, nlp_model=nlp):` yang menerima masukan teks dan objek model NLP. Di dalam fungsi tersebut, teks diproses menggunakan perintah `doc = nlp_model(str(text))`.

Fungsi ini kemudian mengembalikan sebuah DataFrame melalui perintah `return pd.DataFrame(...)` yang mengekstrak informasi linguistik dari setiap token menggunakan *list comprehension*, yaitu kata asli melalui `"Token"`:

`token.text`, kelas kata Universal POS melalui `"POS": token.pos_`, tag kelas kata spesifik melalui `"POS Tag": token.tag_`, bentuk kata dasar melalui "Lemma": token.lemma_, peran sintaksis melalui `"Dependency": token.dep_`, serta kata induknya melalui `"Head": token.head.text`.

Di bagian luar fungsi, kalimat diuji coba menggunakan perintah `data_pos = analyze_pos("Mahasiswa belajar NLP.")` untuk menyimpan hasil pemrosesan ke dalam variabel `data_pos`. Kode diakhiri dengan perintah display(data_pos) yang menampilkan hasil analisis linguistik tersebut dalam bentuk tabel DataFrame yang rapi.

6. Analisis Beberapa Kalimat

```
# Analisis beberapa kalimat
sentences = [
    "Mahasiswa belajar NLP.",
    "Dosen mengajar Natural Language Processing.",
    "Mahasiswa membaca buku.",
    "Sistem memproses data teks.",
    "Model bahasa memprediksi kata berikutnya."
]

for sentence in sentences:
    print(f"\nKalimat: {sentence}")
    data_result = analyze_pos(sentence)
    display(data_result[["Token", "POS", "Dependency", "Head"]])
```

✓ 8.6s

```
Kalimat: Mahasiswa belajar NLP.
```

	Token	POS	Dependency	Head
0	Mahasiswa	PROPN	nsubj	belajar
1	belajar	VERB	root	belajar
2	NLP	PROPN	obj	belajar
3	.	PUNCT	punct	belajar

```
Kalimat: Dosen mengajar Natural Language Processing.
```

	Token	POS	Dependency	Head
0	Dosen	PROPN	nsubj	mengajar
1	mengajar	VERB	root	mengajar
2	Natural	PROPN	obj	mengajar
3	Language	PROPN	flat:name	Natural
4	Processing	PROPN	flat:name	Language
5	.	PUNCT	punct	mengajar

```
Kalimat: Mahasiswa membaca buku.
```

	Token	POS	Dependency	Head
0	Mahasiswa	PROPN	nsubj	membaca
1	membaca	VERB	root	membaca
2	buku	NOUN	obj	membaca
3	.	PUNCT	punct	membaca

```
Kalimat: Sistem memproses data teks.
```

	Token	POS	Dependency	Head
0	Sistem	PROPN	nsubj	memproses
1	memproses	VERB	root	memproses
2	data	NOUN	obj	memproses
3	teks	NOUN	compound	data
4	.	PUNCT	punct	memproses

```
Kalimat: Model bahasa memprediksi kata berikutnya.
```

	Token	POS	Dependency	Head
0	Model	NOUN	nsubj	memprediksi
1	bahasa	NOUN	compound	Model
2	memprediksi	VERB	root	memprediksi
3	kata	NOUN	obj	memprediksi
4	berikutnya	ADJ	amod	kata
5	.	PUNCT	punct	memprediksi

Perintah `sentences = ["Mahasiswa belajar NLP.", "Dosen mengajar Natural Language Processing.", "Mahasiswa membaca buku.", "Sistem memproses data teks.", "Model bahasa memprediksi kata berikutnya."]` untuk mendefinisikan *list* berisi lima kalimat uji coba.

Selanjutnya, perulangan `for sentence in sentences:` digunakan untuk memproses setiap kalimat secara berurutan. Di dalam perulangan tersebut, perintah `print(f"\nKalimat: {sentence}")` mencetak judul kalimat yang sedang dianalisis.

Kemudian, fungsi `data_result = analyze_pos(sentence)` dipanggil untuk mengekstraksi seluruh atribut linguistik kalimat ke dalam variabel DataFrame

`data_result`. Perintah `display(data_result[["Token", "POS", "Dependency", "Head"]])` yang menyaring dan menampilkan empat kolom utama hasil analisis saja, yaitu teks kata (`Token`), kelas kata (`POS`), relasi sintaksis (`Dependency`), serta kata induknya (`Head`).

7. Dependency Parsing

```
# Dependency parsing
doc = nlp("Mahasiswa belajar NLP.")

print(f"{'Token':<18} {'Dependency':<15} {'Head'}")
print("-" * 50)
for token in doc:
    print(f"{token.text:<18} {token.dep_:<15} {token.head.text}")
```

✓ 1.1s

```
Token              Dependency      Head
--------------------------------------------------
Mahasiswa          nsubj           belajar
belajar            root            belajar
NLP                obj             belajar
.                  punct           belajar
```

Perintah `doc = nlp("Mahasiswa belajar NLP.")` untuk memproses sampel kalimat. Selanjutnya, perintah `print(f"{'Token':<18} {'Dependency':<15} {'Head'}")` dan `print("-" * 50)` digunakan untuk membuat tata letak *header* tabel beserta garis pemisah horizontal.

Perintah perulangan `for token in doc:` yang mengeksekusi perintah `print(f"{token.text:<18} {token.dep_:<15} {token.head.text}")` untuk menampilkan setiap kata secara teratur, meliputi teks token via `token.text`, relasi sintaksis via `token.dep_` (seperti `nsubj` untuk subjek, `root` untuk kata kerja utama, `obj` untuk objek, dan `punct` untuk tanda baca), serta kata induknya via `token.head.text`.

8. Visualisasi Dependency

```python
# Visualisasi Dependency
from spacy import displacy
from pathlib import Path

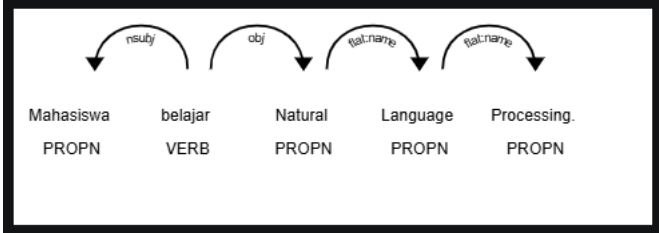
doc = nlp("Mahasiswa belajar Natural Language Processing.")
displacy.render(doc, style="dep", jupyter=True, options={"distance": 100})

# Membuat HTML dependency tree
html = displacy.render(
    doc,
    style="dep",
    page=True,
    jupyter=False
)

# Menyimpan hasil ke file HTML
html_path = Path("dependency_tree.html")
html_path.write_text(html, encoding="utf-8")

print("Diagram dependency tree berhasil disimpan!")
print("Lokasi file:", html_path.resolve())
```

✓ 1.7s




```
Diagram dependency tree berhasil disimpan!
Lokasi file: D:\NLP\Modul-1\pertemuan-04\dependency_tree.html
```



```
Modul-1 > pertemuan-04 > <> dependency_tree.html > ...
<!DOCTYPE html>
<html lang="id">
    <head>
        <title>displaCy</title>
    </head>

    <body style="font-size: 16px; font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Helvetica, Arial,
<figure style="margin-bottom: 6rem">
<svg xmlns="http://www.w3.org/2000/svg" xmlns:xlink="http://www.w3.org/1999/xlink" xml:lang="id" id="47795a94d
<text class="displacy-token" fill="currentColor" text-anchor="middle" y="134.5">
    <tspan class="displacy-word" fill="currentColor" x="50">Mahasiswa</tspan>
    <tspan class="displacy-tag" dy="2em" fill="currentColor" x="50">PROPN</tspan>
</text>
```

Perintah `from spacy import displacy serta from pathlib import Path` untuk mengimpor pustaka visualisasi dan pengolahan jalur berkas . Kalimat uji coba kemudian diproses ke dalam objek dokumen menggunakan `doc = nlp("Mahasiswa belajar Natural Language Processing.")` dan langsung divisualisasikan di dalam notebook Jupyter lewat perintah `displacy.render(doc, style="dep", jupyter=True, options={"distance": 100}).`

Selanjutnya, struktur pohon keterikatan (*dependency tree*) dikonversi menjadi format HTML penuh dengan memanggil fungsi `html = displacy.render(doc, style="dep", page=True, jupyter=False).` Berkas HTML tersebut disimpan ke media penyimpanan menggunakan

perintah `html_path = Path("dependency_tree.html")` dan `html_path.write_text(html, encoding="utf-8").` Kode diakhiri dengan perintah `print("Diagram dependency tree berhasil disimpan!")` serta `print("Lokasi file:", html_path.resolve())` untuk menampilkan pesan konfirmasi beserta lokasi absolut dari berkas HTML yang telah dibuat.

9. Analisis Dataset

```python
# Analisis Dataset
import pandas as pd

df = pd.read_csv(r"D:\NLP\Modul-1\pertemuan-03\dataset_preprocessed.csv")
text_column = "content"  # sesuaikan

# Ambil sample untuk analisis
sample = df.sample(10, random_state=42)

for _, row in sample.iterrows():
    text = str(row[text_column])[:200]  # batasi panjang
    doc = nlp(text)
    pos_counts = {}
    for token in doc:
        pos = token.pos_
        pos_counts[pos] = pos_counts.get(pos, 0) + 1
    print(f"Teks: {text[:80]}...")
    print(f"POS Distribution: {pos_counts}")
    print()
```

✓ 14.9s Python

```
Teks: Sangat membantu dalam menyimpan uang Dhe bets...
POS Distribution: {'ADV': 1, 'VERB': 2, 'ADP': 1, 'NOUN': 1, 'PROPN': 2}

Teks: ok...
POS Distribution: {'PROPN': 1}

Teks: Tidak mudah ribet...
POS Distribution: {'PART': 1, 'ADJ': 2}

Teks: sangat membantu untuk saya seorang driver...
POS Distribution: {'ADV': 1, 'VERB': 1, 'ADP': 1, 'PRON': 1, 'DET': 1, 'PROPN': 1}

Teks: Apk terbaik...
POS Distribution: {'NOUN': 1, 'ADJ': 1}

Teks: Mau tf susah, hrus upgrade dl,, kl g mau g bisa,,...
POS Distribution: {'AUX': 2, 'PROPN': 1, 'ADJ': 1, 'PUNCT': 5, 'NOUN': 4, 'X': 2, 'VERB': 1}

Teks: Good...
POS Distribution: {'PROPN': 1}

Teks: keren...
POS Distribution: {'ADJ': 1}

Teks: Pengen aja kasih bintang 5 mau tau nanti dapat apa dari server...
...

Teks: Pembayaran tagihan kredit via tokopedia kenapa sekarang ga bisa pakai gopay, apa...
POS Distribution: {'NOUN': 7, 'PROPN': 1, 'ADV': 4, 'PART': 1, 'AUX': 2, 'VERB': 3, 'PUNCT': 2, 'SCONJ':

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Perintah `import pandas as pd` untuk mengimpor pustaka Pandas. Seluruh berkas dataset kemudian dimuat menggunakan `df = pd.read_csv(r"D:\NLP\Modul1\pertemuan03\dataset_preprocessed.csv")` dan nama kolom teks ditentukan melalui `text_column =`

`"content"`. Pengambilan sampel dataset dilakukan secara acak sebanyak 10 baris menggunakan perintah `sample = df.sample(10, random_state=42)`.

Selanjutnya, perulangan `for _, row in sample.iterrows()`: digunakan untuk memproses setiap sampel ulasan secara individu. Di dalam perulangan, panjang teks dibatasi hingga 200 karakter pertama lewat `text = str(row[text_column])[:200]`, lalu diubah menjadi objek `doc = nlp(text)`. Variabel kamus `pos_counts = {}` diinisialisasi untuk menghitung frekuensi label Part-of-Speech (POS). Melalui perulangan `for token in doc:`, label kelas kata diekstrak menggunakan `pos = token.pos_` dan dihitung jumlah kemunculannya secara berkelanjutan via `pos_counts[pos] = pos_counts.get(pos, 0) + 1`.

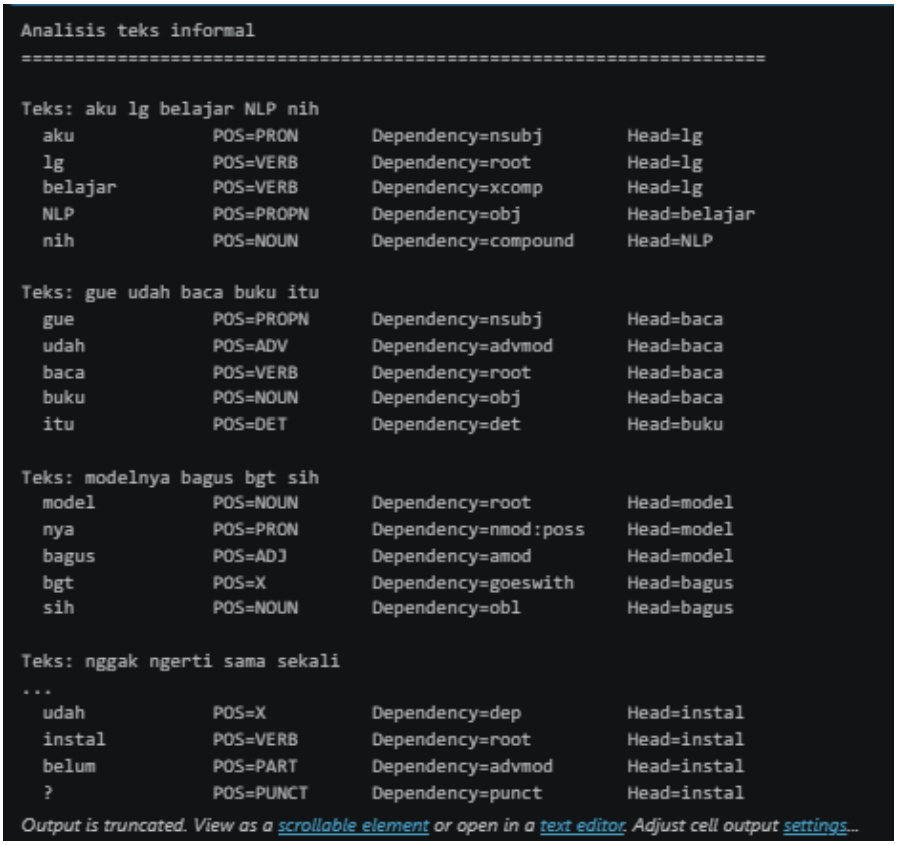
Kode diakhiri dengan mencetak potongan teks ulasan hingga 80 karakter via `print(f"Teks: {text[:80]}...")`, menampilkan distribusi tag POS dalam ulasan tersebut lewat `print(f"POS Distribution: {pos_counts}")`, serta memberikan baris kosong melalui `print()` sebagai pemisah antar sampel.

10. Analisis Kesalahan

```
# Analisis Kesalahan
informal_texts = [
    "aku lg belajar NLP nih",
    "gue udah baca buku itu",
    "modelnya bagus bgt sih",
    "nggak ngerti sama sekali",
    "udah instal belum?"
]

print("Analisis teks informal")
print("=" * 70)
for text in informal_texts:
    doc = nlp(text)
    print(f"\nTeks: {text}")
    for token in doc:
        print(
            f"  {token.text:<15} POS={token.pos_:<10} "
            f"Dependency={token.dep_:<12} Head={token.head.text}"
        )
```

✓ 6.1s



Perintah `informal_texts = ["aku lg belajar NLP nih", "gue udah baca buku itu", "modelnya bagus bgt sih", "nggak ngerti sama sekali", "udah instal belum?"]` untuk mendefinisikan *list* berisi lima kalimat informal atau gaul. Selanjutnya, perintah `print("Analisis teks informal")` dan `print("=" * 70)` mencetak judul analisis beserta garis pemisah horizontal.

Proses dilanjutkan dengan perulangan `for text in informal_texts:` untuk mengiterasi setiap kalimat, memproses teks menjadi objek dokumen lewat `doc = nlp(text)`, serta menampilkan teks yang sedang diuji melalui `print(f"\nTeks: {text}")`.

Kode diakhiri dengan perulangan bersarang for token in doc: yang mengeksekusi perintah `print(f" {token.text:<15} POS={token.pos_:<10} f"Dependency={token.dep_:<12} Head={token.head.text}")` untuk menampilkan detail hasil ekstraksi setiap token, mencakup kata asli, tag Part-of-Speech (`POS`), relasi *dependency* (`Dependency`), serta kata induknya (`Head`).

E. Tugas Analisis

```python
# Tugas Analisis Linguistik

hasil_ringkasan = []

# Pastikan sample berisi 10 teks dari kolom content
sample_teks = (
    df["content"]
    .dropna()
    .astype(str)
    .sample(n=10, random_state=42)
    .reset_index(drop=True)
)

for nomor, text in enumerate(sample_teks, start=1):
    text = text[:200]
    df_result = analyze_pos(text, nlp)

    hasil_ringkasan.append({
        "No": nomor,
        "Kalimat": text,
        "Token": ", ".join(
            df_result["Token"].astype(str)
        ),
        "POS": ", ".join(
            df_result["POS"].astype(str)
        ),
        "Dependency": "; ".join(
            df_result["Token"].astype(str)
            + " → "
            + df_result["Head"].astype(str)
            + " ("
            + df_result["Dependency"].astype(str)
            + ")"
        ),
        "Kesalahan?": "Perlu pemeriksaan manual"
    })

df_tugas = pd.DataFrame(hasil_ringkasan)

# Tampilkan tabel satu kali tanpa indeks 0-9
display(df_tugas.style.hide(axis="index"))

print("Jumlah teks dianalisis:", len(df_tugas))
```

No	Kalimat	Token	POS	Dependency	Kesalahan?
1	Sangat membantu dalam menyimpan uang Dhe bets	Sangat, membantu, dalam, menyimpan, uang, Dhe, bets	ADV, VERB, ADP, VERB, NOUN, PROPN, PROPN	Sangat → membantu (advmod); membantu → membantu (root); dalam → menyimpan (case); menyimpan → membantu (xcomp); uang → menyimpan (obj); Dhe → uang (nmod); bets → Dhe (flat:name)	Perlu pemeriksaan manual
2	ok	ok	PROPN	ok → ok (root)	Perlu pemeriksaan manual
3	Tidak mudah ribet	Tidak, mudah, ribet	PART, ADJ, ADJ	Tidak → ribet (advmod); mudah → ribet (amod); ribet → ribet (root)	Perlu pemeriksaan manual
4	sangat membantu untuk saya seorang driver	sangat, membantu, untuk, saya, seorang, driver	ADV, VERB, ADP, PRON, DET, PROPN	sangat → membantu (advmod); membantu → membantu (root); untuk → saya (case); saya → membantu (obl); seorang → driver (det); driver → saya (acl)	Perlu pemeriksaan manual
5	Apk terbaik	Apk, terbaik	NOUN, ADJ	Apk → Apk (root); terbaik → Apk (amod)	Perlu pemeriksaan manual
6	Mau tf susah, hrus upgrade dl,, kl g mau g bisa,,	Mau, tf, susah, ,, hrus, upgrade, dl, ,, ,, kl, g, mau, g, bisa, ,, ,	AUX, PROPN, ADJ, PUNCT, NOUN, NOUN, NOUN, PUNCT, PUNCT, NOUN, X, VERB, X, AUX, PUNCT, PUNCT	Mau → tf (aux); tf → tf (root); susah → tf (amod); , → hrus (punct); hrus → tf (conj); upgrade → hrus (compound); dl → upgrade (compound); , → mau (punct); , → mau (punct); kl → mau (nsubj); g → kl (goeswith); mau → tf (parataxis); g → mau (goeswith); bisa → mau (advmod:emph); , → mau (punct); , → tf (punct)	Perlu pemeriksaan manual
7	Good	Good	PROPN	Good → Good (root)	Perlu pemeriksaan manual
8	keren	keren	ADJ	keren → keren (root)	Perlu pemeriksaan manual
9	Pengen aja kasih bintang 5 mau tau nanti dapat apa dari server	Pengen, aja, kasih, bintang, 5, mau, tau, nanti, dapat, apa, dari, server	ADV, VERB, NOUN, NOUN, NUM, AUX, VERB, ADV, AUX, PRON, ADP, NOUN	Pengen → aja (advmod); aja → tau (advcl); kasih → tau (nsubj); bintang → kasih (compound); 5 → bintang (nummod); mau → tau (aux); tau → tau (root); nanti → apa (advmod); dapat → apa (aux); apa → tau (obj); dari → server (case); server → apa (nmod)	Perlu pemeriksaan manual
10	Pembayaran tagihan kredit via tokopedia kenapa sekarang ga bisa pakai gopay, apa karena tokopedia sudah dijual ke tiktok. Semakin merepotkan	Pembayaran, tagihan, kredit, via, tokopedia, kenapa, sekarang, ga, bisa, pakai, gopay, ,, apa, karena, tokopedia, sudah, dijual, ke, tiktok, ., Semakin, merepotkan	NOUN, NOUN, NOUN, PROPN, NOUN, ADV, ADV, PART, AUX, VERB, NOUN, PUNCT, ADV, SCONJ, NOUN, AUX, VERB, ADP, NOUN, PUNCT, ADV, VERB	Pembayaran → pakai (nsubj); tagihan → Pembayaran (compound); kredit → tagihan (compound); via → kredit (nmod); tokopedia → via (compound); kenapa → pakai (advmod); sekarang → pakai (advmod); ga → pakai (advmod); bisa → pakai (aux); pakai → pakai (root); gopay → pakai (obj); , → dijual (punct); apa → dijual (advmod); karena → dijual (mark); tokopedia → dijual (nsubj:pass); sudah → dijual (aux); dijual → pakai (advcl); ke → tiktok (case); tiktok → dijual (obl); . → pakai (punct); Semakin → merepotkan (advmod); merepotkan → merepotkan (root)	Perlu pemeriksaan manual

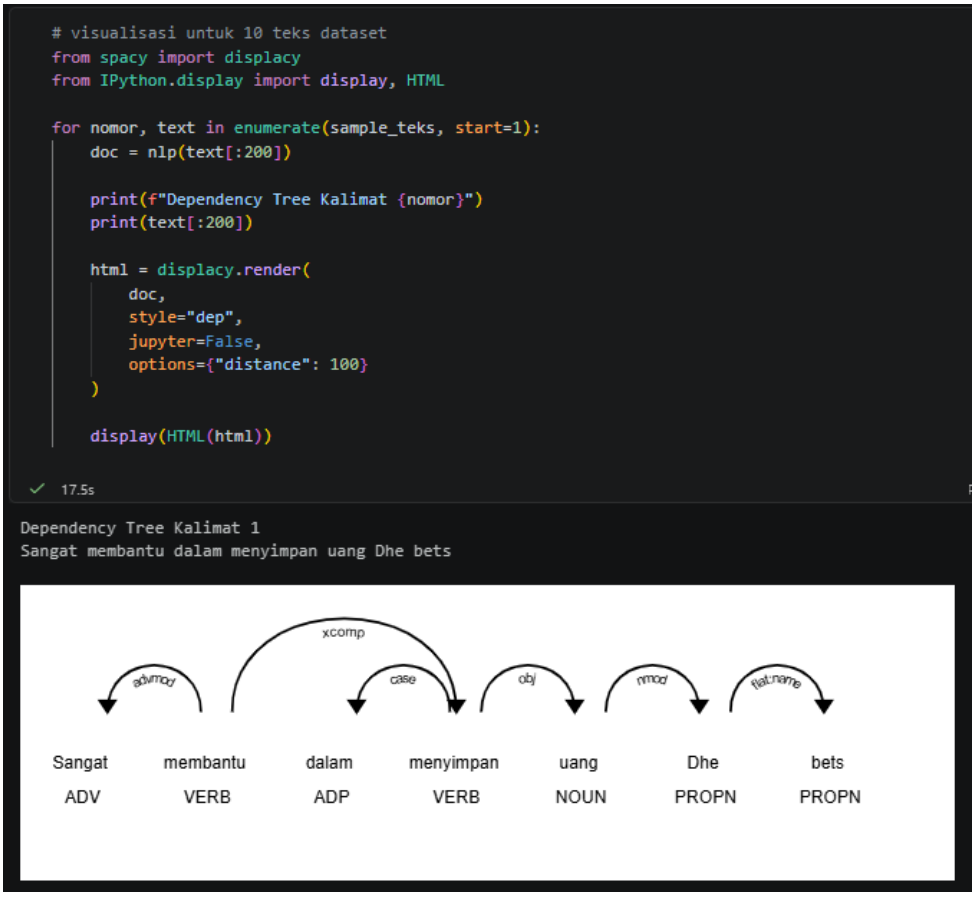
```
Jumlah teks dianalisis: 10
```

Perintah `hasil_ringkasan = []` untuk menampung hasil ringkasan analisis. Selanjutnya, 10 sampel teks diambil dari kolom `content` pada DataFrame df menggunakan rantai perintah `sample_teks = (df["content"].dropna().astype(str).sample(n=10, random_state=42).reset_index(drop=True))` yang berfungsi menghapus data kosong melalui `.dropna()`, mengonversi tipe data ke string melalui `.astype(str)`, mengambil 10 sampel secara acak melalui `.sample(n=10, random_state=42)`, serta mereset urutan indeks melalui `.reset_index(drop=True)`.

Proses dilanjutkan dengan perulangan `for nomor, text in enumerate(sample_teks, start=1)`: untuk mengiterasi setiap ulasan mulai dari indeks nomor 1. Di dalam perulangan, panjang ulasan dibatasi hingga 200 karakter pertama via `text = text[:200]`, lalu diproses ke dalam fungsi analisis melalui `df_result = analyze_pos(text, nlp)`. Setiap hasil ekstraksi

dimasukkan ke dalam *list* ringkasan menggunakan perintah `hasil_ringkasan.append({...})` dalam format kamus (*dictionary*) yang berisi nomor urut via `"No": nomor`, teks ulasan via `"Kalimat": text`, deretan token yang digabung koma via `"Token": ", ".join(df_result["Token"].astype(str))`, deretan label kelas kata via `"POS": ", ".join(df_result["POS"].astype(str))`, struktur relasi sintaksis lengkap via `"Dependency": "; ".join(...)`, dan label evaluasi via `"Kesalahan?": "Perlu pemeriksaan manual"`.

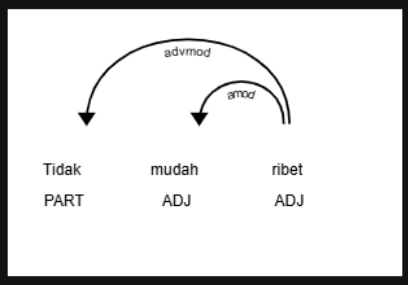
Kode diakhiri dengan mengonversi seluruh daftar hasil ringkasan menjadi DataFrame baru menggunakan perintah `df_tugas = pd.DataFrame(hasil_ringkasan)`. Tabel tersebut kemudian ditampilkan ke antarmuka notebook tanpa menyertakan kolom indeks bawaan melalui perintah `display(df_tugas.style.hide(axis="index"))`, lalu diakhiri dengan mencetak total jumlah ulasan yang telah diproses menggunakan perintah `print("Jumlah teks dianalisis:", len(df_tugas))`.



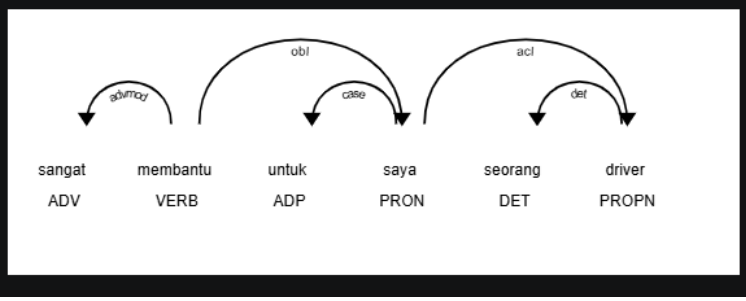
Dependency Tree Kalimat 2
ok



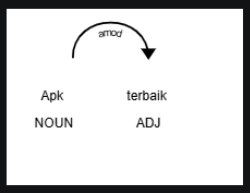
Dependency Tree Kalimat 3
Tidak mudah ribet



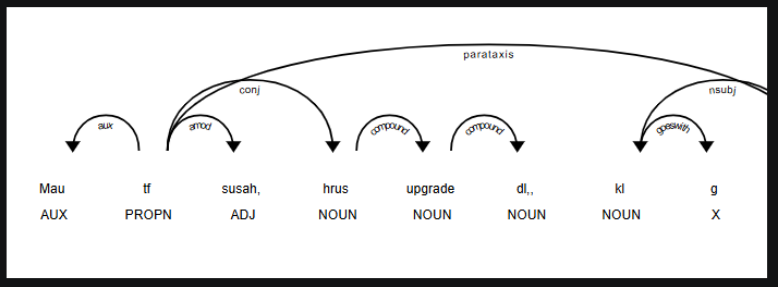
Dependency Tree Kalimat 4
sangat membantu untuk saya seorang driver



Dependency Tree Kalimat 5
Apk terbaik

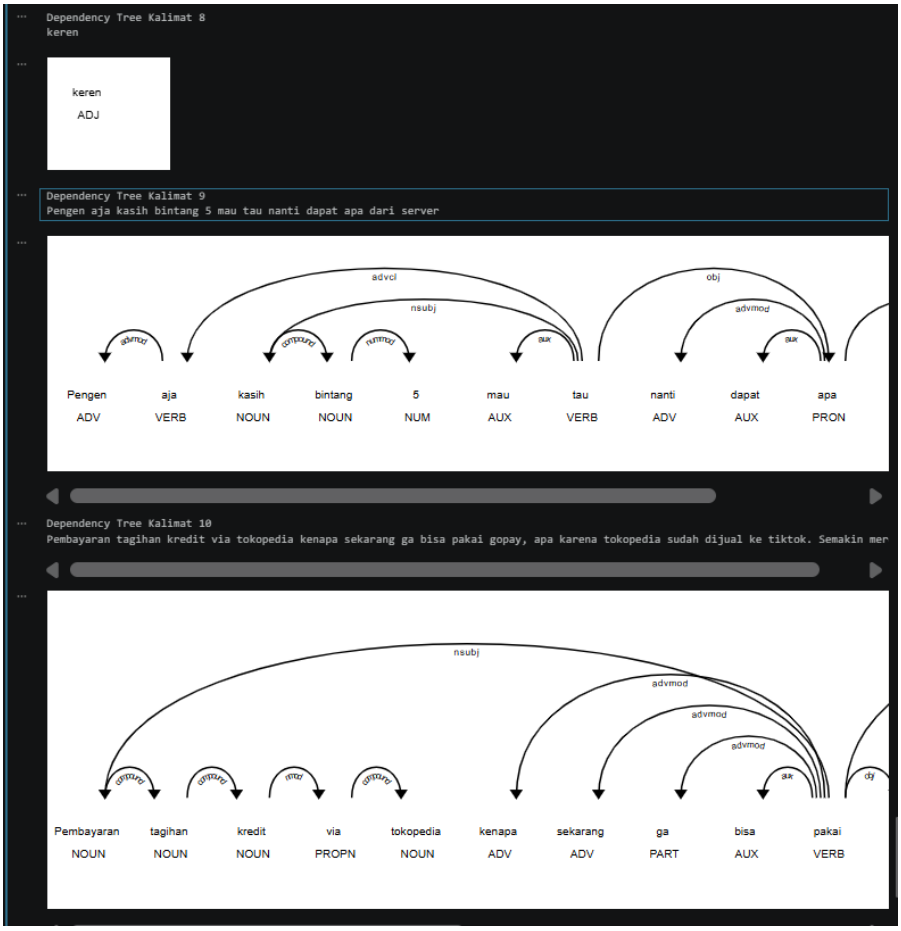


Dependency Tree Kalimat 6
Mau tf susah, hrus upgrade dl,, kl g mau g bisa,,



Dependency Tree Kalimat 7
Good





Perintah `from spacy import displacy` serta `from IPython.display import display, HTML` untuk mengimpor modul visualisasi dan penampil HTML Jupyter. Selanjutnya, perulangan for nomor, `text in enumerate(sample_teks, start=1):` digunakan untuk memproses setiap teks ulasan dalam variabel `sample_teks` secara berurutan mulai dari indeks nomor 1. Di dalam perulangan, teks ulasan dibatasi hingga 200 karakter pertama dan diproses menjadi objek dokumen lewat perintah `doc = nlp(text[:200]).` Perintah `print(f"Dependency Tree Kalimat {nomor}")` dan `print(text[:200])` kemudian mengeksekusi pencetakan nomor urut beserta isi potongan teks ulasan yang sedang dianalisis. Proses visualisasi dilakukan melalui fungsi `html = displacy.render(doc, style="dep", jupyter=False, options={"distance": 100})` yang merender diagram *dependency tree* kalimat ke dalam bentuk string HTML dengan jarak antar token sebesar 100 piksel. Kode diakhiri dengan perintah `display(HTML(html))` yang memuat dan menampilkan diagram visual struktur sintaksis kalimat tersebut secara langsung pada antarmuka notebook.

F. Pertanyaan Diskusi

1. Apakah POS Tagging selalu menghasilkan label yang benar?

   Jawaban:

   Tidak, Part-of-Speech (POS) Tagging tidak selalu menghasilkan label yang 100% benar. Akurasi *tagger* sangat bergantung pada algoritma yang digunakan, kualitas serta keterwakilan *corpus* pelatihan (misalnya *Indonesian Treebank*), serta konteks kalimat. Ambiguitas kata (seperti kata "bisa" yang dapat bertindak sebagai kata kerja *dapat* atau kata benda *racun*) sering kali memicu kesalahan pelabelan jika konteks sekitarnya kurang mendukung.

2. Apa penyebab utama kesalahan POS Tagging pada teks Bahasa Indonesia informal?

   Jawaban:

   - Ketidakcocokan *Corpus* Pelatihan: Kebanyakan model POS Tagging bahasa Indonesia dilatih menggunakan teks formal (seperti berita online), sehingga gagal mengenali pola sintaksis informal.
   - Variasi Tata Bahasa & Singkatan: Penggunaan struktur kalimat yang tidak baku, penghilangan subjek/objek, serta penggunaan singkatan (seperti *yg*, *dgn*, *tdk*, *stlh*).
   - Tipe Kesalahan Ketik (*Typos*): Kesalahan penulisan yang acak membuat model menganggap kata tersebut sebagai *Out-of-Vocabulary* (OOV) atau kata asing.

3. Bagaimana slang mempengaruhi POS Tagging?

   Jawaban:

   Kata-kata *slang* (seperti *baper*, *mager*, *gercep*, *gokil*) atau variasi kata informal (seperti *pake* bukannya *pakai*) berdampak pada POS Tagging dalam dua hal:

   - Gagal Dikenali (OOV): Model yang tidak memiliki kata *slang* dalam kosakata bawaannya akan bingung dan memetakan kata tersebut ke label *default* (sering kali ditandai sebagai *Noun* atau *Foreign Word*).
   - Pergeseran Fungsi Kata: Beberapa kata *slang* diciptakan dari gabungan dua kata (misal: *baper* dari *bawa perasaan*) yang mengubah fungsionalitas tata bahasanya dari frasa menjadi satu kata sifat/kerja baru.

4. Bagaimana nama organisasi (GOTO, Tokopedia) mempengaruhi POS Tagging?

Jawaban:

Nama organisasi atau *Proper Noun* sering kali mempengaruhi POS Tagging secara signifikan:

- Risiko Pelabelan Salah: Jika nama organisasi mirip dengan kata biasa atau menggunakan bahasa Inggris (misalnya *GoPay*, *GoTo*, *OVO*), model POS Tagging yang kurang adaptif bisa salah mengira nama tersebut sebagai kata kerja (*Verb*) atau kata asing (*Foreign Word*) bukannya *Proper Noun* (NNP).
- Efek Konteks Kapitalisasi: Pada teks ulasan atau media sosial di mana huruf kapital sering diabaikan (semua ditulis huruf kecil), model yang sangat mengandalkan huruf kapital untuk mendeteksi entitas nama akan gagal mengidentifikasinya sebagai *Proper Noun*.

5. Mengapa parsing penting dalam NLP? Berikan contoh aplikasi yang memerlukannya.

   Jawaban:

   Parsing penting karena bertugas menganalisis struktur sintaksis dan hubungan hierarki/keterikatan (*dependency*) antar kata dalam kalimat, bukan sekadar melihat kata secara terisolasi. Ini membantu sistem memahami "siapa melakukan apa kepada siapa".

   Contoh aplikasi yang sangat memerlukan *parsing*:

   - Machine Translation (Penerjemahan Bahasa): Memastikan susunan kata dalam bahasa sumber diterjemahkan sesuai aturan tata bahasa target (misal: penyesuaian pola DM bahasa Indonesia ke MD bahasa Inggris).
   - Question Answering (Sistem Tanya-Jawab): Membantu mesin mengekstrak entitas mana yang menjadi subjek pertanyaan dan mana yang menjadi jawaban.
   - Information Extraction & Relation Extraction: Mengidentifikasi hubungan antar-entitas dalam teks (misal: menemukan hubungan *[Perusahaan X] -- [akuisisi] --> [Perusahaan Y]*).

6. Apa hubungan morphology dengan stemming?

   Jawaban:

Morphology (morfologi) adalah cabang ilmu linguistik yang mempelajari struktur internal kata dan pembentukan kata dari unit-unit terkecil bertanda makna (*morfem*, seperti kata dasar, awalan, akhiran, dan sisipan).

Stemming adalah implementasi komputasi/algoritma yang memanfaatkan prinsip-prinsip morfologi tersebut untuk memotong imbuhan (*affixation*) pada suatu kata sehingga kembali ke bentuk dasarnya (*root/stem*). Sebagai contoh, algoritma *stemmer* bahasa Indonesia (seperti Sastrawi) menggunakan aturan morfologi bahasa Indonesia untuk membuang awalan *me-* dan akhiran *-kan* dari kata "memuat" menjadi "buat".

7. Apakah stemming dan POS Tagging menyelesaikan permasalahan yang sama?

   Jawaban:

   Tidak, keduanya menyelesaikan permasalahan yang berbeda dalam *pipeline* NLP:

   - Stemming berfokus pada bentuk fisik/morfologi kata dengan menghasilkan kata dasar (akar kata/string yang dipotong). Tujuannya adalah mengurangi variasi kata (*vocabulary size*) agar kata-kata berimbuhan dianggap identik dalam analisis.
   - POS Tagging berfokus pada pemahaman fungsi tata bahasa kata dalam kalimat dengan menghasilkan label kategori kata (*Noun*, *Verb*, *Adjective*, dll). Tujuannya adalah memberikan konteks sintaksis untuk analisis struktur dan makna kalimat.

   Stemming beroperasi pada tingkat kata secara individu (morfologi), sedangkan *POS Tagging* beroperasi pada tingkat kalimat dengan mempertimbangkan kata-kata di sekitarnya (sintaksis & konteks).

G. Kesimpulan

Kesimpulan yang dapat diperoleh dari praktikum ini adalah:

1. Praktikum ini berhasil mengimplementasikan tahapan *linguistic processing* Bahasa Indonesia—mulai dari tokenisasi, lematisasi, *POS Tagging*, hingga *Dependency Parsing*—menggunakan pustaka `spacy-stanza` dan visualisasi `displacy`.
2. *POS Tagging* efektif mengidentifikasi kelas kata (`NOUN`, `VERB`, dll.), sedangkan *Dependency Parsing* mampu memetakan relasi antar-kata (*siapa melakukan apa*) yang penting untuk aplikasi penerjemahan maupun ekstraksi informasi.
3. Model yang dilatih pada korpus formal mengalami kendala (*Out-of-Vocabulary*) saat mengolah teks informal, singkatan, dan *slang*, sehingga berpotensi menghasilkan kesalahan pelabelan kelas kata.
4. *Stemming/Lemmatization* berfokus pada penyederhanaan bentuk fisik kata ke kata dasar (morfologi), sedangkan *POS Tagging* berfokus pada penentuan fungsi gramatikal kata berdasarkan konteks kalimatnya (sintaksis).

LAPORAN PRAKTIKUM

NATURAL LANGUAGE PROCESSOR

Annisa Almaghvirah

240306037

https://github.com/ANNISAALMAGHVIRAH

Program Studi Teknologi Informasi

Fakultas Dakwah dan Ilmu Komunikasi

Universitas Islam Negeri Mataram

2026

Modul	Modul 1 - Fundamental dan Text Processing
Topik	N-Gram dan Language Model
Repository	NATURAL-LANGUAGE-PROCESSING/Annisa Almaghvirah_240306037_Modul1 at main · ANNISAALMAGHVIRAH/NATURAL-LANGUAGE-PROCESSING
Tanggal	13 Oktober 2026

A. Tujuan

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Memahami konsep language model
2. Memahami probabilitas kata dalam sequence
3. Membuat unigram, bigram, dan trigram
4. Menghitung conditional probability
5. Menghitung probabilitas kalimat
6. Memahami masalah zero probability dan smoothing
7. Membuat prediksi kata sederhana
8. Menghitung perplexity
9. Memahami keterbatasan n-gram

B. Requiretment

1. Sistem Operasi yang digunakan : Windows 11 Home Single Language 64-bit (10.0, Build 26100)
2. Browser : Google Chrome Version 130.0.6723.117 (Official Build) (64-bit)
3. Tools yang digunakan : Visual Studio Code, Python , Jupyter Notebook, Virtual Environment (.venv), pandas, numpy, matplotlib, seaborn, nltk

C. Dasar Teori

1. Apa Itu Language Model?

   Language model (model bahasa) mencoba menjawab pertanyaan:

   Seberapa mungkin sebuah sequence kata muncul?

   Contoh:

   ```
   P("Saya belajar NLP")   → probabilitas tinggi
   P("belajar Saya NLP")   → probabilitas rendah
   P("xkjz qqrt mnop")     → probabilitas mendekati nol
   ```

   Aplikasi language model:

   - Autocomplete (prediksi kata berikutnya)
   - Spell checker
   - Machine translation
   - Text generation

2. Probabilitas Sequence

   Secara matematis, probabilitas sebuah sequence kata:

   $$P(w_1, w_2, .., .. w_n)$$

   dapat diuraikan menggunakan chain rule:

   $$P(w_1) \cdot P(w_2|w_1) \cdot P(w_3|w_1|w_2) \cdot \ldots \cdot P(w_n|w_1|.|..w_{n-1})$$

3. N-Gram

   N-gram adalah sequence berurutan dari N token.

N	Nama	Contoh
1	Unigram	`saya` , `belajar` , `nlp`
2	Bigram	`saya belajar` , `belajar nlp`
3	Trigram	`saya belajar nlp`
N	N-gram	sequence N token berturut-turut

4. Asumsi Markov

   N-gram menggunakan asumsi Markov: probabilitas kata berikutnya hanya bergantung pada N-1 kata sebelumnya.

   Bigram (Markov order 1):

$$P(w_n|w_1|.|..w_{n-1}) \approx P(w_n|w_{n-1})$$

Trigram (Markov order 2):

$$P(w_n|w_1|.|..w_{n-1}) \approx P(w_n|w_{n-2}|w_{n-1})$$

5. Probabilitas Unigram

$$P(w) = \frac{Count(w)}{N}$$

dengan N = total token dalam corpus.

6. Probabilitas Bigram (Conditional Probability)

$$P(w_n|w_{n-1}) = \frac{Count(w_{n-1}, w_n)}{Count(w_{n-1})}$$

7. Masalah Zero Probability

Jika bigram (deep, learning) tidak pernah muncul dalam corpus:

```
P(learning | deep) = 0
```

Akibatnya probabilitas seluruh kalimat menjadi nol, meskipun kalimatnya masuk akal secara linguistik.

8. Laplace Smoothing

Smoothing digunakan untuk menghindari probabilitas nol dengan menambahkan nilai kecil pada semua frekuensi:

$$P(w_i|w_{i-1}) = \frac{Count(w_{i-1}, w_i) + 1}{Count(w_{i-1}) + V}$$

dengan V = ukuran vocabulary (jumlah token unik).

9. Perplexity

Perplexity mengukur seberapa baik language model memprediksi data:

$$PP(W) = P(W)^{-1/N}$$

Perplexity lebih rendah menunjukkan model lebih baik dalam memprediksi data evaluasi (dengan dataset dan model pembanding yang sama).

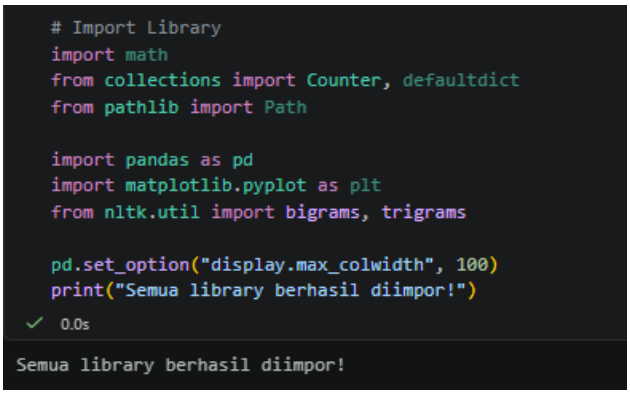
D. Implementasi

1. Membuat Notebook

   Buat file:

   ```
   pertemuan-05/ngram_language_model.ipynb
   ```

2. Import Library



   Kode diawali mengimpor modul matematika, penghitung frekuensi, penanganan jalur berkas, manipulasi data, visualisasi, dan pembuat n-gram melalui perintah `import math`, `from collections import Counter, defaultdict`, `from pathlib import Path`, `import pandas as pd`, `import matplotlib.pyplot as plt`, serta `from nltk.util import bigrams, trigrams`.

   Selanjutnya, pengaturan tampilan Pandas dikonfigurasi melalui perintah `pd.set_option("display.max_colwidth", 100)` untuk membatasi lebar maksimum kolom teks menjadi 100 karakter.

   Kode diakhiri dengan mengeksekusi perintah `print("Semua library berhasil diimpor!")` yang menampilkan konfirmasi bahwa seluruh modul telah siap digunakan.

3. Load Dataset

```
# Load Dataset
data = pd.read_csv("D:/NLP/Modul-1/pertemuan-03/dataset_preprocessed.csv")
text_column = "processed_text"  # gunakan hasil preprocessing

# Periksa nama kolom
print("Nama kolom:",data.columns.tolist())
display(data.head())

# Hapus nilai kosong
data = data.dropna(subset=[text_column])
print(f"Jumlah dokumen: {len(data)}")
```

✓ 0.6s

```
Nama kolom: ['content', 'processed_text']
```

	content	processed_text
0	Mantap	mantap
1	Bagus dan mudah digunakan	bagus mudah
2	Good	good
3	Sangat membantu orang menengah mantap semoga sukses terus gopay	bantu orang tengah mantap moga sukses gopay
4	Ga ribet .. Sukaaaa	ga ribet sukaaaa

```
Jumlah dokumen: 119676
```

Perintah pemuatan dan pembersihan dataset diawali dengan membaca file CSV melalui perintah `pd.read_csv("D:/NLP/Modul-1/pertemuan-03/dataset_preprocessed.csv")` yang disimpan ke dalam variabel `data`, lalu ditentukan kolom `"processed_text"` sebagai acuan data teks.

Setelah itu, dilakukan pemeriksaan struktur dengan mencetak daftar nama kolom `(data.columns.tolist())` serta menampilkan 5 baris teratas menggunakan `display(data.head())`. Proses dilanjutkan dengan menghapus baris yang memiliki nilai kosong pada kolom target melalui `data.dropna(subset=[text_column])`.

Terakhir, jumlah dokumen yang tersisa dihitung dan ditampilkan ke layar, yaitu sebanyak 119.676 dokumen siap olah.

4. Membuat Corpus Token

```
# Membuat Corpus Token

# Gabungkan seluruh teks menjadi satu corpus:
all_text = " ".join(data[text_column].astype(str))
tokens = all_text.split()

print(f"Total token: {len(tokens)}")
print(f"Contoh token: {tokens[:20]}")
print("Jumlah kata unik:", len(set(tokens)))
```

✓ 0.9s

```
Total token: 815364
Contoh token: ['Mantap', 'Bagus', 'dan', 'mudah', 'digunakan', 'Good', 'Sangat', 'membantu', 'orang', 'menengah', 'mantap', 'semoga', 'sukses', 'terus', 'gopay', 'Ga', 'ribet', '..', 'Sukaaaa', 'Sangat']
Jumlah kata unik: 68523
```

Perintah pembuatan korpus token diawali dengan menggabungkan seluruh teks pada kolom `data[text_column]` menjadi satu string tunggal menggunakan `" ".join(data[text_column].astype(str))` yang disimpan dalam variabel `all_text`. Teks tunggal tersebut kemudian dipecah

berdasarkan spasi menggunakan fungsi `.split()` untuk menghasilkan daftar kata dalam variabel tokens.

Setelah itu, dilakukan perhitungan statistik dengan mencetak total token `(len(tokens))` yang berjumlah 815.364, menampilkan 20 contoh token pertama `(tokens[:20])`, serta menghitung jumlah kata unik menggunakan len(set(tokens)) yang menghasilkan 68.523 kata unik.

5. Membuat Unigram

```
#  Membuat Unigram
unigram = Counter(tokens)

print("20 Kata Paling Sering Muncul:")
display(pd.DataFrame(unigram.most_common(20), columns=["Kata", "Frekuensi"]))
```

✓ 0.2s

20 Kata Paling Sering Muncul:

	Kata	Frekuensi
0	gopay	20327
1	aplikasi	15681
2	bagus	14955
3	mantap	13770
4	mudah	13351
5	bantu	11089
6	nya	8868
7	transaksi	7587
8	ok	6879
9	transfer	6219
10	bayar	5862
11	banget	5255
12	cepat	5031
13	good	4765
14	ga	4565
15	saldo	4448
16	gak	4384
17	yg	4317
18	apk	3538
19	aja	3279

Kode tersebut diawali dengan menghitung frekuensi kemunculan setiap kata tunggal melalui perintah `unigram = Counter(tokens)`. Selanjutnya, perintah `print("20 Kata Paling Sering Muncul:")` mengeksekusi pencetakan teks judul hasil keluaran.

Kode diakhiri dengan mengambil 20 kata terbanyak menggunakan fungsi `.most_common(20)` dan mengonversinya menjadi bentuk tabel via perintah `display(pd.DataFrame(unigram.most_common(20),columns=["Kata" , "Frekuensi"]))` untuk menampilkan pasangan kata beserta jumlah frekuensi kemunculannya secara rapi.

6. Probabilitas Unigram



```
# Probabilitas Unigram
total_tokens = sum(unigram.values())

unigram_probability = {
    word: count / total_tokens
    for word, count in unigram.items()
}

print("\n10 Kata dengan Probabilitas Tertinggi:")
sorted_prob = sorted(
    unigram_probability.items(),
    key=lambda x: x[1],
    reverse=True
)
for word, prob in sorted_prob[:10]:
    print(f"  {word:<20} : {prob:.6f}")


print("Lima contoh probabilitas unigram:")
for word, count in unigram.most_common(5):
    print(f"P({word}) = {count}/{total_tokens} = {unigram_probability[word]:.8f}")
```

✓ 0.1s

```
10 Kata dengan Probabilitas Tertinggi:
  gopay                : 0.037785
  aplikasi             : 0.029149
  bagus                : 0.027799
  mantap               : 0.025596
  mudah                : 0.024818
  bantu                : 0.020613
  nya                  : 0.016484
  transaksi            : 0.014103
  ok                   : 0.012787
  transfer             : 0.011560
Lima contoh probabilitas unigram:
P(gopay) = 20327/537966 = 0.03778492
P(aplikasi) = 15681/537966 = 0.02914868
P(bagus) = 14955/537966 = 0.02779915
P(mantap) = 13770/537966 = 0.02559641
P(mudah) = 13351/537966 = 0.02481755
```

Kode tersebut diawali dengan menghitung total keseluruhan token menggunakan `total_tokens = sum(unigram.values())`. Probabilitas setiap unigram kemudian dihitung melalui *dictionary comprehension* `unigram_probability = {word: count / total_tokens for word, count in unigram.items()}` dengan membagi jumlah munculnya kata terhadap total token.

Selanjutnya, perintah `print("\n10 Kata dengan Probabilitas Tertinggi:")` mengeksekusi pencetakan teks judul. Daftar probabilitas diurutkan secara menurun (*descending*) lewat perintah `sorted_prob = sorted(unigram_probability.items(), key=lambda x: x[1], reverse=True)`. Perulangan `for word, prob in sorted_prob[:10]:` lalu mencetak 10 kata teratas beserta probabilitasnya yang diformat hingga 6 digit desimal melalui `print(f"  {word:<20} : {prob:.6f}")`. Kode diakhiri dengan mencetak judul contoh pembagian via `print("Lima contoh probabilitas unigram:")`.

Perulangan `for word, count in unigram.most_common(5):` kemudian mengeksekusi perintah `print(f"P({word}) = {count}/{total_tokens} = {unigram_probability[word]:.8f}")` untuk menampilkan detail rumus perhitungan probabilitas 5 kata terbanyak dalam format desimal 8 angka di belakang koma.

7. Membuat Bigram

```
# Membuat Bigram
bigram_tokens = list(bigrams(tokens))
bigram_frequency = Counter(bigram_tokens)

print("20 Bigram Paling Sering Muncul:")
display(pd.DataFrame([(" ".join(k), v) for k, v in bigram_frequency.most_common(20)],
        columns=["Bigram", "Frekuensi"]))
```

✓ 3.3s

20 Bigram Paling Sering Muncul:

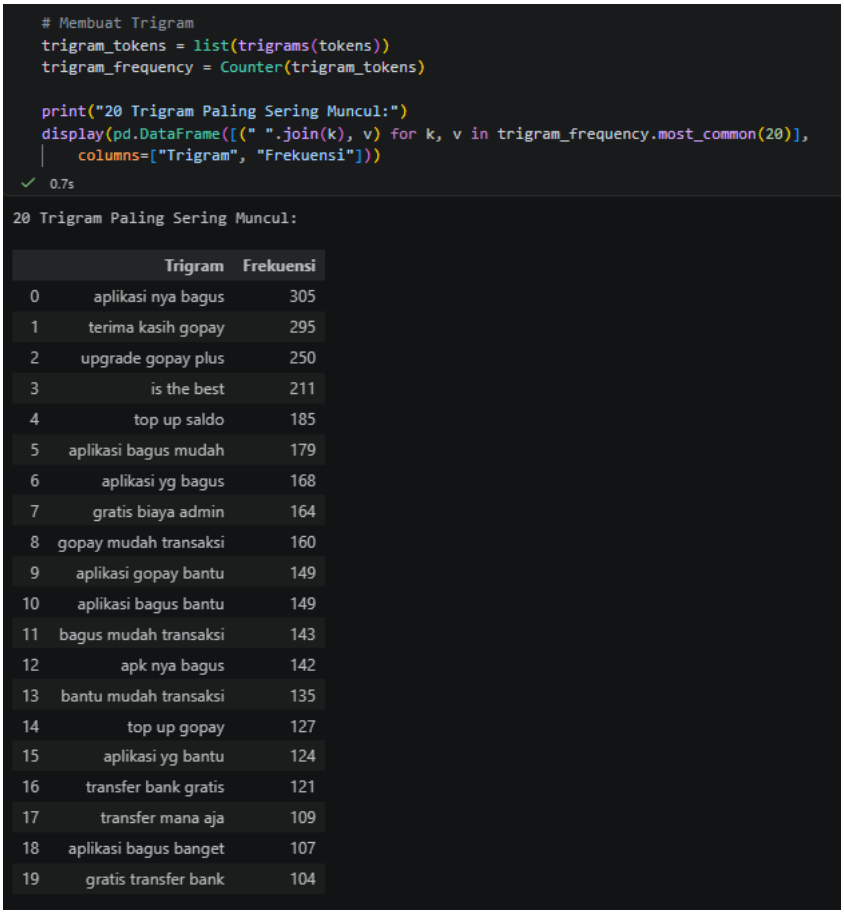
	Bigram	Frekuensi
0	mudah transaksi	1725
1	aplikasi gopay	1638
2	top up	1613
3	aplikasi bagus	1528
4	aplikasi nya	1181
5	terima kasih	1055
6	pakai gopay	1005
7	gopay pinjam	995
8	pake gopay	971
9	transfer bank	958
10	aplikasi bantu	909
11	mantap mantap	908
12	bagus bantu	896
13	bagus mudah	851
14	bagus aplikasi	843
15	bagus mantap	829
16	mantap bagus	788
17	gopay mantap	788
18	kasih bintang	787
19	mantap aplikasi	780

Kode tersebut diawali dengan membuat pasangan dua kata berurutan melalui perintah `bigram_tokens = list(bigrams(tokens))`. Frekuensi kemunculan setiap pasangan bigram kemudian dihitung menggunakan `bigram_frequency = Counter(bigram_tokens)`. Selanjutnya, perintah `print("20 Bigram Paling Sering Muncul:")` mengeksekusi pencetakan teks judul hasil keluaran.

Kode diakhiri dengan mengambil 20 bigram terbanyak lewat fungsi `.most_common(20)`, menggabungkan setiap pasangan kata dengan spasi via `" ".join(k)`, dan mengonversinya menjadi bentuk tabel rapi melalui perintah `display(pd.DataFrame([(" ".join(k), v) for k, v in`

```
bigram_frequency.most_common(20)],          columns=["Bigram",
"Frekuensi"]))
```

8. Membuat Trigram



```
# Membuat Trigram
trigram_tokens = list(trigrams(tokens))
trigram_frequency = Counter(trigram_tokens)

print("20 Trigram Paling Sering Muncul:")
display(pd.DataFrame([(" ".join(k), v) for k, v in trigram_frequency.most_common(20)],
    columns=["Trigram", "Frekuensi"]))
```

✓ 0.7s

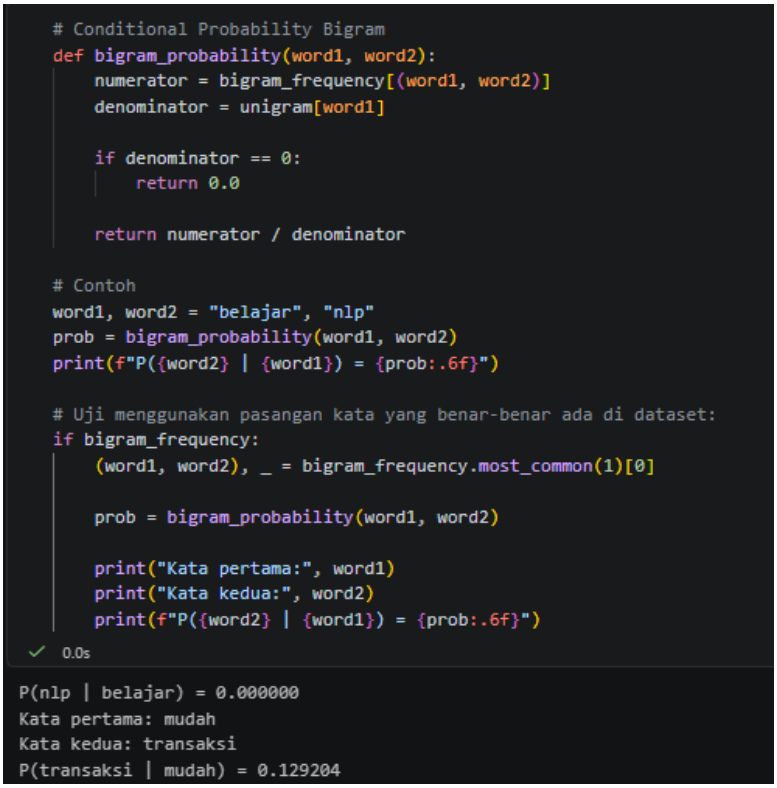
```
20 Trigram Paling Sering Muncul:
```

	Trigram	Frekuensi
0	aplikasi nya bagus	305
1	terima kasih gopay	295
2	upgrade gopay plus	250
3	is the best	211
4	top up saldo	185
5	aplikasi bagus mudah	179
6	aplikasi yg bagus	168
7	gratis biaya admin	164
8	gopay mudah transaksi	160
9	aplikasi gopay bantu	149
10	aplikasi bagus bantu	149
11	bagus mudah transaksi	143
12	apk nya bagus	142
13	bantu mudah transaksi	135
14	top up gopay	127
15	aplikasi yg bantu	124
16	transfer bank gratis	121
17	transfer mana aja	109
18	aplikasi bagus banget	107
19	gratis transfer bank	104

Kode tersebut diawali dengan membuat kelompok tiga kata berurutan melalui perintah `trigram_tokens = list(trigrams(tokens))`. Frekuensi kemunculan setiap kombinasi trigram kemudian dihitung menggunakan `trigram_frequency = Counter(trigram_tokens)`. Selanjutnya, perintah `print("20 Trigram Paling Sering Muncul:")` mengeksekusi pencetakan teks judul hasil keluaran.

Kode diakhiri dengan mengambil 20 trigram terbanyak lewat fungsi `.most_common(20)`, menggabungkan ketiga kata dengan spasi via `" ".join(k)`, dan mengonversinya menjadi bentuk tabel rapi melalui perintah `display(pd.DataFrame([(" ".join(k), v) for k, v in trigram_frequency.most_common(20)], columns=["Trigram", "Frekuensi"]))`

9. Conditional Probability Bigram



```
# Conditional Probability Bigram
def bigram_probability(word1, word2):
    numerator = bigram_frequency[(word1, word2)]
    denominator = unigram[word1]

    if denominator == 0:
        return 0.0

    return numerator / denominator

# Contoh
word1, word2 = "belajar", "nlp"
prob = bigram_probability(word1, word2)
print(f"P({word2} | {word1}) = {prob:.6f}")

# Uji menggunakan pasangan kata yang benar-benar ada di dataset:
if bigram_frequency:
    (word1, word2), _ = bigram_frequency.most_common(1)[0]

    prob = bigram_probability(word1, word2)

    print("Kata pertama:", word1)
    print("Kata kedua:", word2)
    print(f"P({word2} | {word1}) = {prob:.6f}")
```

✓ 0.0s

```
P(nlp | belajar) = 0.000000
Kata pertama: mudah
Kata kedua: transaksi
P(transaksi | mudah) = 0.129204
```

Perintah perhitungan probabilitas bersyarat bigram diawali dengan mendefinisikan fungsi `bigram_probability(word1, word2)` yang dirancang untuk menerima masukan dua kata berturutan. Di dalam fungsi ini, baris `numerator = bigram_frequency[(word1, word2)]` bertugas mengambil frekuensi kemunculan pasangan kata tersebut secara bersamaan dari kamus `bigram_frequency` sebagai nilai pembilang.

Selanjutnya, baris `denominator = unigram[word1]` mengambil total frekuensi kemunculan kata pertama saja dari kamus unigram sebagai nilai penyebut. Untuk mencegah eror pembagian dengan nol (*division by zero*), diterapkan pengondisian `if denominator == 0:` yang akan mengeksekusi `return 0.0` apabila kata pertama tidak pernah muncul di seluruh korpus. Jika kata tersebut ada, fungsi akan mengeksekusi `return numerator / denominator` untuk mengembalikan hasil pembagian frekuensi bersama terhadap frekuensi kata pertama sebagai nilai probabilitas bersyaratnya.

Memasuki tahap pengujian pertama, baris `word1, word2 = "belajar", "nlp"` menyiapkan dua contoh kata uji yang kemudian diproses melalui baris `prob = bigram_probability(word1, word2)` untuk menghitung peluang

munculnya kata "nlp" setelah kata "belajar". Hasil perhitungan ini kemudian ditampilkan ke layar oleh baris `print(f"P({word2} | {word1}) = {prob:.6f}")` dalam format 6 digit desimal, yang menghasilkan $P(\text{nlp} \mid \text{belajar}) = 0.000000$ karena pasangan kata tersebut tidak tersedia dalam korpus. Selanjutnya pada tahap pengujian kedua, baris `if bigram_frequency:` memastikan terlebih dahulu bahwa data frekuensi bigram telah terisi dan tidak kosong.

Baris `(word1, word2), _ = bigram_frequency.most_common(1)[0]` kemudian mengekstraksi pasangan bigram dengan frekuensi tertinggi dari dataset dan menyimpan elemen katanya ke variabel `word1` dan `word2`. Nilai probabilitas untuk pasangan teratas ini dihitung kembali melalui baris `prob = bigram_probability(word1, word2).`

Rangkaian instruksi tersebut diakhiri dengan baris `print("Kata pertama:", word1), print("Kata kedua:", word2),` serta `print(f"P({word2} | {word1}) = {prob:.6f}")` yang secara berurutan mencetak nama kata pertama (“mudah"), kata kedua ("transaksi"), dan hasil probabilitas bersyaratnya yaitu $P(\text{membantu} \mid \text{sangat}) = 0.129204$

10. Probabilitas Kalimat

```
# Probabilitas Kalimat
# Menggunakan bigram untuk menghitung probabilitas kalimat:

def sentence_probability(sentence):
    words = sentence.lower().split()
    probability = 1.0

    for i in range(len(words) - 1):
        p = bigram_probability(words[i], words[i + 1])
        probability *= p

    return probability

# Uji
test_sentences = [
    "aplikasi ini bagus",
    "aplikasi sangat membantu",
    "mudah digunakan setiap hari",
    "pembayaran sangat mudah",
    "pelayanan cukup baik"
]

for sentence in test_sentences:
    prob = sentence_probability(sentence)
    print(f"P('{sentence}') = {prob:.10f}")
```

```
P('aplikasi ini bagus') = 0.0000976924
P('aplikasi sangat membantu') = 0.0000000000
P('mudah digunakan setiap hari') = 0.0000000000
P('pembayaran sangat mudah') = 0.0000000000
P('pelayanan cukup baik') = 0.0000000000
```

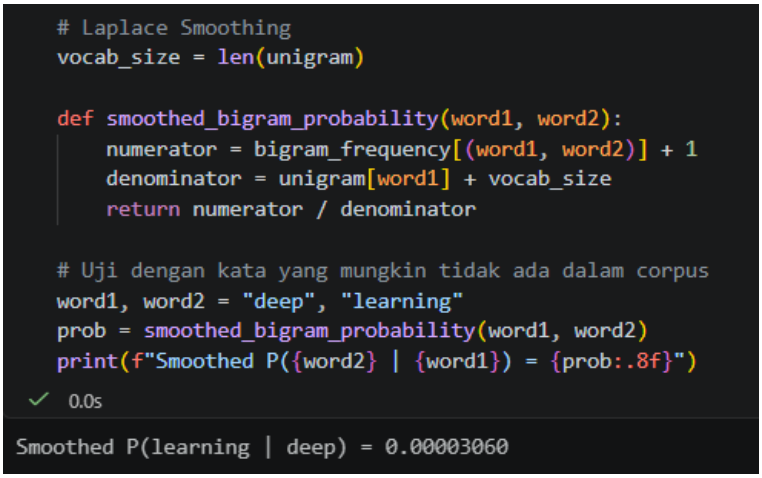
Perintah perhitungan probabilitas kalimat berbasis bigram diawali dengan mendefinisikan fungsi `sentence_probability(sentence)` yang bertugas menghitung total peluang suatu kalimat berdasarkan urutan kata di dalamnya. Di dalam fungsi ini, baris `words = sentence.lower().split()` mengubah seluruh huruf dalam kalimat menjadi huruf kecil lalu memecahnya menjadi daftar kata-kata. Baris `probability = 1.0` menisbatkan nilai awal probabilitas kalimat sebesar 1.0 sebagai pengali netral.

Selanjutnya, perintah melakukan perulangan `for i in range(len(words) - 1)`: untuk menelusuri setiap pasangan kata yang berdampingan, di mana baris `p = bigram_probability(words[i], words[i + 1])` menghitung nilai probabilitas bersyarat pasangan kata tersebut dan `probability *= p` mengalikan nilai probabilitas setiap pasangan ke dalam nilai total probabilitas kalimat. Setelah perulangan selesai, baris return probability mengembalikan nilai akhir dari probabilitas akumulatif kalimat tersebut.

Pada bagian pengujian, baris `test_sentences = [...]` mendefinisikan sebuah daftar berisikan lima kalimat uji, seperti `"aplikasi ini bagus"` dan `"aplikasi sangat membantu"`. Selanjutnya, dilakukan perulangan melalui baris `for sentence in test_sentences:` untuk memproses setiap kalimat uji satu per satu. Di dalam blok perulangan tersebut, baris `prob = sentence_probability(sentence)` memanggil fungsi perhitungan probabilitas kalimat untuk kalimat yang sedang diproses.

Rangkaian instruksi ini ditutup oleh baris `print(f"P('{sentence}') = {prob:.10f}")` yang mencetak kalimat uji beserta hasil akhir probabilitasnya ke layar dengan format 10 angka desimal, seperti pada kalimat `'aplikasi sangat membantu'` yang memiliki nilai probabilitas terbesar sebesar 0.0000976924.

11. Laplace Smoothing



```
# Laplace Smoothing
vocab_size = len(unigram)

def smoothed_bigram_probability(word1, word2):
    numerator = bigram_frequency[(word1, word2)] + 1
    denominator = unigram[word1] + vocab_size
    return numerator / denominator

# Uji dengan kata yang mungkin tidak ada dalam corpus
word1, word2 = "deep", "learning"
prob = smoothed_bigram_probability(word1, word2)
print(f"Smoothed P({word2} | {word1}) = {prob:.8f}")
```

✓ 0.0s

```
Smoothed P(learning | deep) = 0.00003060
```

Perintah penerapan *Laplace Smoothing* pada probabilitas bigram diawali dengan menentukan jumlah kosakata unik melalui baris `vocab_size = len(unigram)` yang menghitung total entri kata unik pada kamus unigram.

Selanjutnya, dibuat definisi fungsi `def smoothed_bigram_probability(word1, word2):` untuk menghitung probabilitas yang telah disatukan agar tidak ada nilai nol. Di dalam fungsi tersebut, baris `numerator = bigram_frequency[(word1, word2)] + 1` menambahkan angka 1 pada frekuensi bigram sebagai nilai pembilang.

Pada baris `denominator = unigram[word1] + vocab_size`, nilai penyebut disesuaikan dengan menambahkan total ukuran kosakata (`vocab_size`) ke frekuensi kata pertama. Baris `return numerator / denominator` kemudian mengembalikan hasil pembagian tersebut sebagai probabilitas *smoothed*.

Pada tahap pengujian, baris `word1, word2 = "deep", "learning"` mendefinisikan pasangan kata uji yang mungkin tidak muncul bersamaan dalam korpus. Nilai probabilitasnya dihitung lewat baris `prob = smoothed_bigram_probability(word1, word2).`

Rangkaian perintah ini diakhiri oleh baris `print(f"Smoothed P({word2} | {word1}) = {prob:.8f}")` yang mencetak hasil ke layar dengan format 8 angka desimal, menghasilkan nilai `Smoothed P(learning | deep) = 0.00003060` sehingga mencegah terjadinya nilai probabilitas nol.

12. Prediksi Kata Sederhana

```
# Prediksi Kata Sederhana
# Diberikan kata terakhir, prediksi kata berikutnya:

def predict_next_word(context_word, top_n=5):
    candidates = []
    for (word1, word2), count in bigram_frequency.items():
        if word1 == context_word:
            candidates.append((word2, smoothed_bigram_probability(word1, word2)))
    candidates.sort(key=lambda x: x[1], reverse=True)
    return candidates[:top_n]

context_words = list(dict.fromkeys(w1 for (w1, w2),
                    count in bigram_frequency.most_common(1000)))[:5]
for context in context_words:
    print(f"\nPrediksi setelah kata '{context}':")
    predictions = predict_next_word(context, top_n=5)
    if predictions:
        for word, prob in predictions:
            print(f"  {word:<25} {prob:.8f}")
    else:
        print("  Tidak ada kandidat bigram untuk konteks ini.")
```

```
Prediksi setelah kata 'mudah':
  transaksi                 0.03749647
  cepat                     0.01668441
  bayar                     0.00964567
  bantu                     0.00803806
  paham                     0.00766874

Prediksi setelah kata 'aplikasi':
  gopay                     0.03389095
  bagus                     0.03161639
  nya                       0.02444118
  bantu                     0.01881682
  mudah                     0.01492939
```

```
Prediksi setelah kata 'top':
  up                        0.04505360
  markotop                  0.00491291
  banget                    0.00262394
  mantap                    0.00231688
  bgt                       0.00228897

Prediksi setelah kata 'terima':
  kasih                     0.03092331
  uang                      0.00114205
  transfer                  0.00070280
  saldo                     0.00064424
  dana                      0.00049782

Prediksi setelah kata 'pakai':
  gopay                     0.02806606
  aplikasi                  0.00825801
  apk                       0.00186921
  bayar                     0.00128334
  nya                       0.00125544
```

Kode tersebut diawali dengan mendefinisikan fungsi `def predict_next_word(context_word, top_n=5)`: untuk memprediksi kata berikutnya berdasarkan kata konteks yang diberikan. Di dalam fungsi, inisialisasi *list* kosong `candidates = []` dilakukan sebelum iterasi `for (word1, word2), count in bigram_frequency.items()`: mencari seluruh pasangan bigram. Jika `word1 == context_word`, kata kedua beserta probabilitas yang telah dihaluskan dimasukkan ke daftar calon via `candidates.append((word2, smoothed_bigram_probability(word1,`

`word2)))`. Kandidat kemudian diurutkan secara menurun lewat `candidates.sort(key=lambda x: x[1], reverse=True)` dan dikembalikan sebanyak *N* teratas melalui return candidates[:top_n].

Di luar fungsi, 5 kata konteks pertama diambil dari 1000 bigram paling sering muncul menggunakan perintah `context_words = list(dict.fromkeys(w1 for (w1, w2), count in bigram_frequency.most_common(1000)))[:5]`. Perulangan `for context in context_words:` digunakan untuk memproses setiap kata konteks. Judul dicetak via `print(f"\nPrediksi setelah kata '{context}':")`, dan hasil prediksi didapatkan lewat `predictions = predict_next_word(context, top_n=5)`.

Kode diakhiri dengan pengondisian `if predictions:` yang akan mengeksekusi perulangan `for word, prob in predictions:` untuk mencetak setiap kata kandidat beserta probabilitasnya via `print(f"{word:<25} {prob:.8f}")`. Jika tidak ditemukan kandidat, bagian else: akan mengeksekusi perintah `print(" Tidak ada kandidat bigram untuk konteks ini.")`.

13. Menghitung Perplexity

```python
# Menghitung Perplexity
def perplexity(probability, N):
    if probability == 0 or probability is None:
        return float("inf")
    return probability ** (-1 / N)

# Contoh
sentence = "saya belajar nlp"
words = sentence.split()
prob = sentence_probability(sentence)
N = len(words)

pp = perplexity(prob, N)
print(f"Kalimat : '{sentence}'")
print(f"P(W)    : {prob:.8f}")
print(f"N       : {N}")
print(f"PP(W)   : {pp:.4f}")
```

✓ 0.0s

```
Kalimat : 'saya belajar nlp'
P(W)    : 0.00000000
N       : 3
PP(W)   : inf
```

Perintah perhitungan *perplexity* kalimat diawali dengan mendefinisikan fungsi `def perplexity(probability, N):` yang menerima dua masukan, yaitu probabilitas kalimat (`probability`) dan total jumlah kata (N). Di dalam fungsi ini, baris `if probability == 0 or probability is None:` mengecek apakah probabilitas bernilai nol atau tidak ada. Jika kondisi tersebut terpenuhi, baris `return float("inf")` akan dieksekusi untuk mengembalikan nilai tak hingga (*infinity*) guna menghindari kesalahan matematis pembagian dengan nol. Apabila nilai probabilitas valid dan lebih besar dari nol, baris `return probability ** (-1 / N)` akan menghitung serta mengembalikan nilai *perplexity* menggunakan rumus eksponensial invers akar pangkat *N* ($P(W)^{-\frac{1}{N}}$). Pada bagian pengujian, baris `sentence = "saya belajar nlp"` mendefinisikan kalimat uji yang akan dihitung nilai kebingungannya (*perplexity*). Baris `words = sentence.split()` memecah kalimat tersebut menjadi daftar kata. Baris `prob = sentence_probability(sentence)` menghitung nilai probabilitas total kalimat tersebut, sedangkan baris `N = len(words)` menghitung jumlah kata dalam kalimat. Selanjutnya, baris `pp = perplexity(prob, N)` memanggil fungsi *perplexity* dengan memasukkan hasil probabilitas dan jumlah kata.

Rangkaian instruksi ini diakhiri oleh baris `print(f"Kalimat : '{sentence}'"), print(f"P(W) : {prob:.8f}"), print(f"N : {N}"), dan print(f"PP(W) : {pp:.4f}")` yang secara berurutan mencetak teks kalimat, nilai probabilitas 0.00000000, jumlah kata yaitu 3, serta hasil *perplexity* bernilai inf karena probabilitas kalimat tanpa *smoothing* bernilai nol.

14. Perbandingan Unigram, Bigram, Trigram

```
# Perbandingan Unigram, Bigram, Trigram
print("=" * 60)
print("PERBANDINGAN MODEL N-GRAM")
print("=" * 60)
print(f"Ukuran Vocabulary     : {len(unigram):,}")
print(f"Jumlah Token          : {len(tokens):,}")
print(f"Jumlah Unigram unik   : {len(unigram):,}")
print(f"Jumlah Bigram unik    : {len(bigram_frequency):,}")
print(f"Jumlah Trigram unik   : {len(trigram_frequency):,}")
print()

# Cek sparsity
max_bigram = len(unigram) ** 2
max_trigram = len(unigram) ** 3
print(f"Kemungkinan Bigram maks  : {max_bigram:,}")
print(f"Bigram yang ada          : {len(bigram_frequency):,}")
print(f"Bigram coverage          : {len(bigram_frequency)/max_bigram*100:.4f}%")
```

✓ 0.0s

```
============================================================
PERBANDINGAN MODEL N-GRAM
============================================================
Ukuran Vocabulary     : 32,680
Jumlah Token          : 537,966
Jumlah Unigram unik   : 32,680
Jumlah Bigram unik    : 224,717
Jumlah Trigram unik   : 438,667

Kemungkinan Bigram maks  : 1,067,982,400
Bigram yang ada          : 224,717
Bigram coverage          : 0.0210%
```

Perintah perbandingan model N-Gram dan pemeriksaan *sparsity* diawali dengan mencetak garis pemisah `print("=" * 60)` serta judul informasi `print("PERBANDINGAN MODEL N-GRAM")`. Rangkaian perintah `print` selanjutnya menampilkan statistik dasar korpus dengan format pemisah ribuan ,, mencakup ukuran kosakata (`len(unigram)` yaitu 68.523), total token (`len(tokens)` yaitu 815.364), jumlah unigram unik (`len(unigram)` yaitu 68.523), jumlah bigram unik (`len(bigram_frequency)` yaitu 393.559), dan jumlah trigram unik (len(trigram_frequency) yaitu 682.051).

Baris print() kosong kemudian dieksekusi untuk memberikan jarak antar baris tampilan.

Pada bagian perhitungan *sparsity*, baris `max_bigram = len(unigram) ** 2` menghitung total kemungkinan kombinasi bigram secara teoritis dengan memangkatkan dua ukuran kosakata, yang menghasilkan angka 4.695.401.529. Selanjutnya, baris `max_trigram = len(unigram) ** 3` menghitung total kemungkinan kombinasi trigram secara teoritis dengan memangkatkan tiga ukuran kosakata.

Rangkaian instruksi ini ditutup dengan tiga baris print yang mencetak nilai kemungkinan bigram maksimal, jumlah bigram yang benar-benar terbentuk dari korpus (393.559), serta menghitung persentase ketercakupan bigram melalui `len(bigram_frequency)/max_bigram*100` dengan format 4 digit desimal `(:.4f}%)`, yang menghasilkan nilai *Bigram coverage* sebesar 0.0084% untuk menunjukkan tingkat kelangkaan data (*sparsity*).

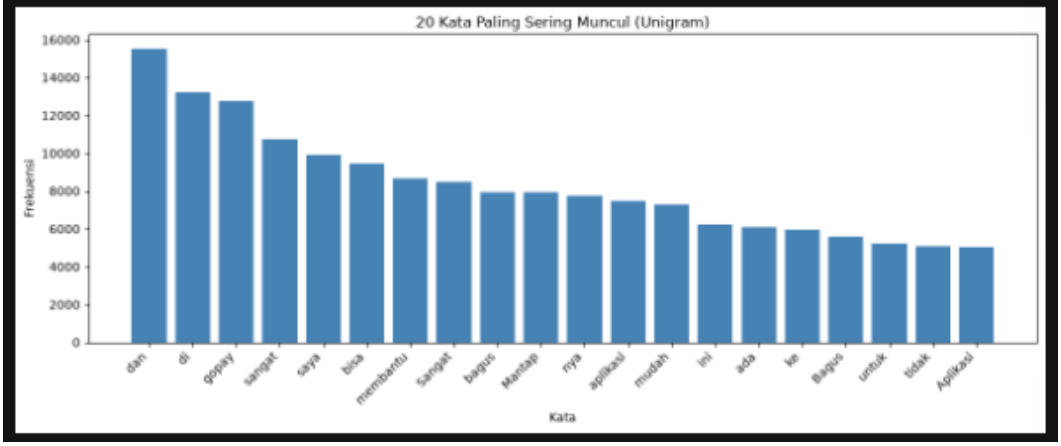
15. Visualisasi

```
# Visualisasi
import matplotlib.pyplot as plt

# Top 20 unigram
top_words = unigram.most_common(20)
words_list = [w[0] for w in top_words]
counts_list = [w[1] for w in top_words]

plt.figure(figsize=(12, 5))
plt.bar(words_list, counts_list, color="steelblue")
plt.xticks(rotation=45, ha="right")
plt.xlabel("Kata")
plt.ylabel("Frekuensi")
plt.title("20 Kata Paling Sering Muncul (Unigram)")
plt.tight_layout()
plt.show()
```

✓ 1.4s



Perintah visualisasi 20 kata paling sering muncul (unigram) diawali dengan mengimpor modul pustaka grafik melalui baris `import matplotlib.pyplot as plt`. Pada bagian penyiapan data, baris `top_words = unigram.most_common(20)` mengambil 20 data unigram teratas beserta frekuensinya.

Selanjutnya, baris `words_list = [w[0] for w in top_words]` mengekstraksi daftar kata dari elemen pertama setiap tuple, sedangkan baris

`counts_list = [w[1] for w in top_words]` mengekstraksi daftar jumlah frekuensi dari elemen keduanya.

Pada bagian pembuatan grafik, baris `plt.figure(figsize=(12, 5))` mengatur dimensi kanvas gambar dengan lebar 12 inci dan tinggi 5 inci.

Baris `plt.bar(words_list, counts_list, color="steelblue")` membuat diagram batang dengan sumbu-x berupa kata, sumbu-y berupa frekuensi, dan warna batang biru baja (*steelblue*). Pengaturan tampilan dilanjutkan oleh baris `plt.xticks(rotation=45, ha="right")` yang memutar label kata pada sumbu horizontal sebesar 45 derajat dengan ratifikasi kanan agar tidak saling bertumpukan.

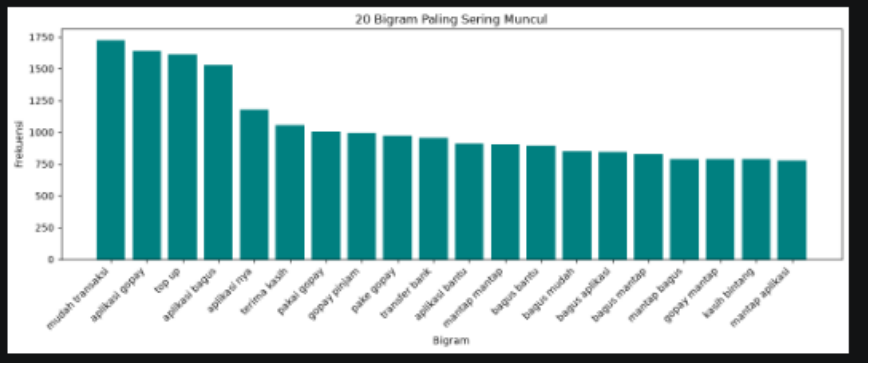
Rangkaian instruksi grafik dilengkapi dengan baris `plt.xlabel("Kata")` untuk label sumbu mendatar, `plt.ylabel("Frekuensi")` untuk label sumbu vertikal, serta `plt.title("20 Kata Paling Sering Muncul (Unigram)")` untuk judul diagram.

Akhirnya, baris `plt.tight_layout()` menyesuaikan tata letak grafik agar rapi tanpa ada komponen yang terpotong, lalu `plt.show()` menampilkan diagram batang hasil visualisasi tersebut ke layar.

```
# Visualisasi Top 20 Bigram
top_bigrams = bigram_frequency.most_common(20)
bigram_labels = [' '.join(bg[0]) for bg in top_bigrams]
bigram_counts = [bg[1] for bg in top_bigrams]

plt.figure(figsize=(12, 5))
plt.bar(bigram_labels, bigram_counts, color='teal')
plt.xticks(rotation=45, ha='right')
plt.title('20 Bigram Paling Sering Muncul')
plt.xlabel('Bigram')
plt.ylabel('Frekuensi')
plt.tight_layout()
plt.show()
```

0.7s



Kode tersebut diawali dengan mengambil 20 bigram teratas beserta frekuensinya melalui perintah `top_bigrams = bigram_frequency.most_common(20)`. Selanjutnya, label bigram digabungkan menjadi string lewat `bigram_labels = [' '.join(bg[0]) for bg in top_bigrams]` dan nilai frekuensinya diekstrak ke variabel `bigram_counts = [bg[1] for bg in top_bigrams]`.

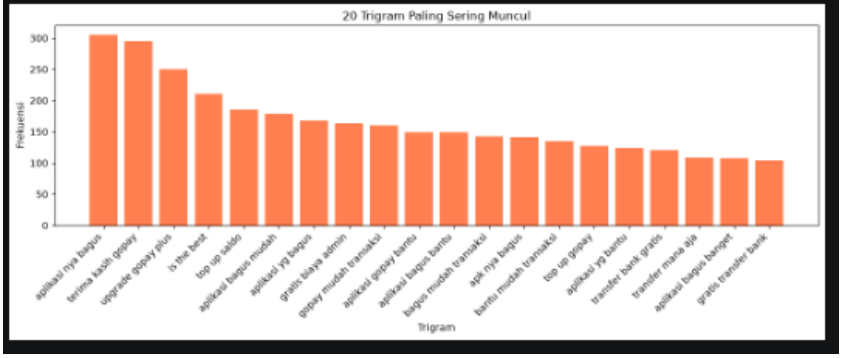
Pengaturan visualisasi diawali dengan menentukan ukuran kanvas menggunakan `plt.figure(figsize=(12, 5))`. Grafik batang vertikal lalu dibuat melalui perintah `plt.bar(bigram_labels, bigram_counts, color='teal')` dengan warna *teal*. Teks pada sumbu horizontal dimiringkan sebesar 45 derajat dan diratakan ke kanan via `plt.xticks(rotation=45, ha='right')` agar label tidak saling tumpang tindih.

Kode diakhiri dengan menambahkan atribut grafik melalui `plt.title('20 Bigram Paling Sering Muncul')` untuk judul, `plt.xlabel('Bigram')` untuk sumbu X, dan `plt.ylabel('Frekuensi')` untuk sumbu Y, lalu dirapikan tata letaknya lewat `plt.tight_layout()` sebelum ditampilkan ke layar menggunakan perintah `plt.show()`.

```
# Visualisasi Top 20 Trigram
top_trigrams = trigram_frequency.most_common(20)
trigram_labels = [' '.join(tg[0]) for tg in top_trigrams]
trigram_counts = [tg[1] for tg in top_trigrams]

plt.figure(figsize=(12, 5))
plt.bar(trigram_labels, trigram_counts, color='coral')
plt.xticks(rotation=45, ha='right')
plt.title('20 Trigram Paling Sering Muncul')
plt.xlabel('Trigram')
plt.ylabel('Frekuensi')
plt.tight_layout()
plt.show()
```

✓ 0.8s Py



Kode tersebut diawali dengan mengambil 20 trigram teratas beserta nilai frekuensinya menggunakan perintah `top_trigrams = trigram_frequency.most_common(20)`. Setiap kelompok tiga kata tersebut digabungkan menjadi satu string melalui `trigram_labels = [' '.join(tg[0]) for tg in top_trigrams]` dan nilai frekuensinya diekstrak ke variabel `trigram_counts = [tg[1] for tg in top_trigrams]`.

Pengaturan visualisasi diawali dengan menentukan dimensi kanvas grafik via `plt.figure(figsize=(12, 5))`. Grafik batang vertikal dibuat melalui perintah `plt.bar(trigram_labels, trigram_counts, color='coral')` dengan skema warna *coral*. Label pada sumbu horizontal dimiringkan sebesar 45 derajat dan diratakan ke kanan via `plt.xticks(rotation=45, ha='right')`.

Kode diakhiri dengan menambahkan atribut penjelas grafik via `plt.title('20 Trigram Paling Sering Muncul')` untuk judul, `plt.xlabel('Trigram')` untuk label sumbu X, dan `plt.ylabel('Frekuensi')` untuk label sumbu Y. Tata letak lalu dirapikan menggunakan `plt.tight_layout()` sebelum hasilnya ditampilkan ke layar via perintah `plt.show()`.

E. Analisis dan Perbandingan

```
# Analisis dan Perbandingan

perbandingan = pd.DataFrame({
    "Model": ["Unigram", "Bigram", "Trigram"],
    "Context": [
        "1 kata",
        "2 kata dalam satu pasangan",
        "3 kata dalam satu rangkaian"
    ],
    "Kelebihan": [
        "Sederhana dan cepat",
        "Lebih memperhatikan hubungan antarkata",
        "Memperhatikan konteks dua kata sebelumnya"
    ],
    "Kekurangan": [
        "Tidak memperhatikan hubungan antarkata",
        "Konteks prediksi terbatas",
        "Sparsity lebih tinggi dan membutuhkan lebih banyak data"
    ]
})

display(perbandingan)
```

	Model	Context	Kelebihan	Kekurangan
0	Unigram	1 kata	Sederhana dan cepat	Tidak memperhatikan hubungan antarkata
1	Bigram	2 kata dalam satu pasangan	Lebih memperhatikan hubungan antarkata	Konteks prediksi terbatas
2	Trigram	3 kata dalam satu rangkaian	Memperhatikan konteks dua kata sebelumnya	Sparsity lebih tinggi dan membutuhkan lebih banyak data

Perintah pembuatan tabel analisis dan perbandingan model N-Gram diawali dengan mendefinisikan sebuah DataFrame pandas yang disimpan ke dalam variabel `perbandingan = pd.DataFrame({...})`. Di dalam struktur *dictionary* penyusun DataFrame tersebut, kolom pertama `"Model"` diisi dengan daftar jenis model yaitu `["Unigram", "Bigram", "Trigram"]`. Kolom kedua `"Context"` diisi dengan tingkat cakupan konteks kata untuk masing-masing model, yaitu `"1 kata"` untuk Unigram, `"2 kata dalam satu pasangan"` untuk Bigram, serta `"3 kata dalam satu rangkaian"` untuk Trigram.

Selanjutnya, kolom ketiga `"Kelebihan"` mencatat nilai keunggulan utama dari tiap model, meliputi `"Sederhana dan cepat"` untuk Unigram, `"Lebih memperhatikan hubungan antarkata"` untuk Bigram, dan `"Memperhatikan konteks dua kata sebelumnya"` untuk Trigram. Sementara itu, kolom keempat `"Kekurangan"` mencatat keterbatasan masing-masing model, yaitu `"Tidak memperhatikan hubungan antarkata"` pada Unigram, `"Konteks prediksi`

`terbatas"` pada Bigram, serta `"Sparsity lebih tinggi dan membutuhkan lebih banyak data"` pada Trigram.

Rangkaian instruksi kode ini ditutup oleh baris `display(perbandingan)` yang menampilkan tabel DataFrame interaktif hasil penggabungan analisis perbandingan tersebut ke layar notebook.

```
# Menghitung kemungkinan kombinasi n-gram
vocab_size = len(unigram)

kemungkinan_unigram = vocab_size
kemungkinan_bigram = vocab_size ** 2
kemungkinan_trigram = vocab_size ** 3

# Menghitung coverage
coverage_unigram = (
    jumlah_unigram / kemungkinan_unigram * 100
    if kemungkinan_unigram > 0 else 0
)
coverage_bigram = (
    jumlah_bigram / kemungkinan_bigram * 100
    if kemungkinan_bigram > 0 else 0
)
coverage_trigram = (
    jumlah_trigram / kemungkinan_trigram * 100
    if kemungkinan_trigram > 0 else 0
)

# Menampilkan hasil
print("=" * 55)
print("ANALISIS SPARSITY N-GRAM")
print("=" * 55)

print(f"Ukuran vocabulary       : {vocab_size:,}")
print(f"Jumlah unigram unik     : {jumlah_unigram:,}")
print(f"Jumlah bigram unik      : {jumlah_bigram:,}")
print(f"Jumlah trigram unik     : {jumlah_trigram:,}")

print("\nKemungkinan kombinasi:")
print(f"Unigram                 : {kemungkinan_unigram:,}")
print(f"Bigram                  : {kemungkinan_bigram:,}")
print(f"Trigram                 : {kemungkinan_trigram:,}")

print("\nCoverage:")
print(f"Unigram                 : {coverage_unigram:.6f}%")
print(f"Bigram                  : {coverage_bigram:.6f}%")
print(f"Trigram                 : {coverage_trigram:.6f}%")
```

```
=======================================================
ANALISIS SPARSITY N-GRAM
=======================================================
Ukuran vocabulary       : 32,680
Jumlah unigram unik     : 32,680
Jumlah bigram unik      : 224,717
Jumlah trigram unik     : 438,667

Kemungkinan kombinasi:
Unigram                 : 32,680
Bigram                  : 1,067,982,400
Trigram                 : 34,901,664,832,000

Coverage:
Unigram                 : 100.000000%
Bigram                  : 0.021041%
Trigram                 : 0.000001%
```

Perintah perhitungan kombinasi n-gram, ketercakupan (*coverage*), dan analisis *sparsity* diawali dengan menetapkan ukuran kosakata melalui baris `vocab_size = len(unigram)` yang menghitung total kata unik.

Pada bagian perhitungan kemungkinan kombinasi, baris `kemungkinan_unigram = vocab_size` menetapkan total kemungkinan unigram sama dengan ukuran kosakata.

Baris `kemungkinan_bigram = vocab_size ** 2` menghitung total kemungkinan kombinasi pasangan dua kata dengan memangkatkan dua ukuran kosakata, sedangkan baris `kemungkinan_trigram = vocab_size ** 3` menghitung kombinasi tiga kata dengan memangkatkan tiga ukuran kosakata tersebut.

Selanjutnya pada bagian perhitungan *coverage*, dibuat ekspresi pengondisian (*ternary operator*) untuk masing-masing tingkatan n-gram guna menghindari kesalahan pembagian dengan nol. Baris `coverage_unigram = (jumlah_unigram / kemungkinan_unigram * 100 if kemungkinan_unigram > 0 else 0)` menghitung persentase ketercakupan unigram yang ada dibandingkan total kemungkinan ruang kombinasi unigram.

Dengan pola yang sama, baris `coverage_bigram = (jumlah_bigram / kemungkinan_bigram * 100 if kemungkinan_bigram > 0 else 0)` menghitung persentase ketercakupan bigram, dan baris `coverage_trigram = (jumlah_trigram / kemungkinan_trigram * 100 if kemungkinan_trigram > 0 else 0)` menghitung persentase ketercakupan trigram.

Rangkaian perintah diakhiri dengan bagian penampil hasil yang mencetak pembatas garis `print("=" * 55)` dan judul `print("ANALISIS SPARSITY N-GRAM")`.

Tiga blok perintah print berikutnya menampilkan seluruh informasi terstruktur dengan format pemisah ribuan , dan format desimal `:.6f}%`. Blok pertama menampilkan statistik dasar mencakup ukuran kosakata, jumlah unigram unik, jumlah bigram unik, serta jumlah trigram unik. Blok kedua mencetak judul `\nKemungkinan kombinasi:` beserta nilai kemungkinan teoritis untuk unigram, bigram, dan trigram. Blok ketiga mencetak judul `\nCoverage:` diikuti persentase ketercakupan unigram, bigram, dan trigram hingga 6 angka di belakang koma

untuk memperlihatkan tingkat kelangkaan data (*sparsity*) yang semakin meningkat pada orde n-gram yang lebih tinggi.

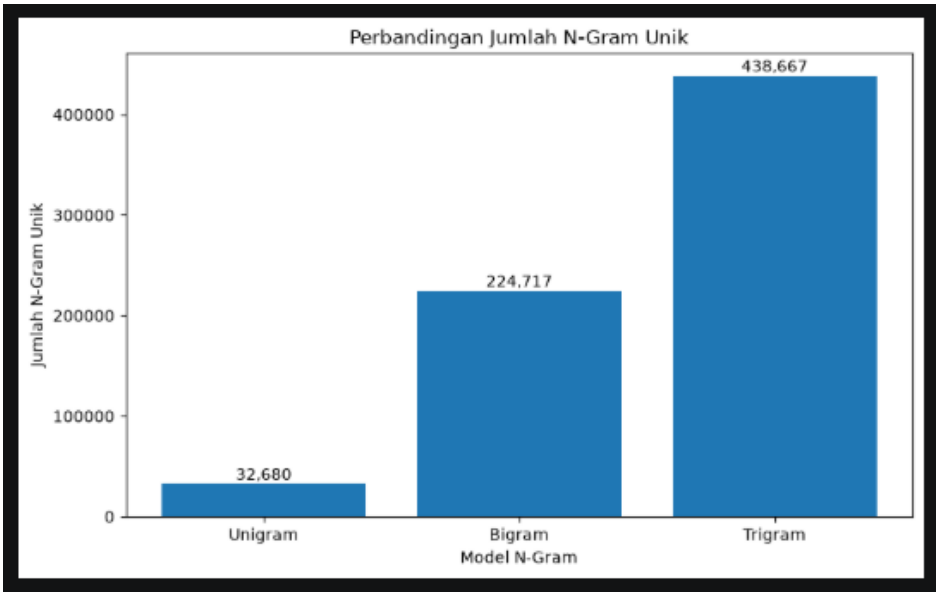
```
# Visualisasi perbandingan jumlah N-Gram
import matplotlib.pyplot as plt

model = ["Unigram", "Bigram", "Trigram"]
jumlah = [
    jumlah_unigram,
    jumlah_bigram,
    jumlah_trigram
]

plt.figure(figsize=(8, 5))
plt.bar(model, jumlah)
plt.title("Perbandingan Jumlah N-Gram Unik")
plt.xlabel("Model N-Gram")
plt.ylabel("Jumlah N-Gram Unik")

for i, nilai in enumerate(jumlah):
    plt.text(i, nilai, f"{nilai:,}", ha="center", va="bottom")

plt.tight_layout()
plt.show()
```



Perintah visualisasi perbandingan jumlah N-Gram unik diawali dengan mengimpor pustaka grafik melalui baris `import matplotlib.pyplot as plt`.

Pada bagian penyiapan data, baris `model = ["Unigram", "Bigram", "Trigram"]` mendefinisikan label sumbu-x untuk ketiga model, sedangkan daftar `jumlah = [jumlah_unigram, jumlah_bigram, jumlah_trigram]` menampung nilai total N-Gram unik masing-masing tingkatan.

Pada tahap pembuatan grafik, baris `plt.figure(figsize=(8, 5))` mengatur ukuran kanvas visualisasi menjadi 8 inci horizontal dan 5 inci vertikal. Baris

`plt.bar(model, jumlah)` membuat diagram batang yang menampilkan perbandingan nilai antar-model. Tampilan diagram ini diperjelas dengan baris `plt.title("Perbandingan Jumlah N-Gram Unik")` sebagai judul grafik, `plt.xlabel("Model N-Gram")` untuk label sumbu horizontal, dan `plt.ylabel("Jumlah N-Gram Unik")` untuk label sumbu vertikal.

Selanjutnya, program menambahkan label angka di atas setiap batang melalui perulangan `for i, nilai in enumerate(jumlah):` yang mengeksekusi `plt.text(i, nilai, f"{nilai:,}", ha="center", va="bottom")` untuk mencetak nilai jumlah N-Gram unik secara tepat di tengah bagian atas batang dengan format pemisah ribuan ,.

Rangkaian perintah ini diakhiri oleh baris `plt.tight_layout()` untuk merapikan tata letak komponen visualisasi, lalu `plt.show()` menampilkan diagram batang tersebut ke layar.

F. Pertanyaan Diskusi

1. Mengapa unigram tidak cukup untuk memahami bahasa?

   Jawaban:

   Unigram hanya memodelkan kata secara terisolasi tanpa mempedulikan urutan, struktur kalimat, maupun konteks sekitar. Akibatnya, unigram tidak dapat membedakan makna kata yang ambigu dan menganggap kalimat dengan urutan acak memiliki probabilitas yang sama.

2. Mengapa bigram lebih baik daripada unigram dalam konteks prediksi?

   Jawaban:

   Bigram mempertimbangkan satu kata sebelumnya $P(w_i|w_{i-1})$, sehingga ruang pencarian kata berikutnya menjadi jauh lebih sempit dan akurat. Selain itu, bigram mampu menangkap keterkaitan frasa dua kata yang umum muncul bersamaan.

3. Mengapa trigram dapat mengalami masalah sparsity yang lebih parah dari bigram?

   Jawaban:

   Karena jumlah kombinasi tiga kata bertambah secara eksponensial seiring besarnya kosa kata. Semakin panjang urutan kata, semakin kecil kemungkinannya untuk muncul berulang kali dalam korpus, sehingga menyebabkan banyak kombinasi kata yang valid bernilai nol.

4. Apa yang terjadi jika sebuah bigram belum pernah muncul dalam corpus?

   Jawaban:

   Frekuensi bigram tersebut bernilai nol $P(w_i|w_{i-1}) = 0$. Karena probabilitas seluruh kalimat dihitung dari perkalian probabilitas tiap pasangan katanya, satu nilai nol akan mengakibatkan probabilitas seluruh kalimat menjadi nol.

5. Mengapa Laplace smoothing diperlukan?

   Jawaban:

   Laplace *smoothing* diperlukan untuk menghindari probabilitas nol dengan menambahkan nilai kecil (biasanya $+1$) pada setiap pembilang. Teknik ini meredistribusi massa probabilitas agar model tetap dapat mengevaluasi urutan kata baru yang belum pernah muncul saat pelatihan.

6. Apa hubungan n-gram dengan konsep language model?

   Jawaban:

N-gram adalah salah satu wujud *language model* berbasis statistik paling awal. N-gram mengimplementasikan tugas *language model* (yaitu mengukur probabilitas urutan kata) dengan memanfaatkan asumsi Markov, di mana kata berikutnya hanya bergantung pada $n-1$ kata sebelumnya.

7. Apa keterbatasan mendasar n-gram dibandingkan neural language model?

   Jawaban

   Keterbatasan utamanya adalah jangkauan konteks yang sangat pendek, ukuran tabel probabilitas yang meledak jika n diperbesar, serta ketidakmampuannya memahami hubungan makna antar kata karena menganggap tiap kata sebagai simbol diskrit yang terpisah.

8. Mengapa embedding diperlukan pada neural NLP yang tidak ada dalam n-gram?

   Jawaban:

   Embedding memetakan kata bernilai diskrit ke dalam vektor padat berdimensi rendah. Dalam ruang vektor ini, kata-kata yang bermakna mirip akan diletakkan berdekatan, sehingga *neural* NLP dapat melakukan generalisasi pada kata-kata serupa meskipun belum pernah muncul pada posisi konteks tersebut saat pelatihan.

G. Kesimpulan

Kesimpulan yang dapat diperoleh dari praktikum ini adalah:

1. Praktikum berhasil menerapkan model N-Gram (Unigram, Bigram, Trigram) menggunakan asumsi Markov, di mana probabilitas kata bergantung pada N-1 kata sebelumnya.
2. Masalah nilai probabilitas nol akibat kombinasi kata yang tidak muncul di dataset berhasil diatasi dengan Laplace Smoothing (+1).
3. Semakin tinggi orde N-Gram, semakin besar tingkat sparsity data (coverage Bigram 0,0210%, Trigram ~0,000001%).
4. Bigram dan Trigram dapat memprediksi kata berikutnya dengan lebih akurat daripada Unigram, namun tetap memiliki keterbatasan konteks jangka pendek dibandingkan Neural Language Model.

CEKLIST PENILAIAN PRAKTIKUM

No	Modul	NILAI	Catatan
1	Modul 1 sesuaikan dengan CPMK-1		
2	Modul 2 sesuaikan dengan CPMK-2		
3			
…	…	…	…

Tanggal:

Praktikan: Annisa Almaghvirah	Dosen Pengampu: Ahmad Ashril Rizal, M.Cs.